



Chapter 1

Introduction to Visual C#

By

Dr. Sara Tarik

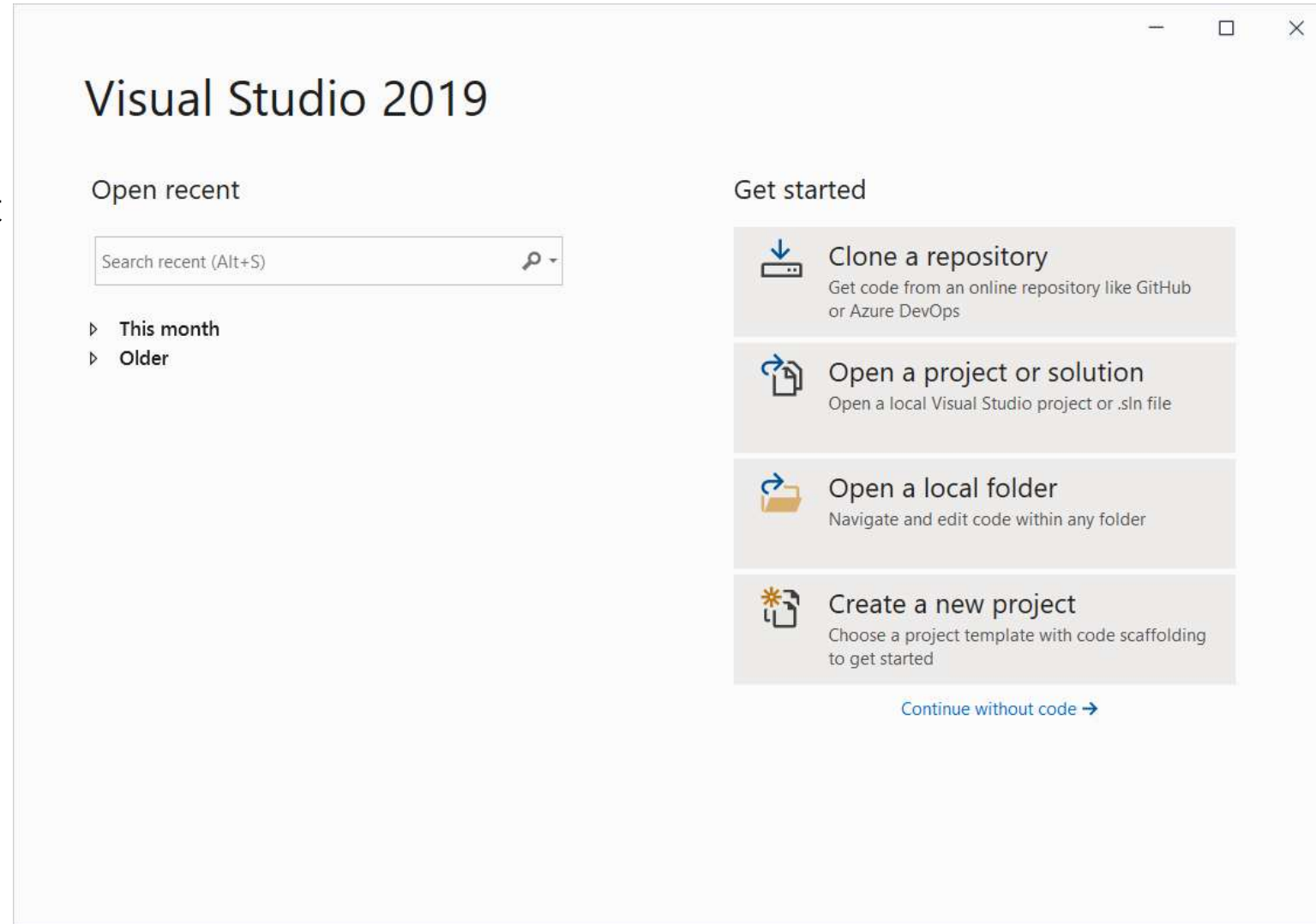
Getting Started with Visual Studio

- Visual Studio provides a collection of tools that you use to build Visual C# applications.
- Visual Studio is a professional integrated development environment (IDE), which means that it provides all the necessary tools for creating, testing, and debugging software.
- It can be used to create applications not only with Visual C#, but also with other languages such as Visual Basic and C++.

Creating a New Project

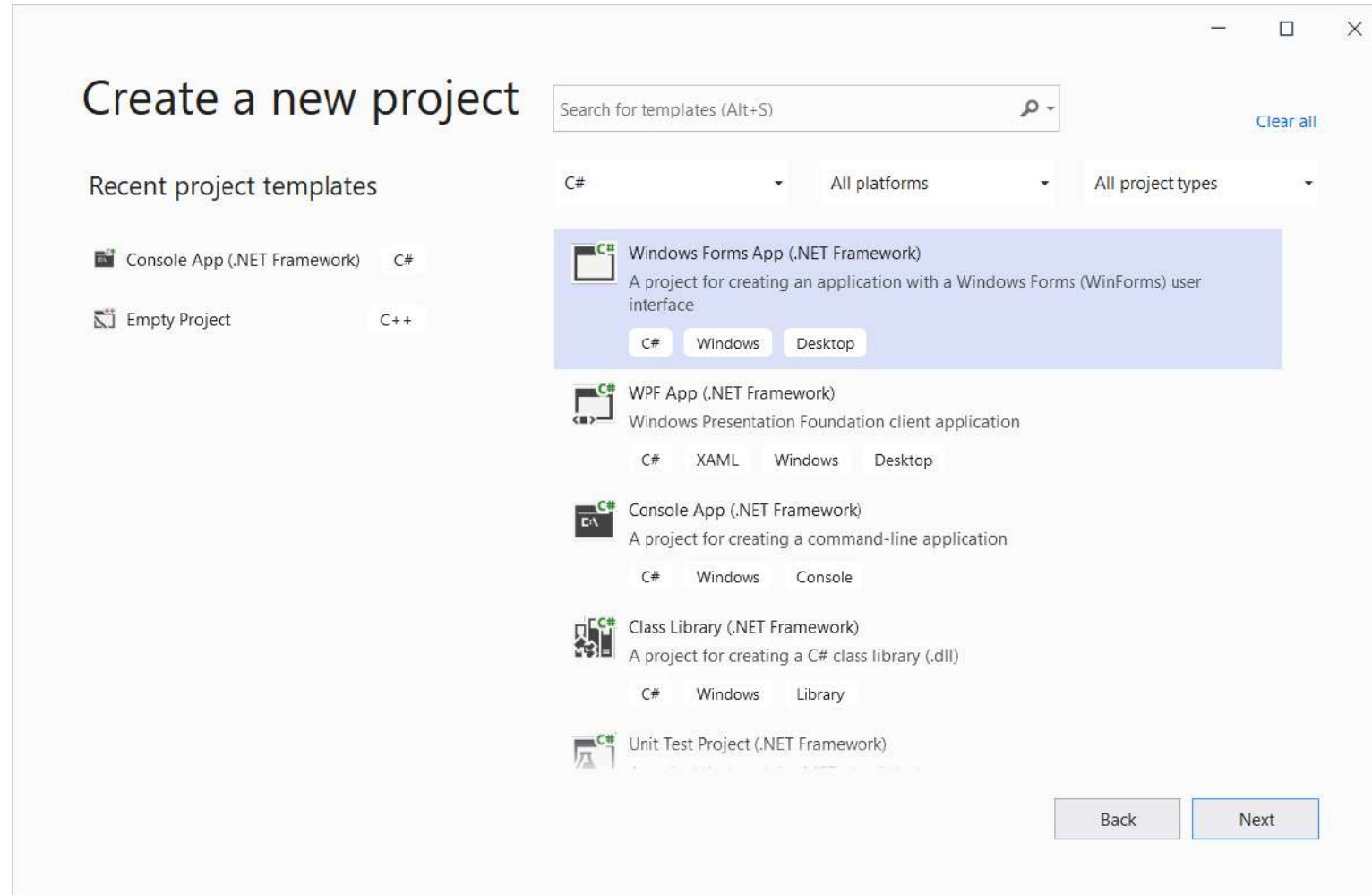
- To create a new project in Visual Studio, follow these steps:

1) Open Visual Studio. From the Start Window, select Create a new project.



Creating a New Project

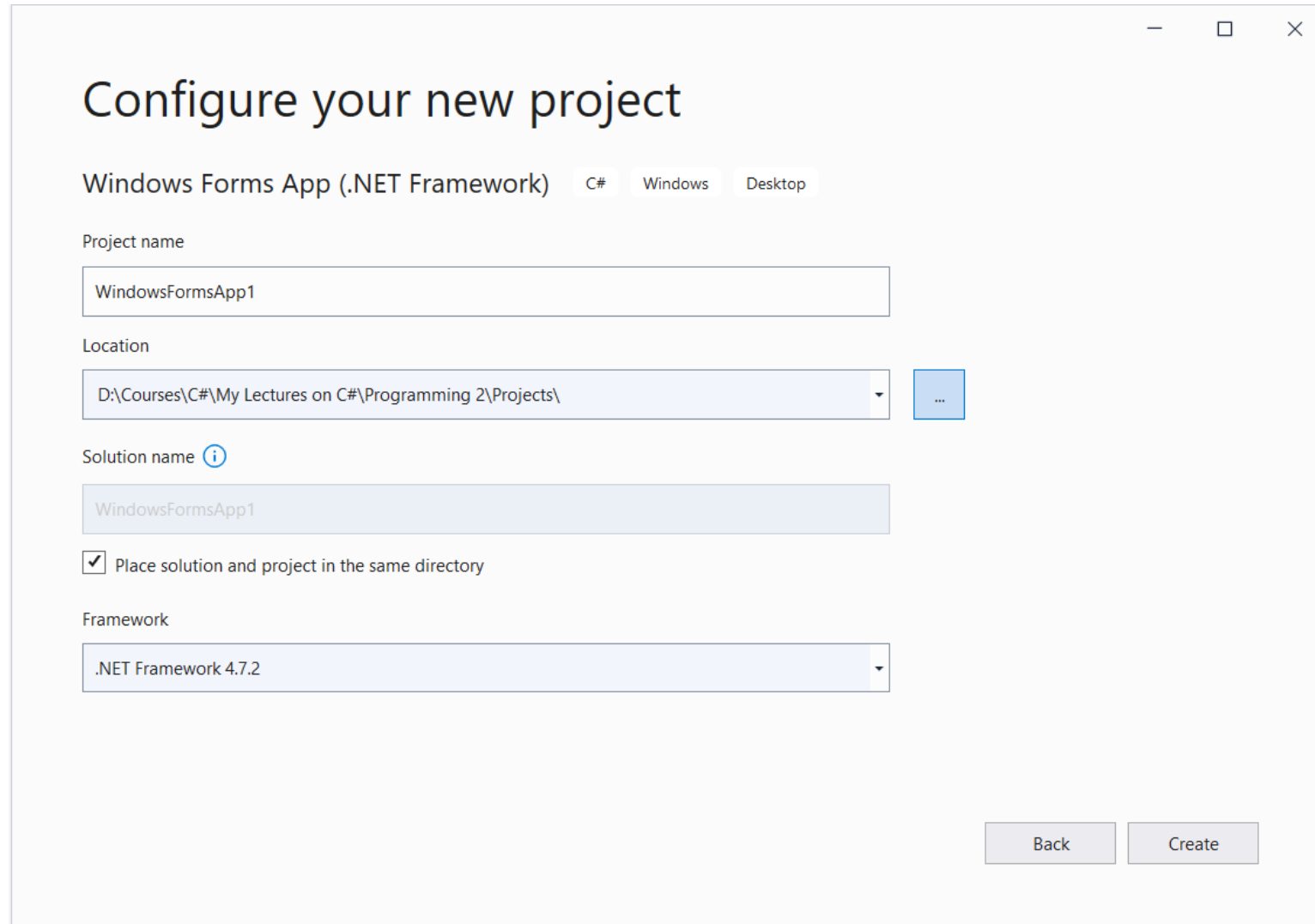
- 2) Select Windows Forms Application and make sure Visual C# is selected. Then, click Next.



Creating a New Project

3) The project name text box will be automatically filled in with a default name WindowsFormsApp1.

- In Project Name: Enter a name for your project (e.g., MyFirstProject).
- In Location: Choose where to save the project.
- Keep the solution name the same as the project name for all the projects that you create.

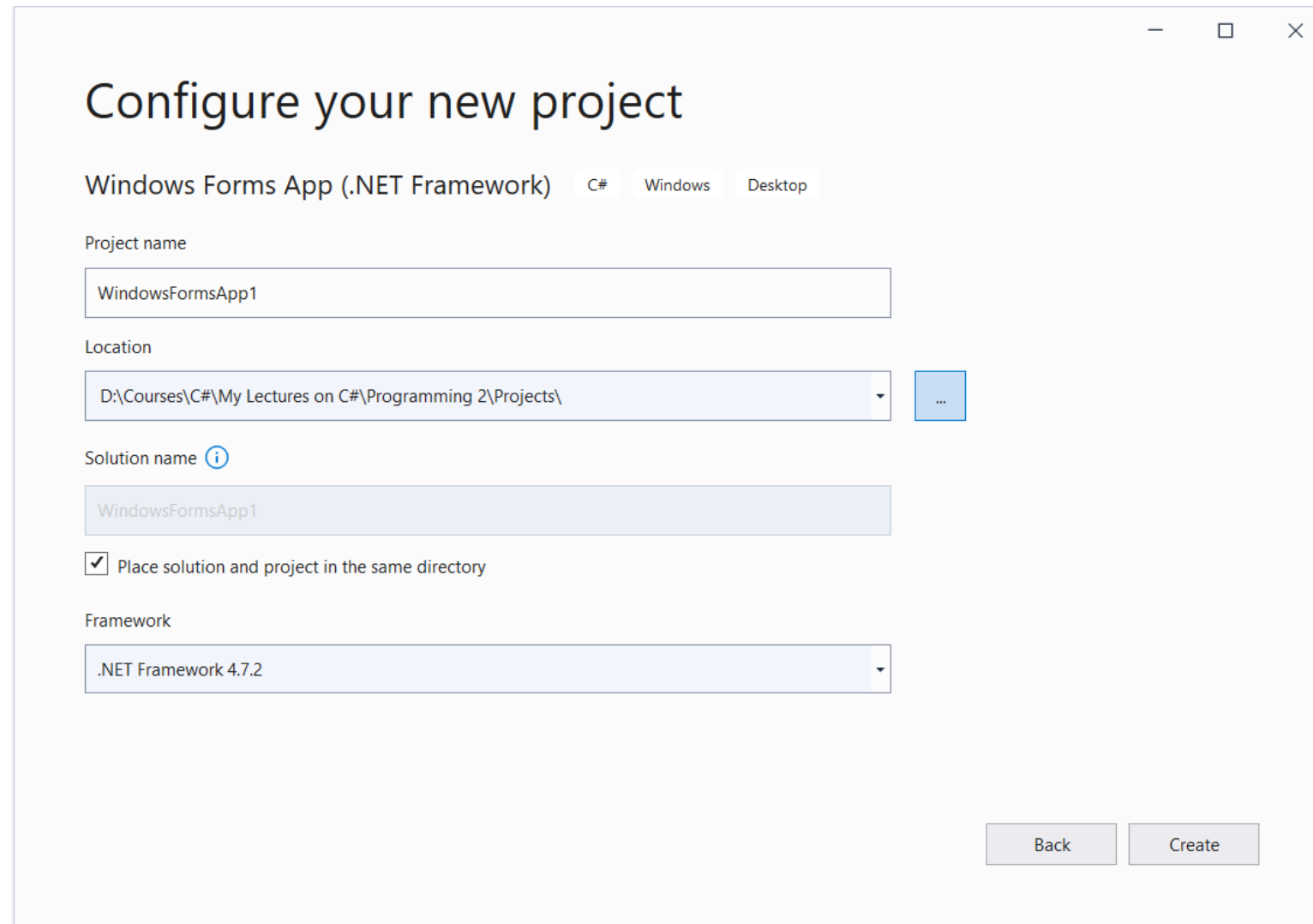


The screenshot shows the 'Configure your new project' dialog box. At the top, it says 'Configure your new project'. Below that, there are three tabs: 'Windows Forms App (.NET Framework)', 'C#', and 'Desktop'. The 'Project name' field is filled with 'WindowsFormsApp1'. The 'Location' field is filled with 'D:\Courses\C#\My Lectures on C#\Programming 2\Projects\'. The 'Solution name' field is filled with 'WindowsFormsApp1'. There is a checkbox labeled 'Place solution and project in the same directory' which is checked. The 'Framework' dropdown is set to '.NET Framework 4.7.2'. At the bottom right, there are 'Back' and 'Create' buttons.

Creating a New Project

A solution is a container that holds a project, and the Solution name text box shows the name of the solution that will hold this project. By default, the solution name is the same as the project name.

4) Click Create.



Configure your new project

Windows Forms App (.NET Framework) C# Windows Desktop

Project name

WindowsFormsApp1

Location

D:\Courses\C#\My Lectures on C#\Programming 2\Projects\

Solution name ⓘ

WindowsFormsApp1

☒ Place solution and project in the same directory

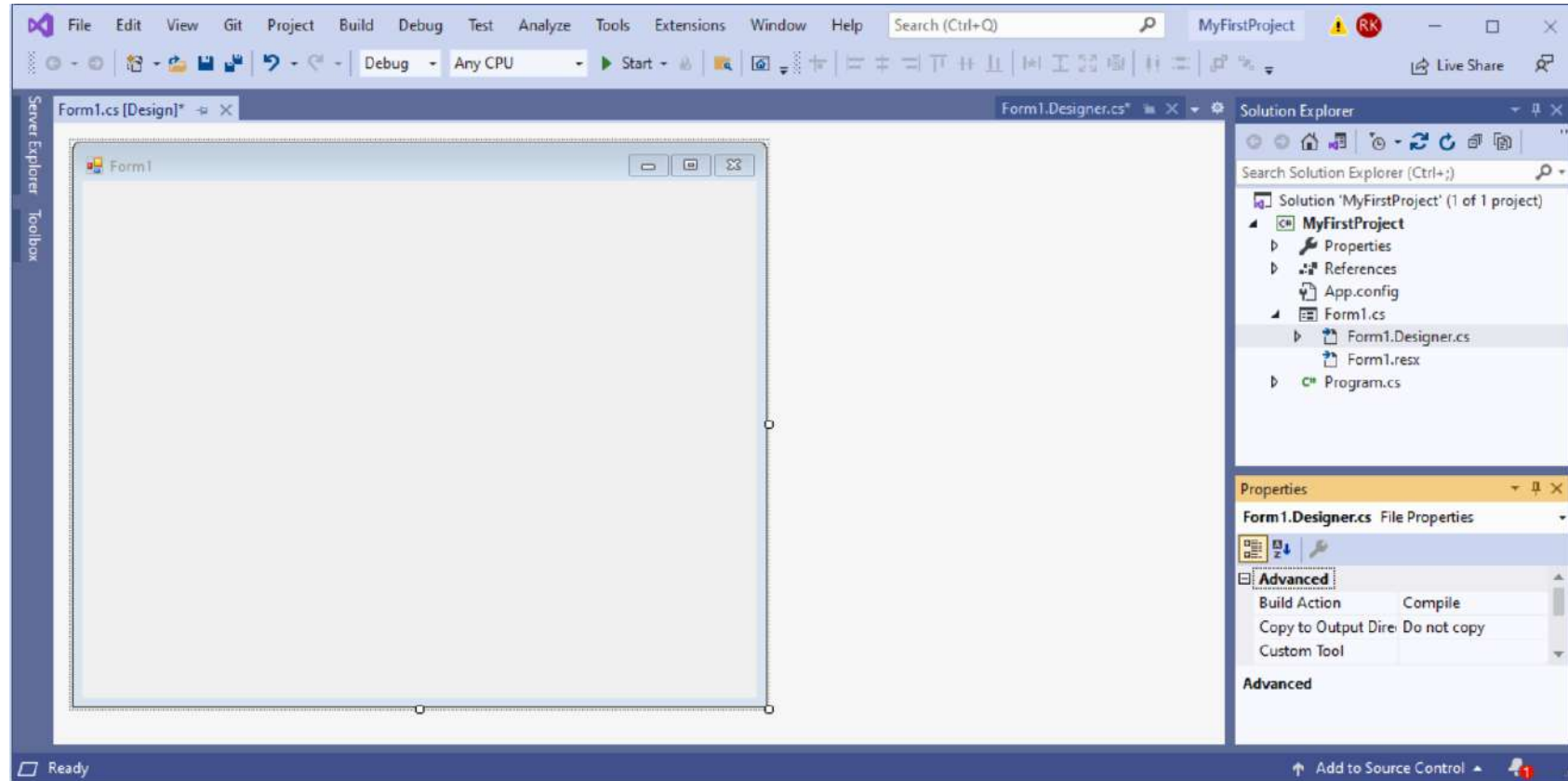
Framework

.NET Framework 4.7.2

Back Create

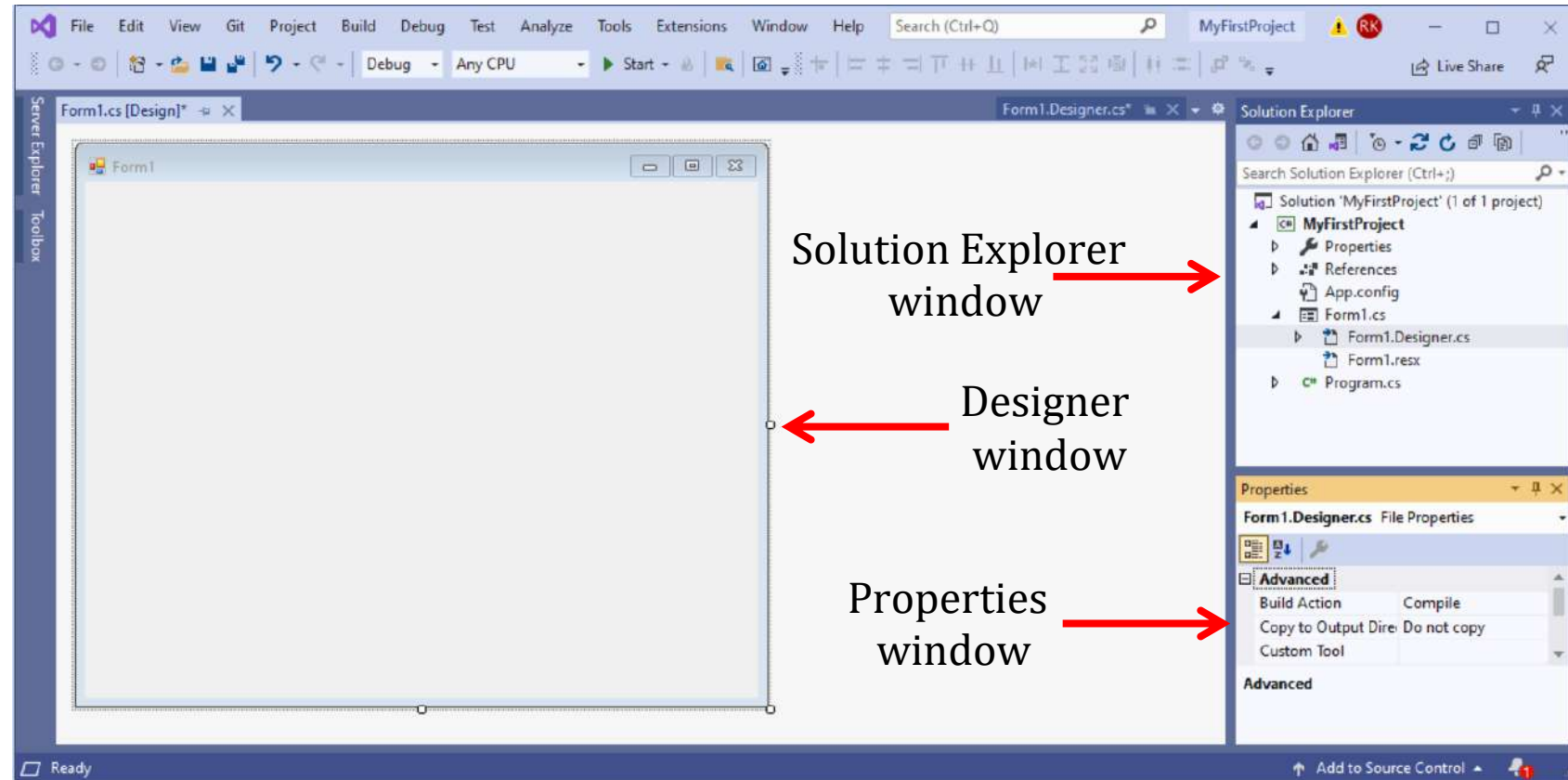
Creating a New Project

- Once you create a new project, the Visual Studio environment should appear, as shown in the figure.
- Notice that the name of the project “MyFirstProject” is displayed in the title bar at the top of the Visual Studio window.



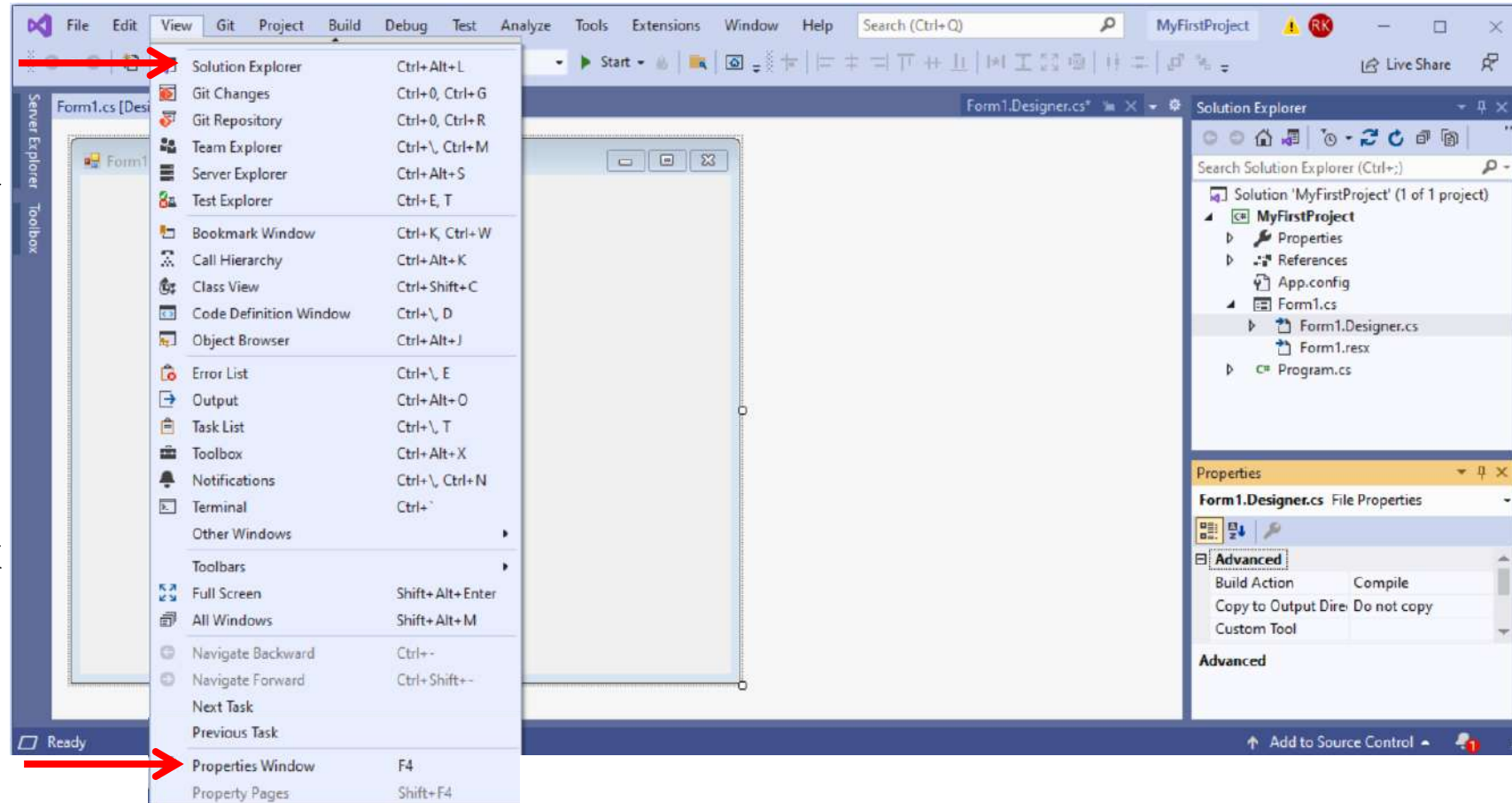
Visual Studio Environment

- The Visual Studio environment consists of a number of windows that you will use on a regular basis.
- This figure shows the locations of the following windows that appear within the Visual Studio environment:
 - Designer window.
 - Solution Explorer window.
 - Properties window.



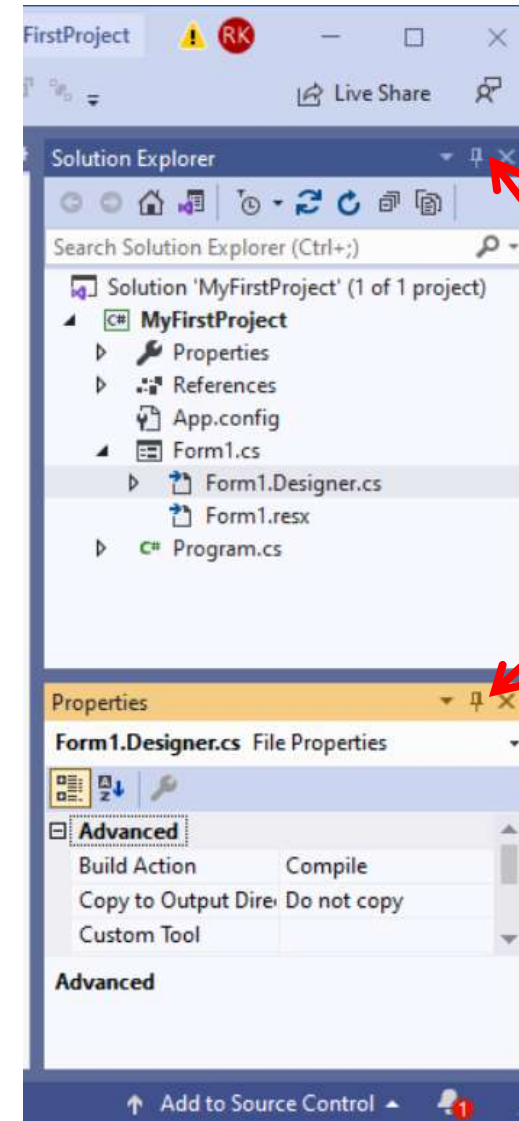
Visual Studio Environment

- If you do not see the Solution Explorer window, click View on the menu bar. On the View menu, click Solution Explorer.
- If you do not see the Properties window, click View on the menu bar. On the View menu, click Properties.



Using Auto Hide

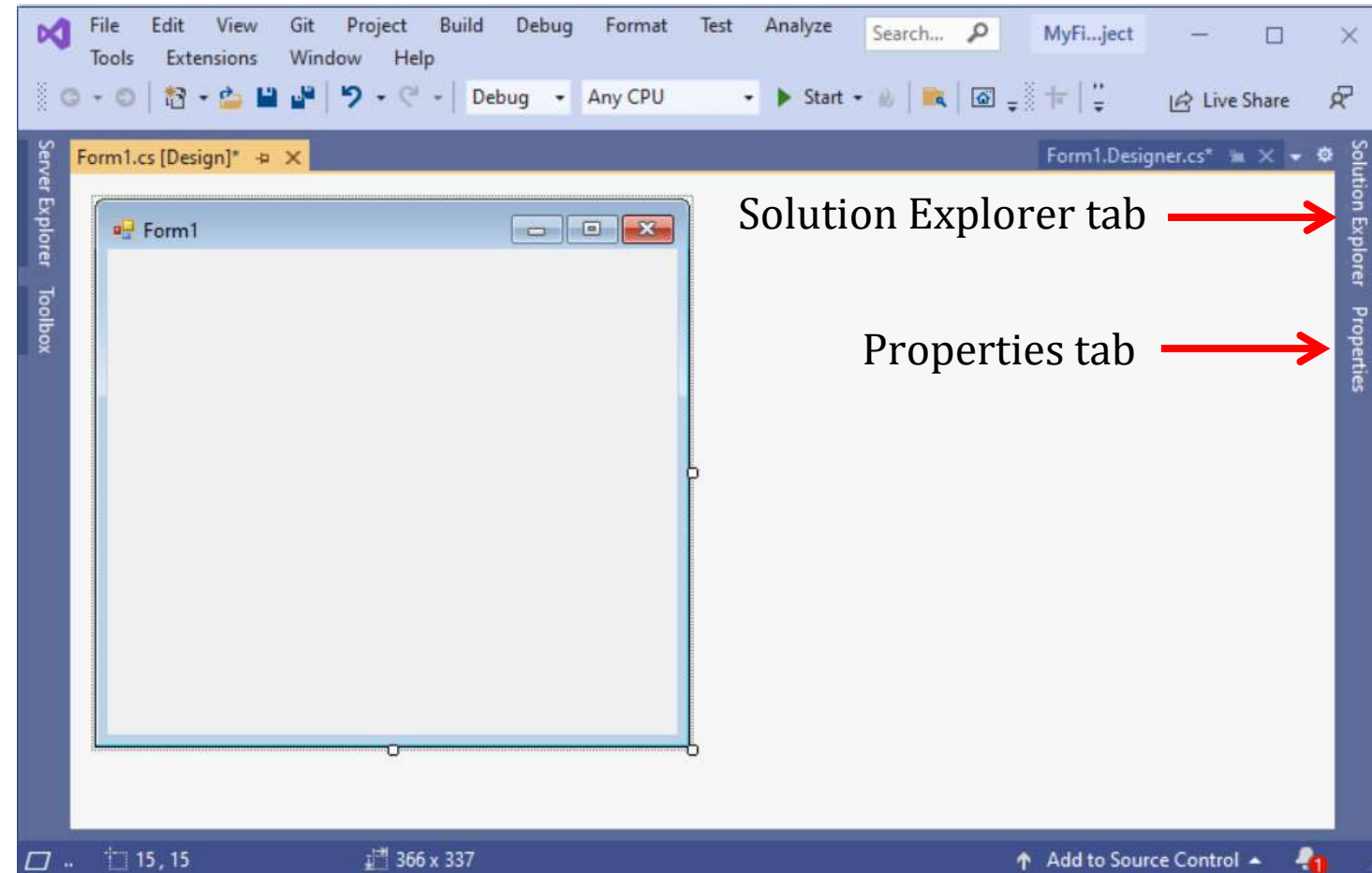
- Many windows in Visual Studio have a feature known as Auto Hide.
- When you see the pushpin icon in a window's title bar, as shown in the figure, you know that the window has Auto Hide capability.
- Click the pushpin icon to turn Auto Hide on or off for a window.



Pushpin icon

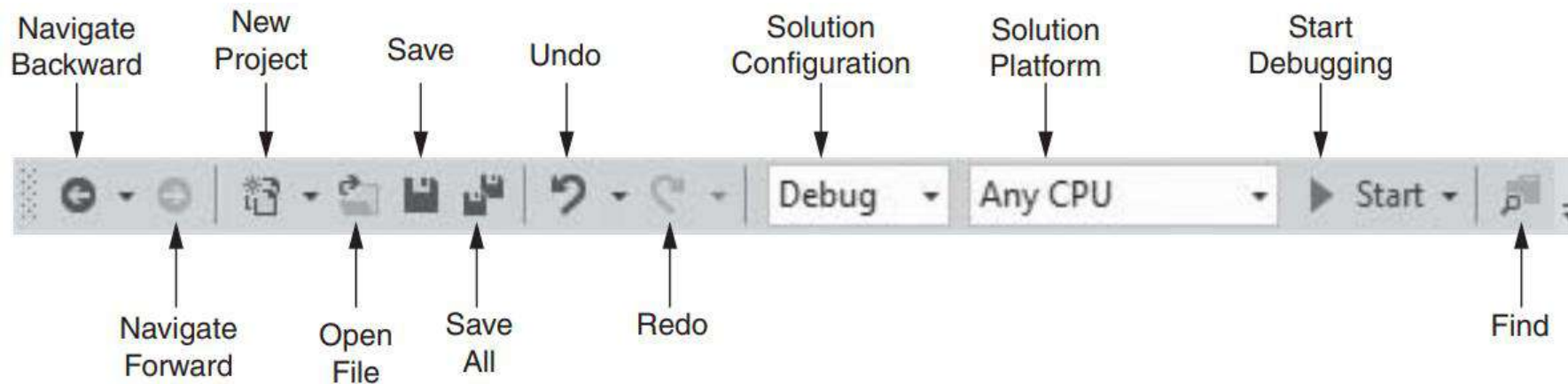
Using Auto Hide

- When Auto Hide is turned on, the window is displayed only as a tab along one of the edges of the Visual Studio environment.
- This feature gives you more room to view your application's forms and code.
- This Figure shows how the Solution Explorer and Properties windows appear when their Auto Hide feature is turned on.
- Notice the tabs that read Solution Explorer and Properties along the right edge of the screen.



Menu Bar and Standard Toolbar

- You've already used the Visual Studio menu bar several times. This is the bar at the top of the Visual Studio window that provides menus such as File, Edit, View, Project, and so forth.
- Below the menu bar is the standard toolbar. The standard toolbar contains buttons that execute frequently used commands.
- The following figure identifies the standard toolbar buttons that you will use most often:



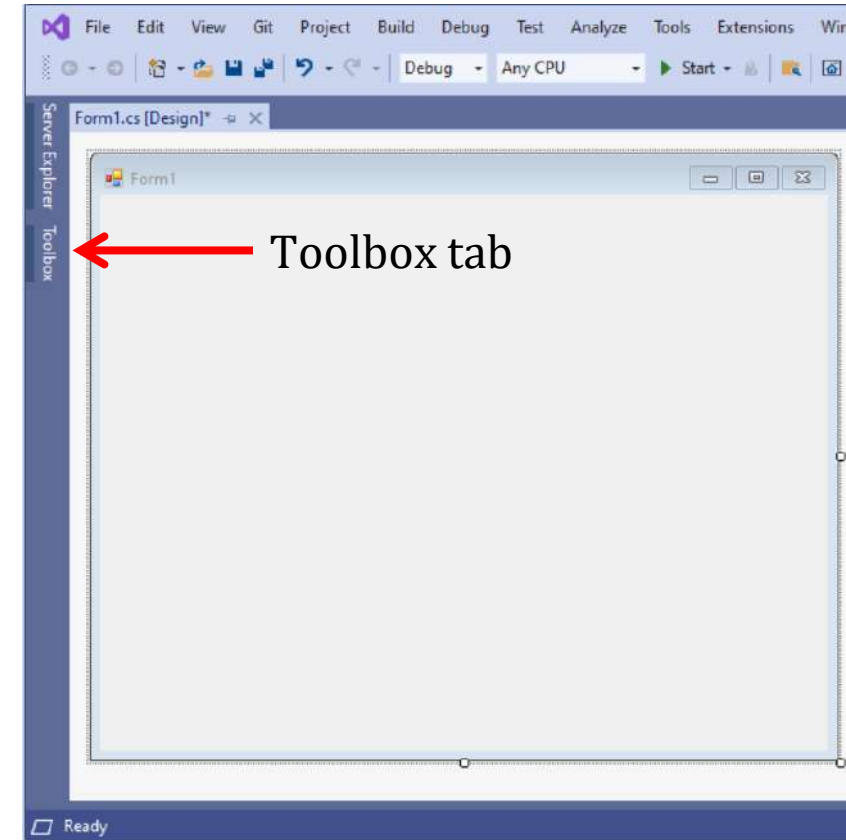
Menu Bar and Standard Toolbar

- This table gives a brief description of each button in the toolbar.

Toolbar Button	Description
Navigate Backward	Moves to the previously active tab in the <i>Designer</i> window
Navigate Forward	Moves to the next active tab in the <i>Designer</i> window
New Project	Starts a new project
Open File	Opens an existing file
Save	Saves the file named by <i>filename</i>
Save All	Saves all the files in the current project
Undo	Undoes the most recent operation
Redo	Redoes the most recently undone operation
Solution Configurations	Configures your project's executable code
Solution Platform	Lets you select the platform on which the application will run
Start Debugging	Starts debugging (running) your program
Find	Searches for text in your application code

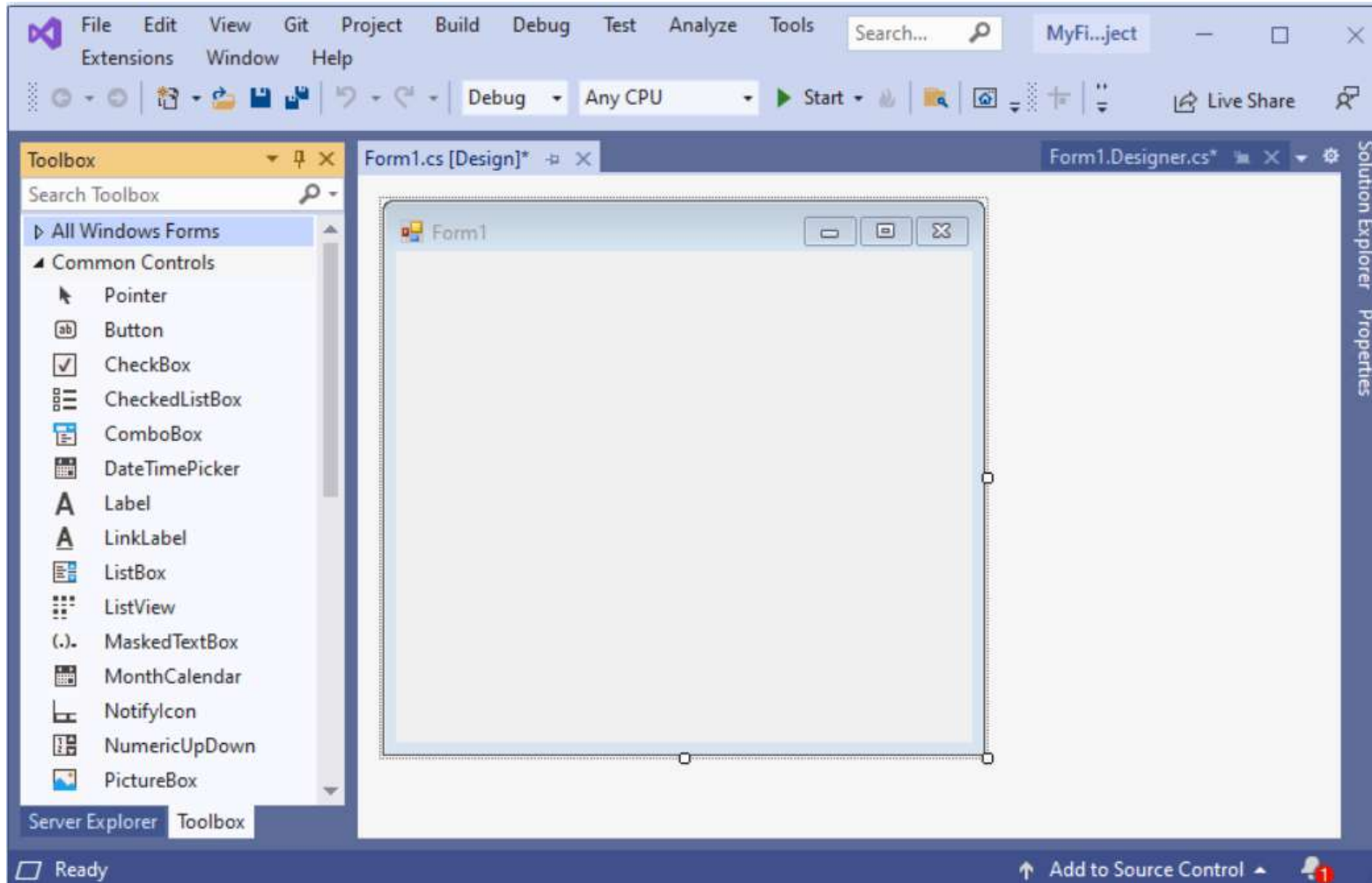
Toolbox

- The Toolbox is a window that allows you to select the controls that you want to use in an application's user interface.
- When you want to place a Button, Label, TextBox, or other control on an application's form, select it in the Toolbox. You will use the Toolbox extensively as you develop Visual C# applications.
- The Toolbox typically appears on the left side of the Visual Studio environment.
- If the Toolbox is in Auto Hide mode, its tab will appear as shown in the figure.



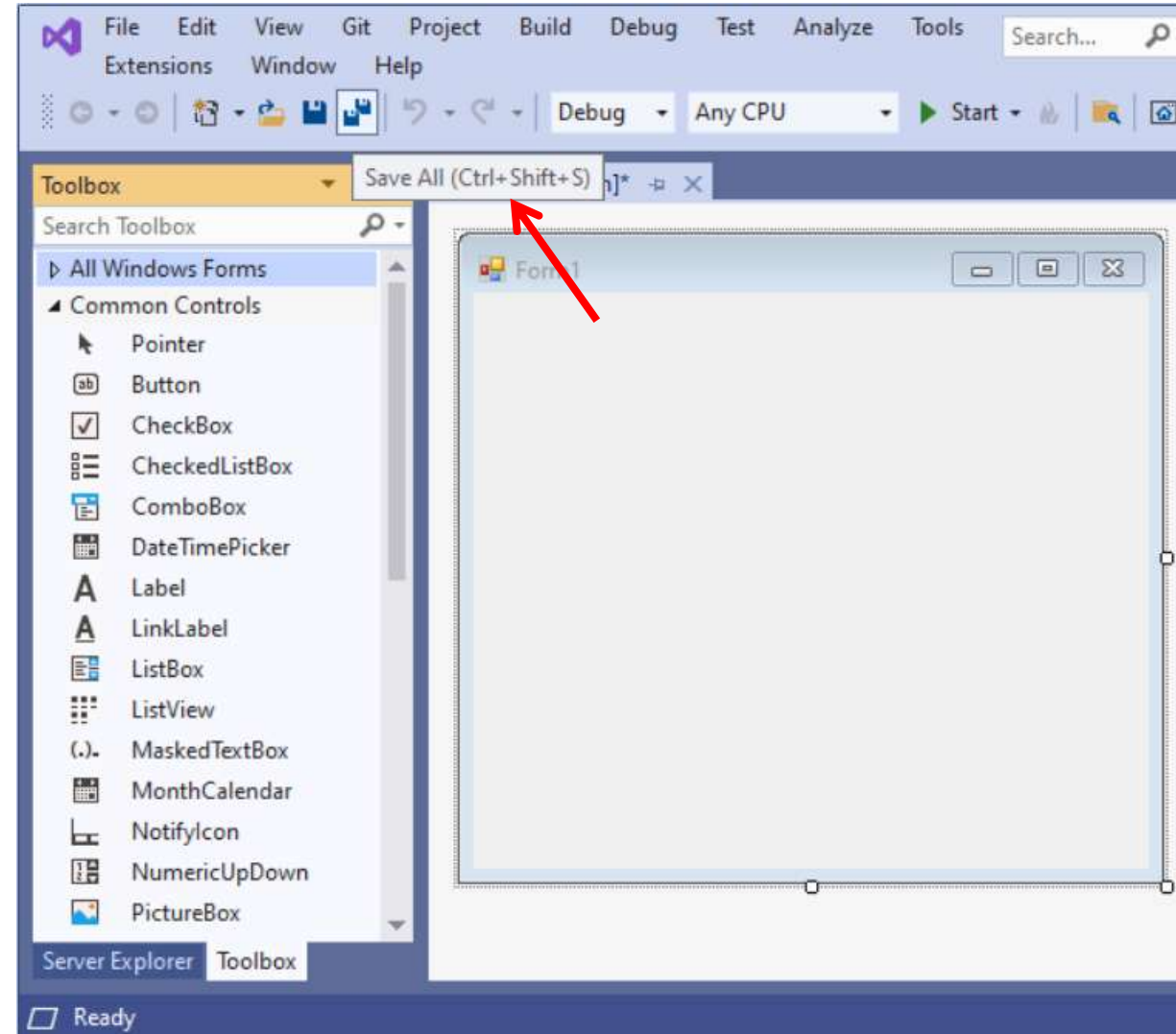
Toolbox

- The following figure shows the Toolbox opened, with Auto Hide turned off.



ToolTips

- A ToolTip is a small rectangular box that pops up when you hover the mouse pointer over a button on the toolbar or in the Toolbox for a few seconds.
- The ToolTip box contains a short description of the button's purpose.
- The figure shows the ToolTip that appears when the cursor is left sitting on the Save All button.
- Use a ToolTip whenever you cannot remember a particular button's function.

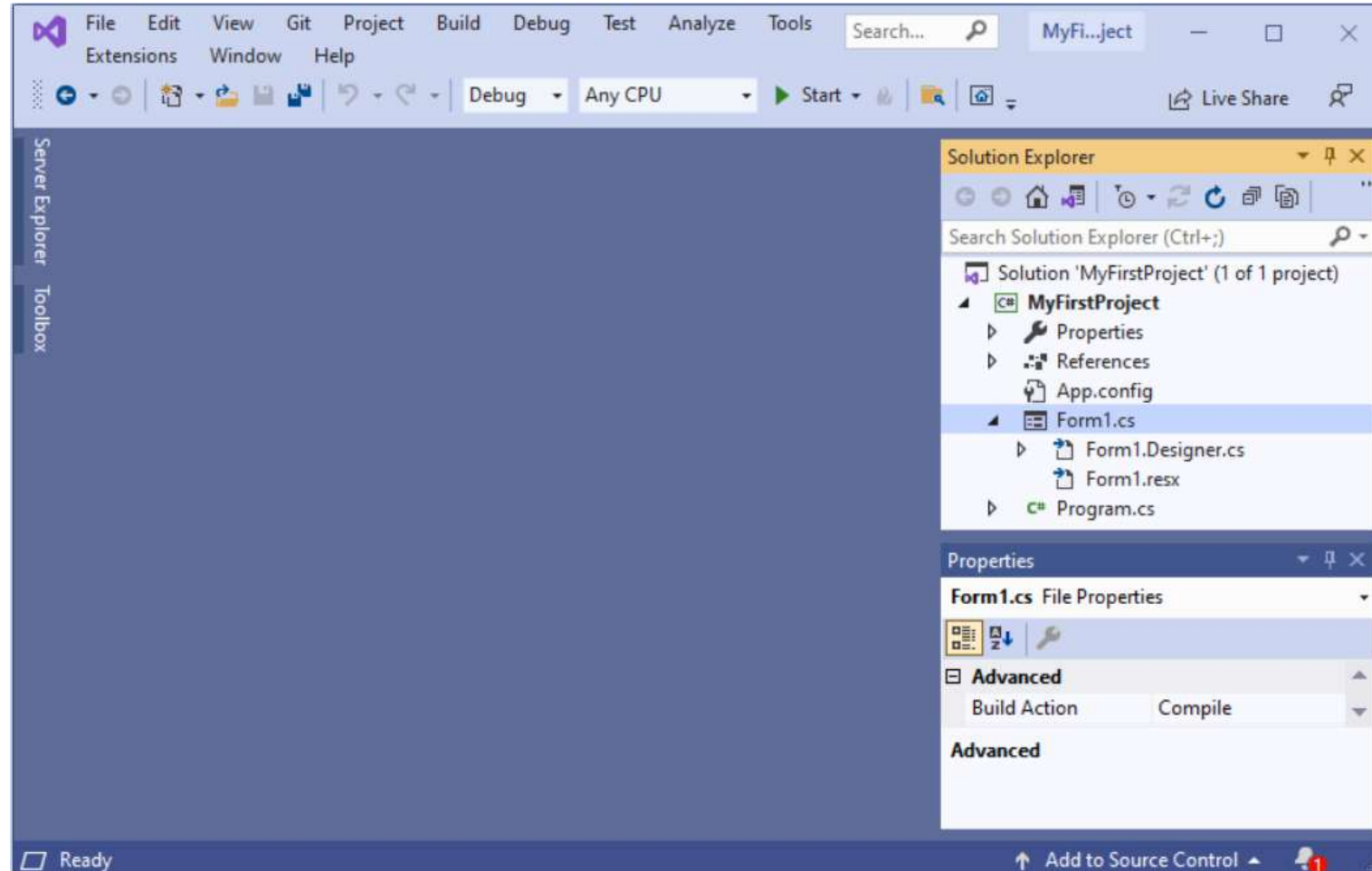


Projects and Solutions

- Each Visual C# application that you create is called a project. A Visual C# project consists of several related files such as the Visual C# code and any images that might make up an application.
- A solution is a container that holds one or more Visual C# projects.
- If you are developing applications for a large organization, you might find it convenient to store several related projects together in the same solution.
- Although it is possible for a solution to hold more than one project, each project that you will create will be saved in its own solution.
- Each time you create a new project, you will also create a new solution to hold it.
- Typically, the solution will be given the same name as the project.

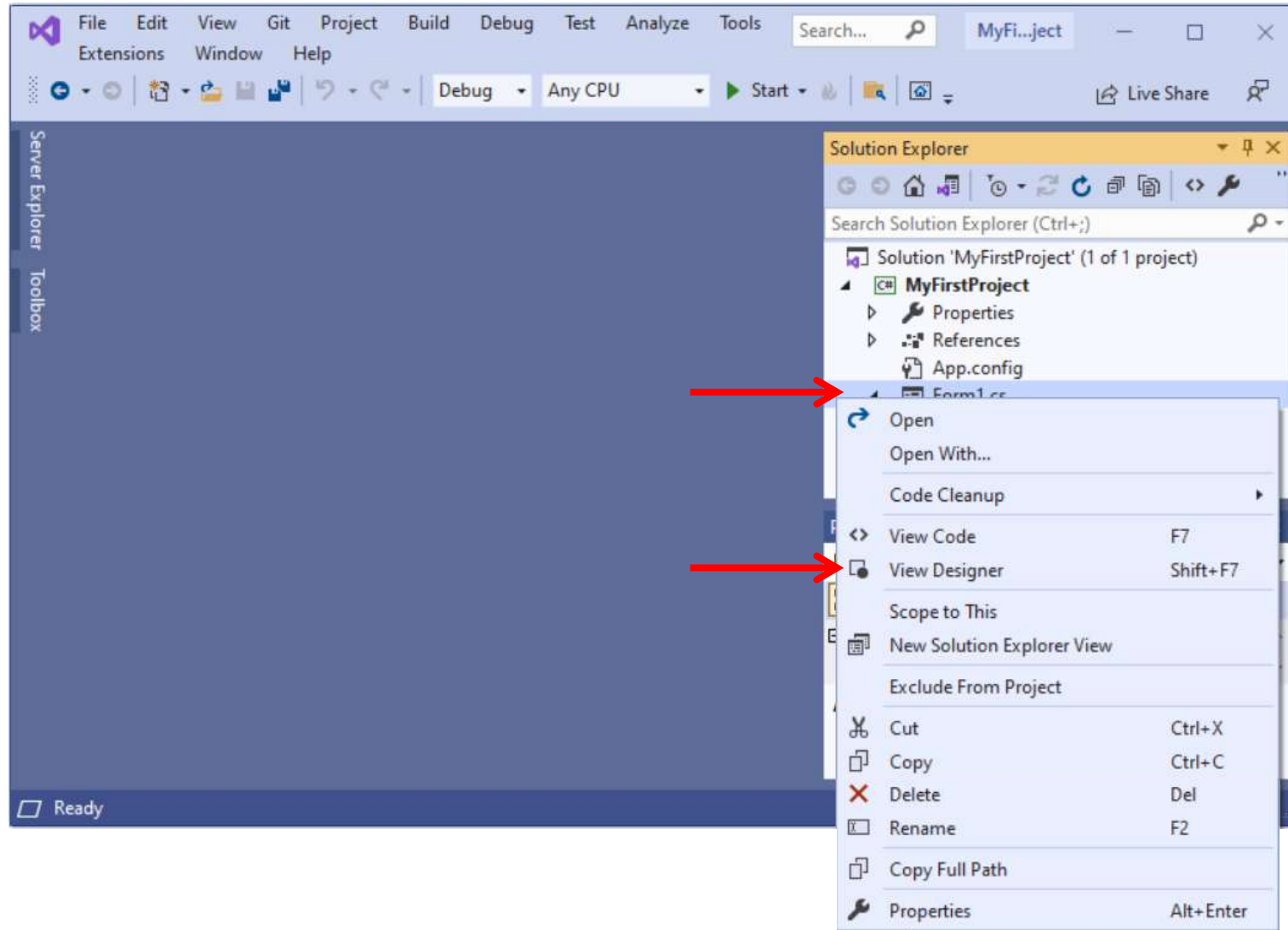
Displaying the Designer

- Sometimes when you open an existing project, the project's form will not be automatically displayed in the Designer.
- This figure shows an example of the Visual Studio environment with an opened project but no form displayed in the Designer.



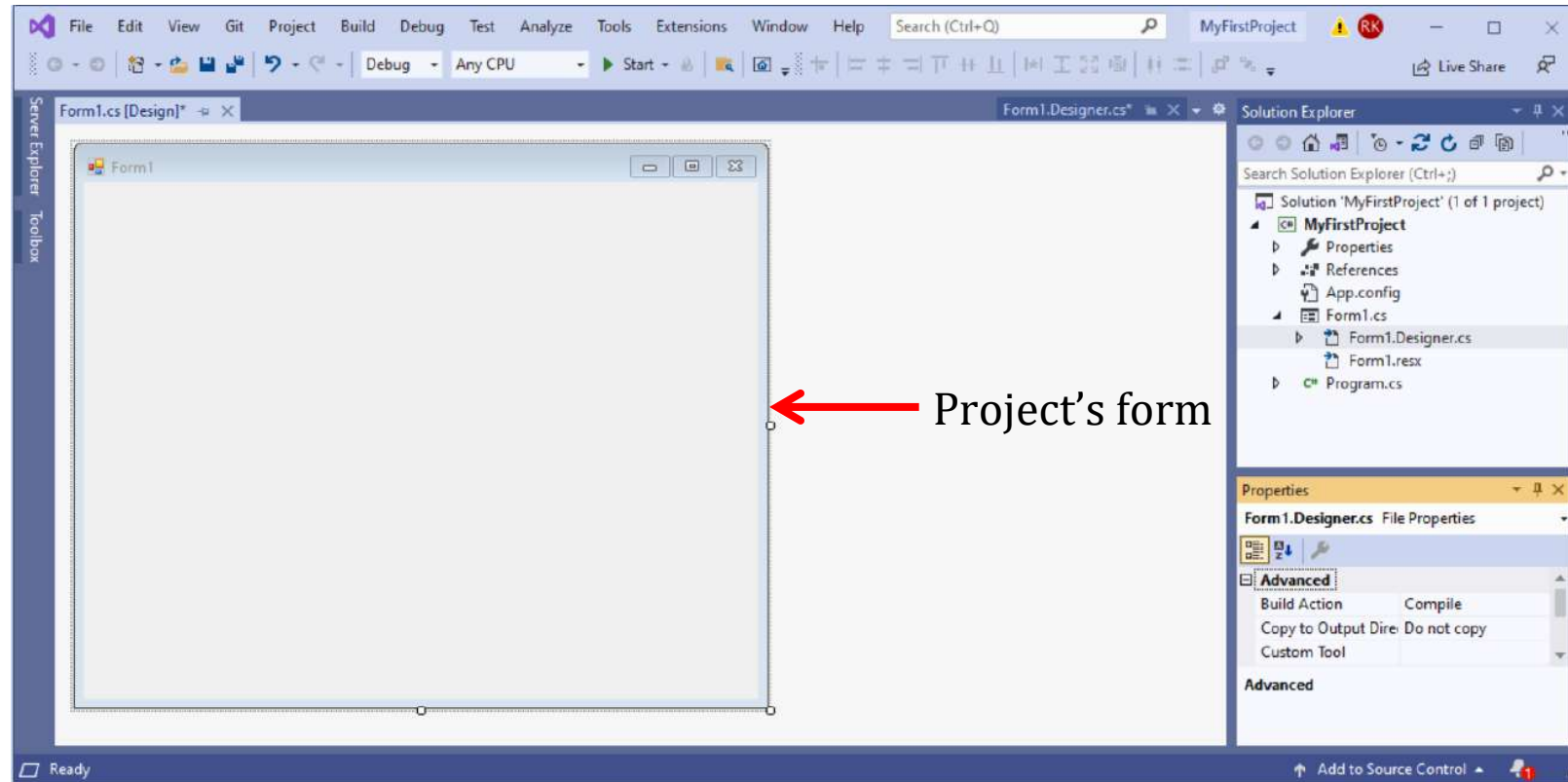
Displaying the Designer

- Perform the following steps to display the project's form in the Designer:
 - As shown in the figure, right-click Form1.cs in the Solution Explorer.
 - Click View Designer in the pop-up menu.



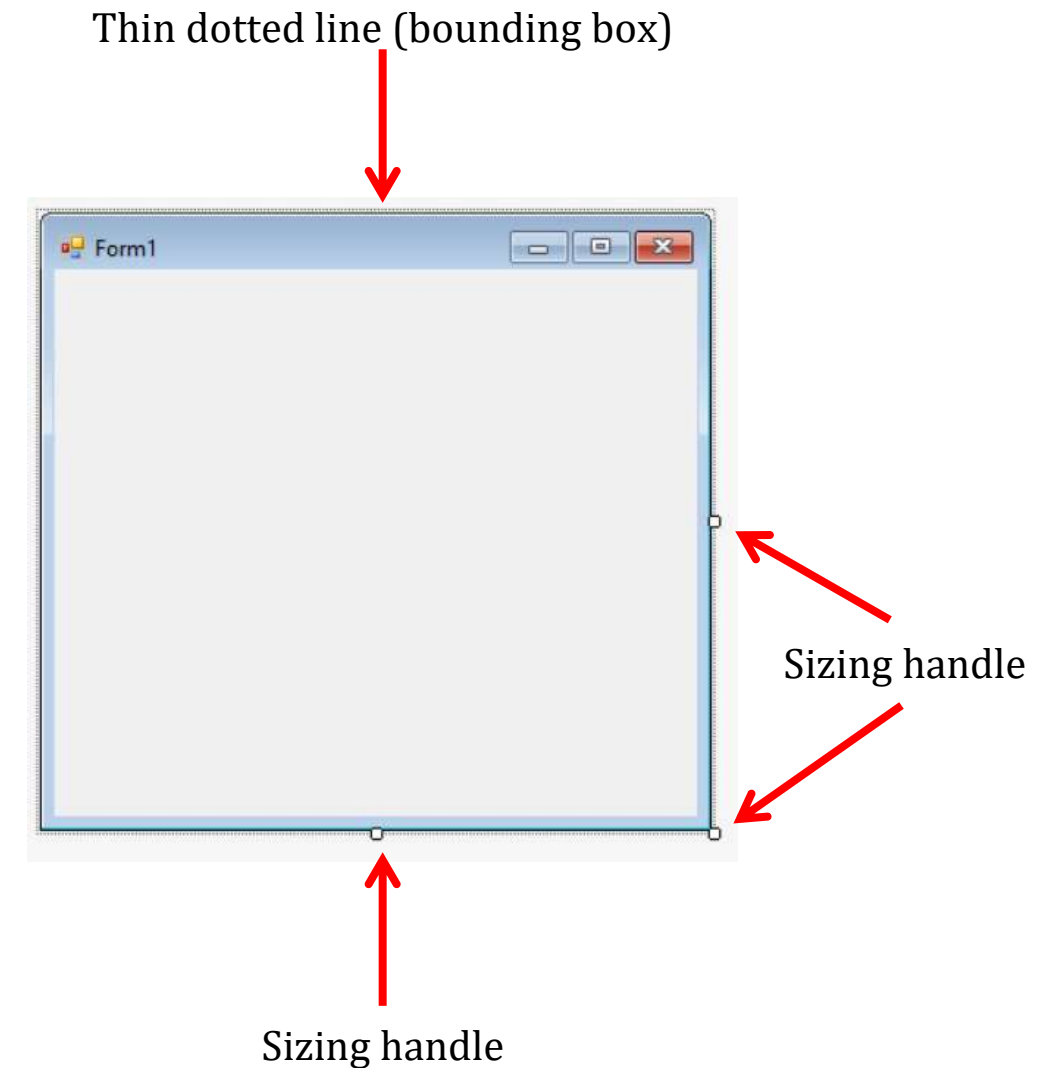
Getting Started with Forms and Controls

- In this chapter, you will create your first Visual C# application.
- When you start a new Visual C# project, Visual Studio automatically creates an empty form and displays it in the Designer, as shown in the figure.
- You can add controls to the form, change the form's size, and modify many of its characteristics.
- When the application runs, the form will be displayed on the screen.



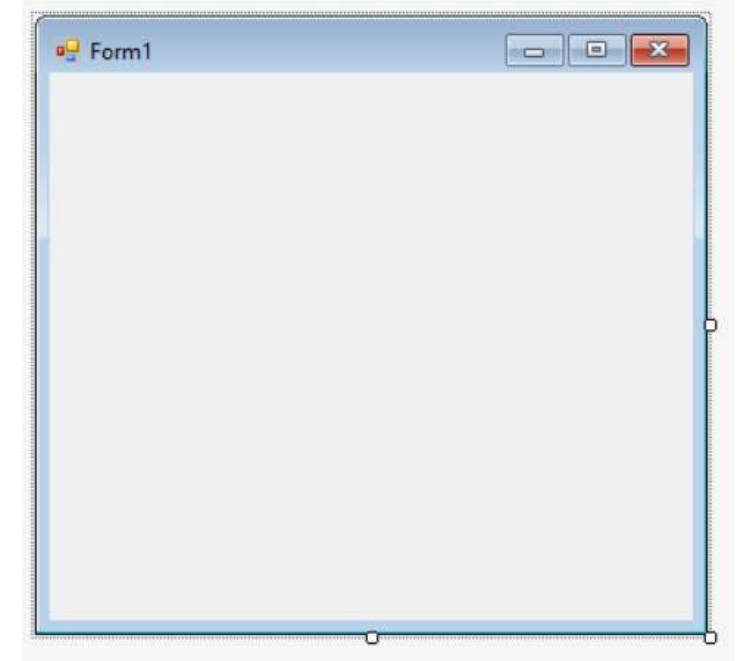
Getting Started with Forms and Controls

- If you take a closer look at the form, you will notice that it is enclosed by a thin dotted line, known as a bounding box.
- As shown in the figure, the bounding box has small sizing handles, which appear on the form's right edge, bottom edge, and lower-right corner.
- When a bounding box appears around an object in the Designer, it indicates that the object is selected and is ready for editing.
- Initially the form's size is 300 pixels wide by 300 pixels high. However, you can easily resize the form with the mouse.



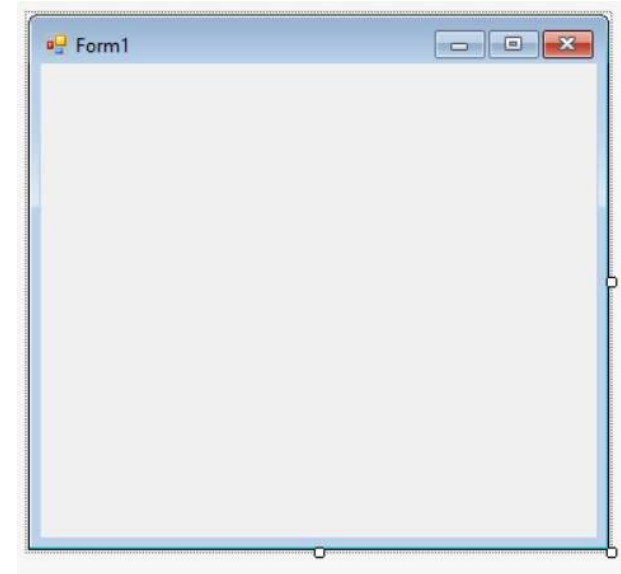
Identifying Forms and Controls by Names

- An application's GUI is made of forms and various controls.
- Each form and control in an application's GUI must have a name that identifies it.
- The blank form that Visual Studio initially creates in a new project is named Form1.

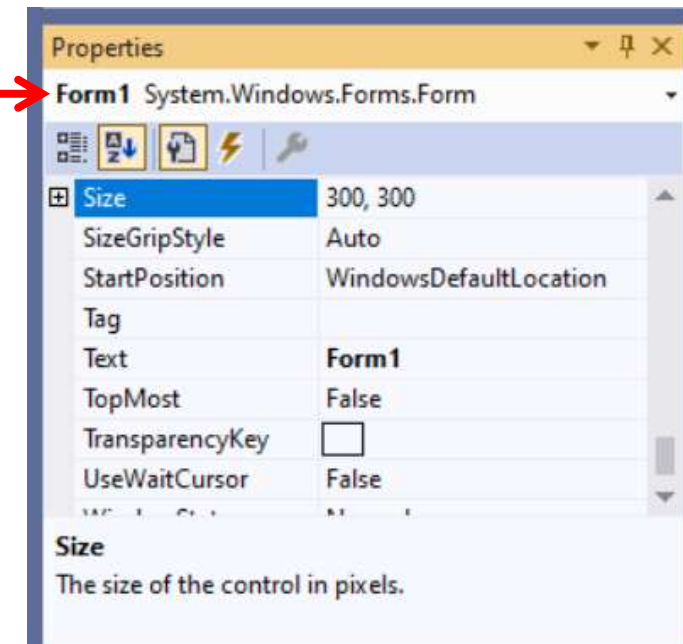


Properties Window

- The appearance and other characteristics of a GUI object are determined by the object's properties.
- When you select an object in the Designer, that object's properties are displayed in the Properties window. That is, you can use the Properties window to view and change its properties.
- For example, when the Form1 form is selected, its properties are displayed in the Properties window, as shown in the figure.
- The area at the top of the Properties window shows the name of the object that is currently selected. You can see in the figure that the name of the selected object is Form1.

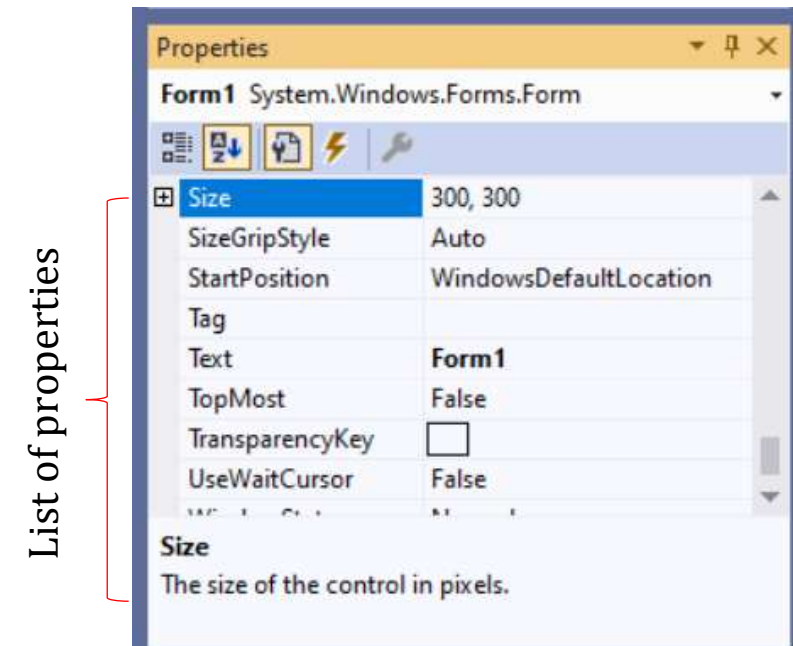
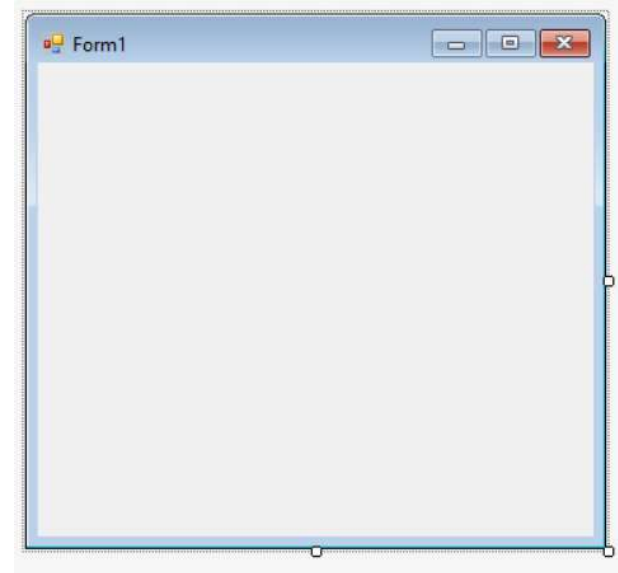


Name of the
selected object



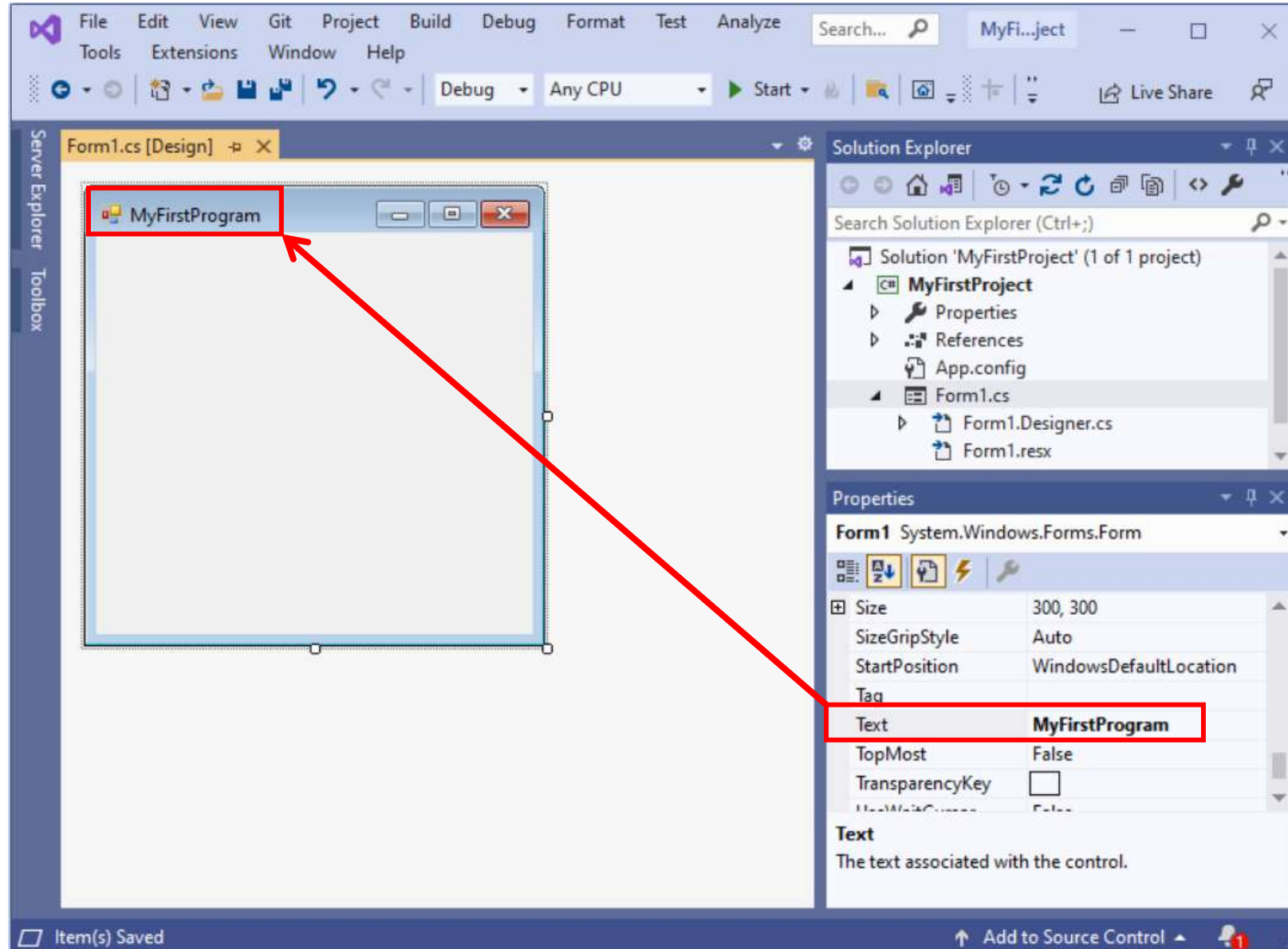
Properties Window

- Below that is a scrollable list of properties. The list of properties has two columns: the left column shows each property's name, and the right column shows each property's value.
- For example, the form's Size property in the pervious Figure its value is 300, 300. This means that the form's size is 300 pixels wide by 300 pixels high.
- The Text property determines the text that is displayed in the form's title bar (the bar that appears at the top of the form).
- Its current value is Form1, so the text Form1 is displayed in the form's title bar.



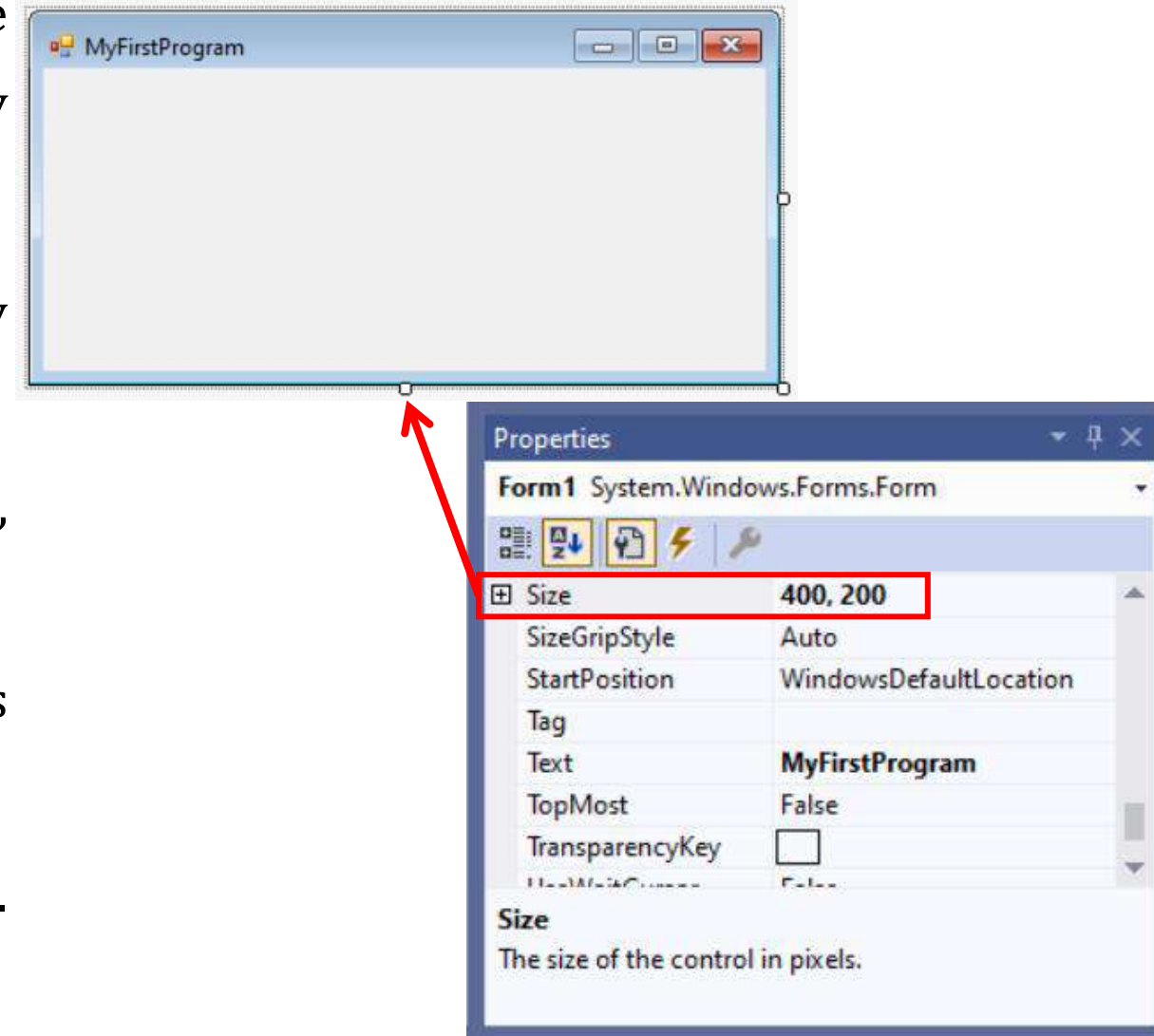
Properties Window

- Changing an object's Text property does not change the object's name.
- For example, if you change the Form1 form's Text property to MyFirstProgram, the form's name is still Form1. You have changed only the text that is displayed in the form's title bar, as shown in the figure.



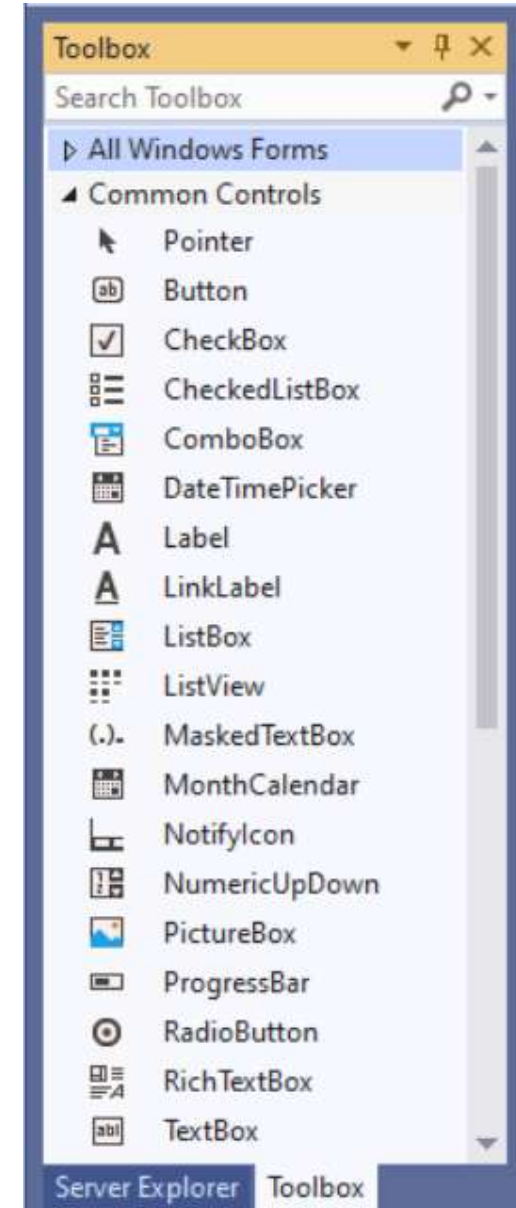
Properties Window

- As we can use the mouse to resize a form in the Designer, we can also change the form's Size property in the Properties window.
- To change the size of the form to 400 pixels wide by 200 pixels high:
 - 1) Select the Form1 form. In the Properties window, locate the Size property.
 - 2) Click inside the area that holds the Size property's value, and then delete the current value.
 - 3) Type 400, 100 in its place and press the enter key. The form will be resized, as shown in the Figure.



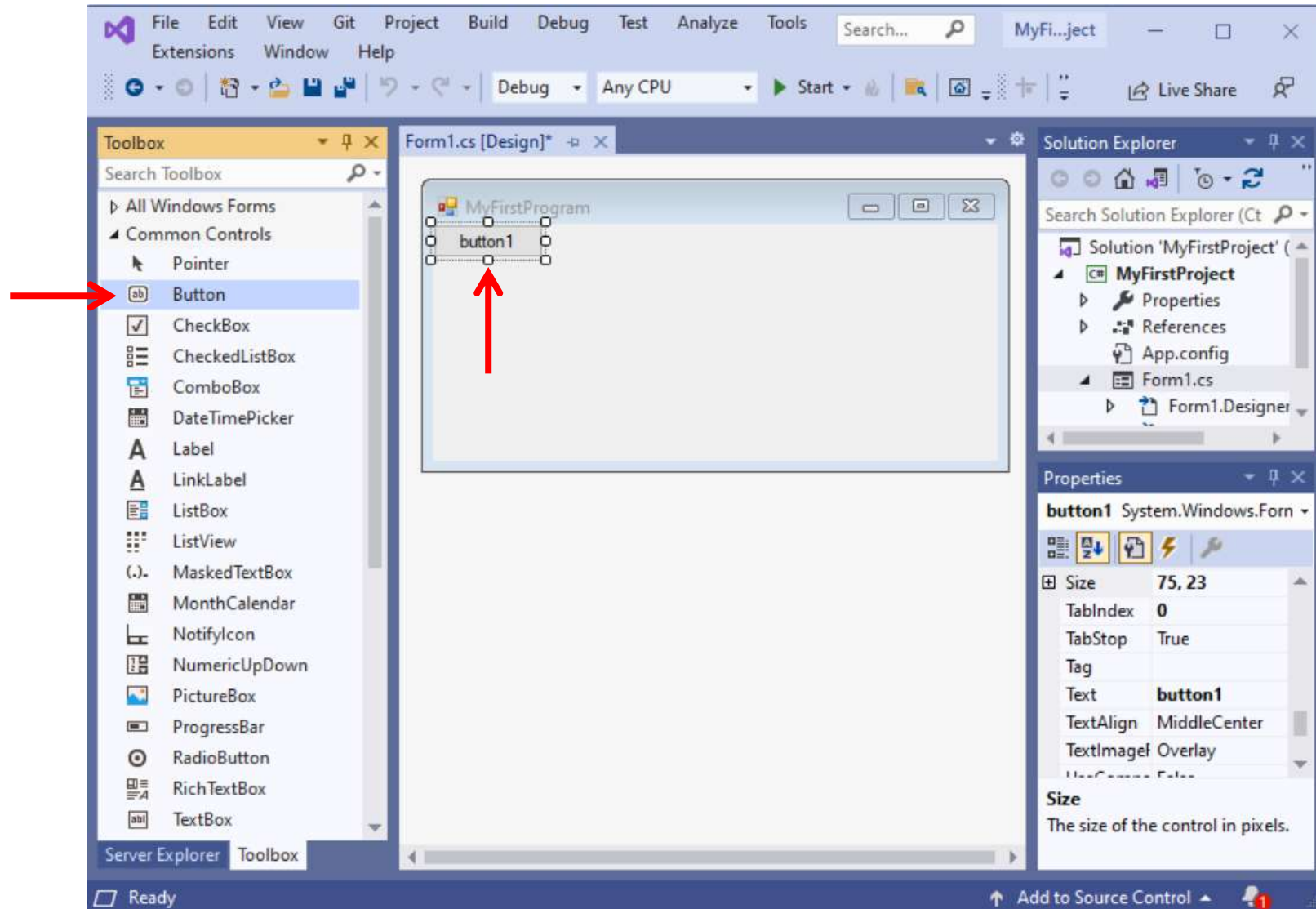
Adding Controls to a Form

- When you are ready to create controls on the application's form, you use the Toolbox.
- The figure shows an example of how the Toolbox typically appears when it is open.
- The Toolbox shows a scrollable list of controls that you can add to a form.



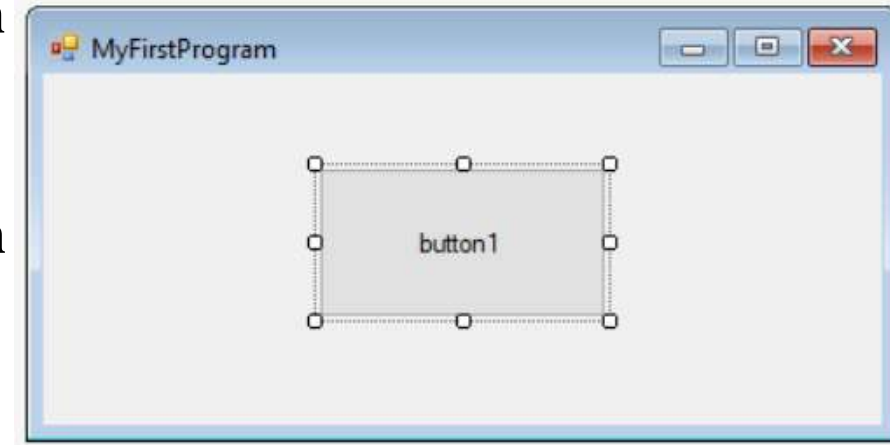
Adding Controls to a Form

- To add a control to a form, you simply find it in the Toolbox and then double-click it. The control will be created on the form as shown in figure.
- You can also click and drag controls from the Toolbox onto the form.



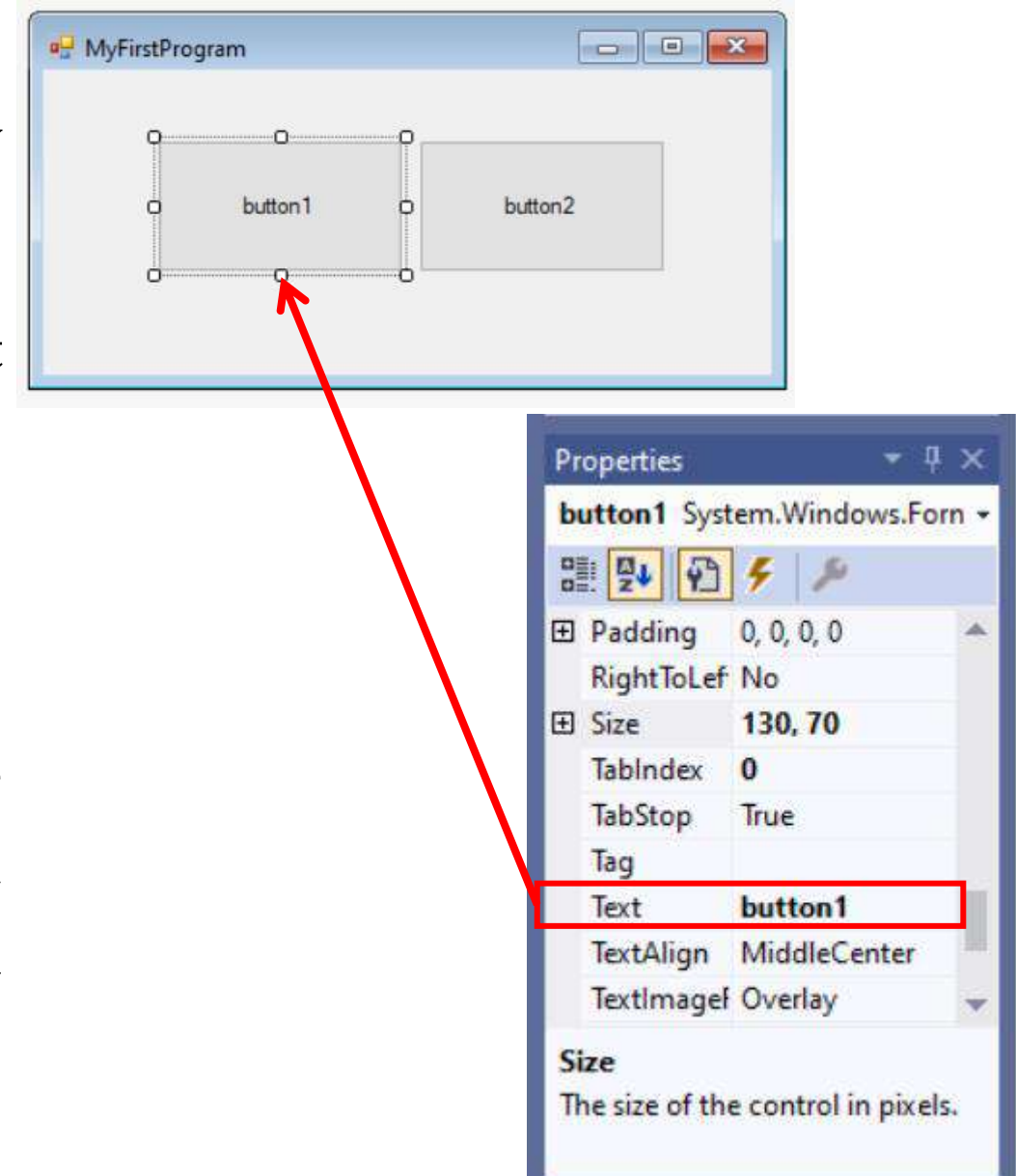
Resizing and Moving Controls

- When a control is selected, you can use the mouse to resize it in the same way that you learned to resize a form earlier.
- You can also use the mouse to move a control to a new location on the form.
- Position () the mouse cursor inside the control, and when the mouse cursor becomes a four-headed arrow, you can click and drag the control to a new location.
- The figure shows a form with a Button control that has been enlarged and moved.
- Deleting a control is simple: you select it and then press the Delete key on the keyboard.



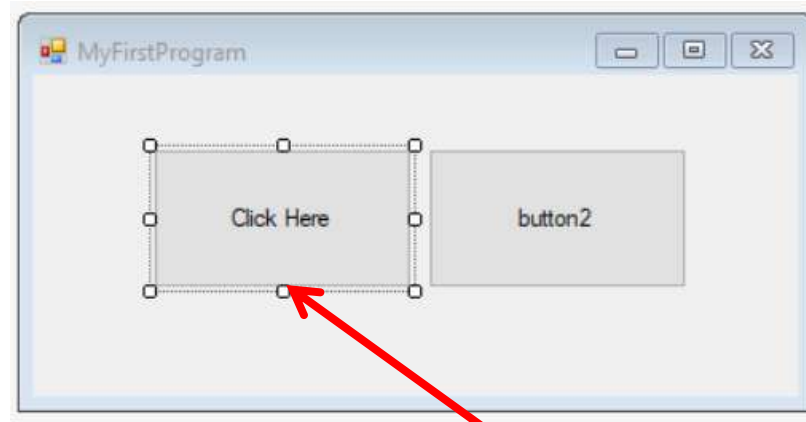
More about Button Control

- When you create Button controls, they are automatically given default names such as button1, button2, and so forth.
- Button controls have a Text property, which holds the text that is displayed on the face of the button.
- When a Button control is created, its Text property is initially set to the same value as the Button control's name.
- When you create a Button control, its name will be displayed on the face of the button. For example, the form in the figure contains three Button controls named button1, button2, and button3.



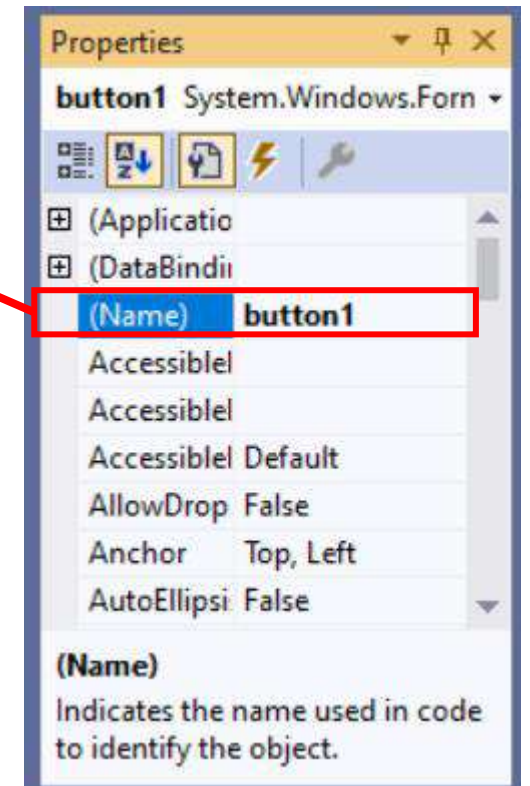
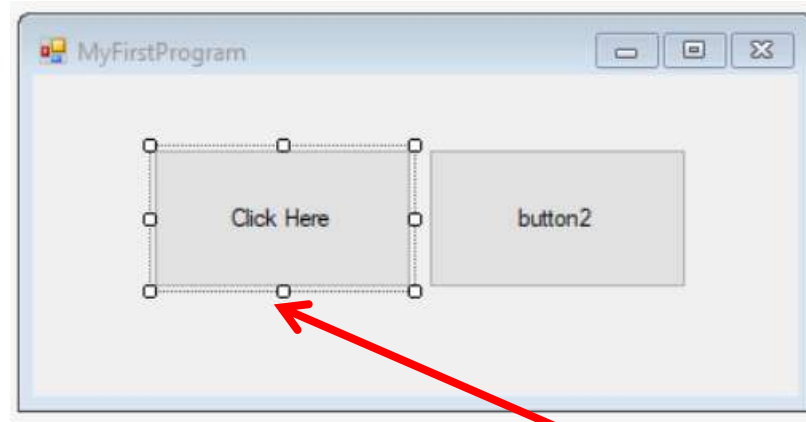
More about Button Control

- This figure shows an example of how changing a Button control's Text property changes the text displayed on the face of the button.



Changing a Control's Name

- A control's name identifies the control in the application's code and in the Visual Studio environment.
- When you create a control on an application's form, you should always change the control's name to something that is more meaningful than the default name that Visual Studio gives it.
- A control's name should reflect the purpose of the control.

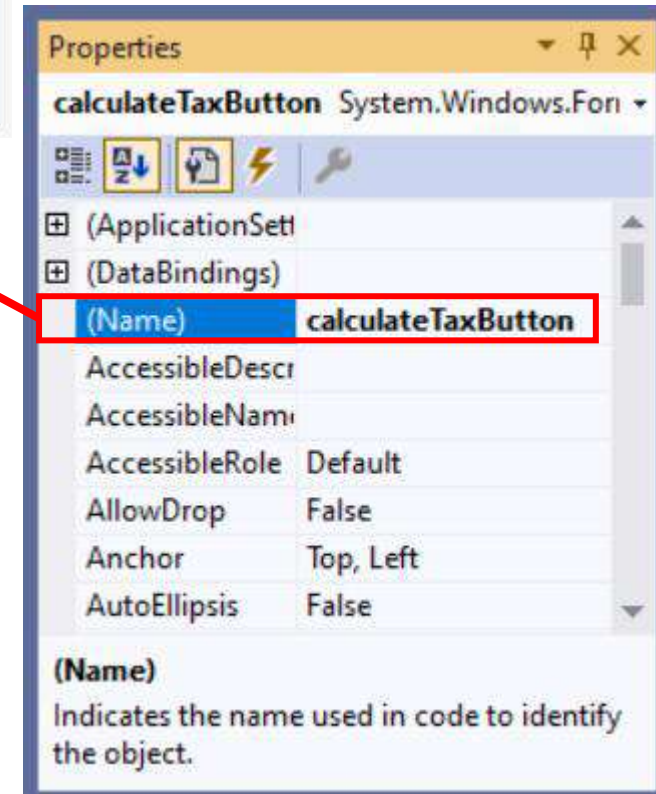
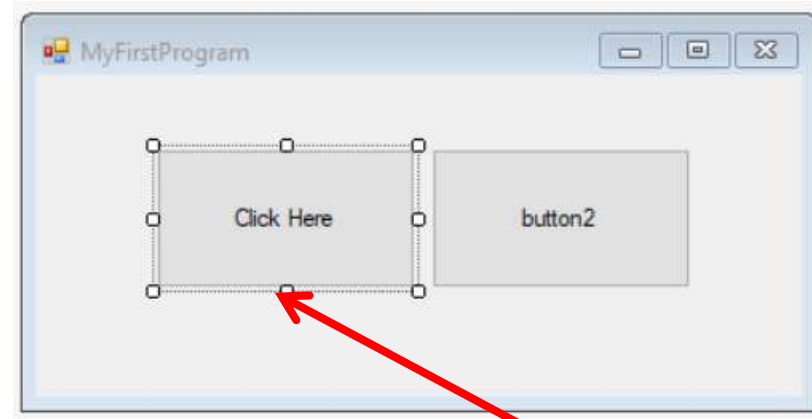


Changing a Control's Name

- You can change a control's name by changing its **Name** property as follows:

- 1) Select the control.
- 2) In the Properties window, scroll up to the top of the list of properties.
- 3) Click inside the area that holds the Name property's value and then delete the current name. Then, type the new name in its place and press the enter key. You have successfully changed the name of the control.

This figure shows the properties window after button control's name has been changed to calculateTaxButton.



Rules for Naming Controls

- Control names are also known as identifiers. When naming a control, you must follow these rules for C# identifiers:
 - The first character must be one of the letters a through z or A through Z or an underscore character (_).
 - After the first character, you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
 - The name cannot contain spaces.

- This table lists some identifiers that might be used for Button control names and indicates whether each is a legal or illegal identifier in C#.

Identifier	Legal or Illegal?
showDayOfWeekButton	Legal
3rdQuarterButton	Illegal because identifiers cannot begin with a digit.
change*color*Button	Illegal because the * character is not allowed.
displayTotalButton	Legal
calculate Tax Button	Illegal because identifiers cannot contain spaces.

Rules for Naming Controls

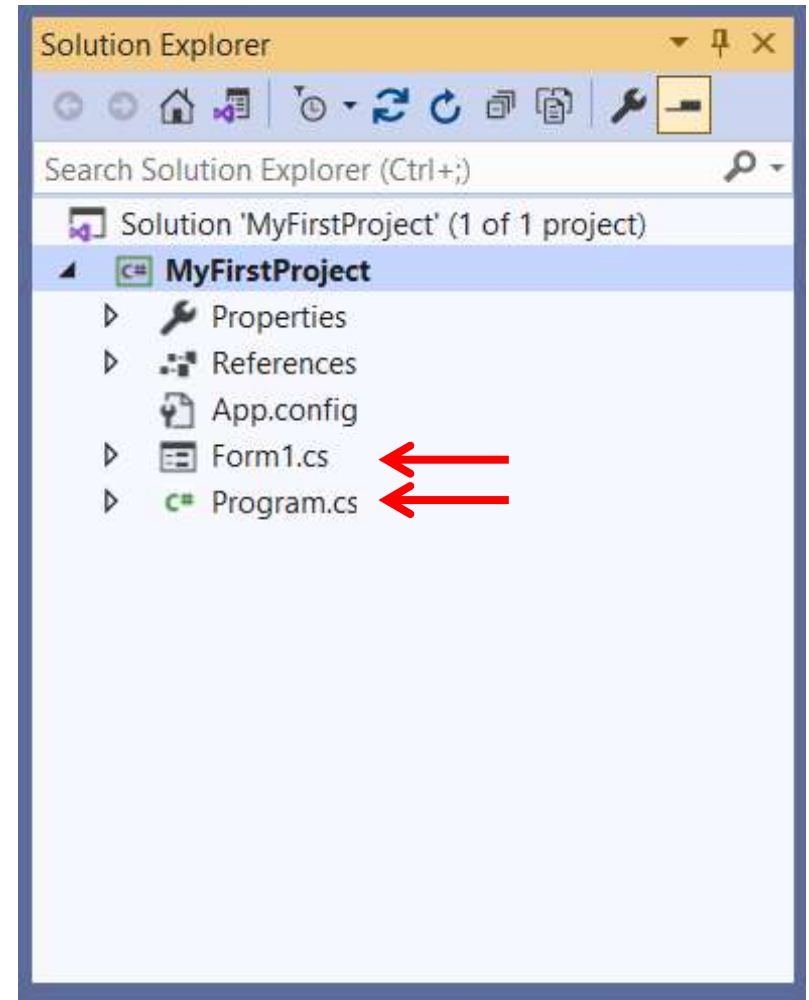
- A control's name should reflect the control's purpose, programmers often find themselves creating names that are made of multiple words. For example, consider the following Button control names:
 - calculatetaxbutton
 - printreportbutton
 - displayanimationbutton
- Unfortunately, these names are not easily read by the human eye because the words are not separated.
- Because we cannot have spaces in control names, we need to find another way to separate the words in a multiword control name to make it more readable to the human eye.
- Most C# programmers address this problem by using the **camelCase** naming convention for controls.

Rules for Naming Controls

- camelCase names are written in the following manner:
 - You begin writing the name with lowercase letters.
 - The first character of the second and subsequent words is written in uppercase.
- For example, the following control names are written in camelCase:
 - calculateTaxButton
 - printReportButton
 - displayAnimationButton

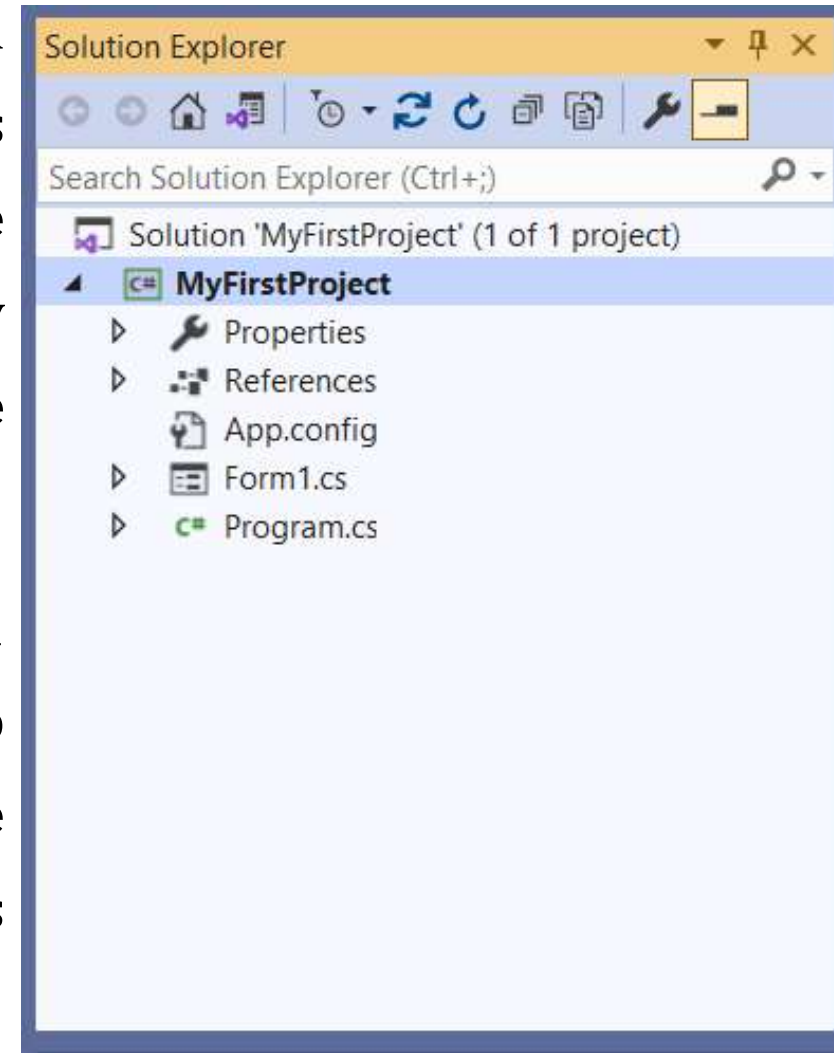
Introduction to C# Code

- If you want your application to perform any meaningful actions, you have to write code.
- A file that contains program code is called a source code file.
- When you start a C# Windows Forms Application project, Visual Studio automatically creates several source code files and adds them to the project.
- If you look at the Solution Explorer, you will see the names of two source code files:
 - Form1.cs
 - Program.cs
- C# source code files always end with the .cs extension.



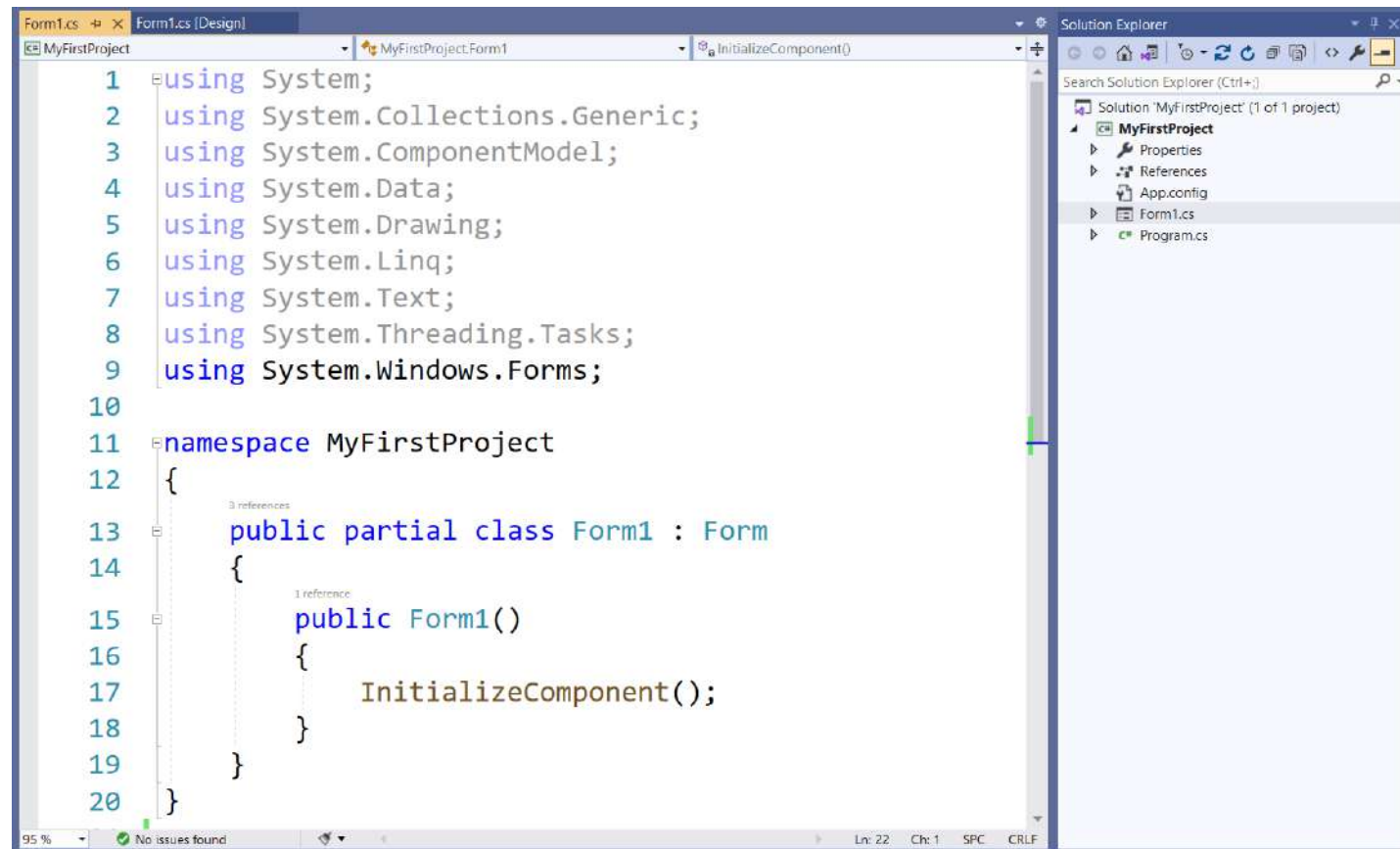
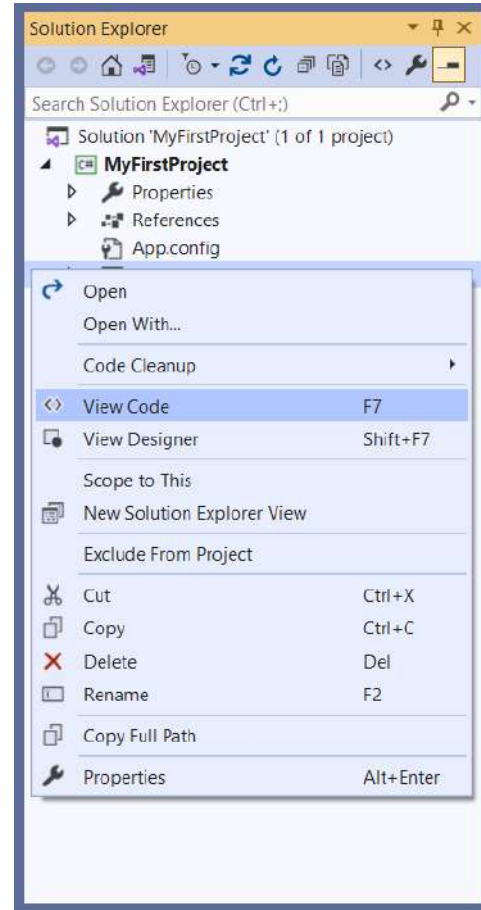
Introduction to C# Code

- The **Program.cs** file contains the application's start-up code, which executes when the application runs. The code in this file performs behind-the-scenes initialization tasks that are necessary to get the application up and running. It is important that you do not modify the contents of this file because doing so could prevent the application from executing.
- The **Form1.cs** file contains code that is associated with the Form1 form. When you write code that defines some action related to Form1 (such as responding to a button click), you will write the code in this file. The Form1.cs file already contains code that was generated by Visual Studio when the project was created.



Introduction to C# Code

- Right-click Form1.cs in the Solution Explorer, a pop-up menu will appear. On the pop-up menu, click View Code.
- The file's contents will be displayed in the Visual Studio code editor, as shown in the figure.

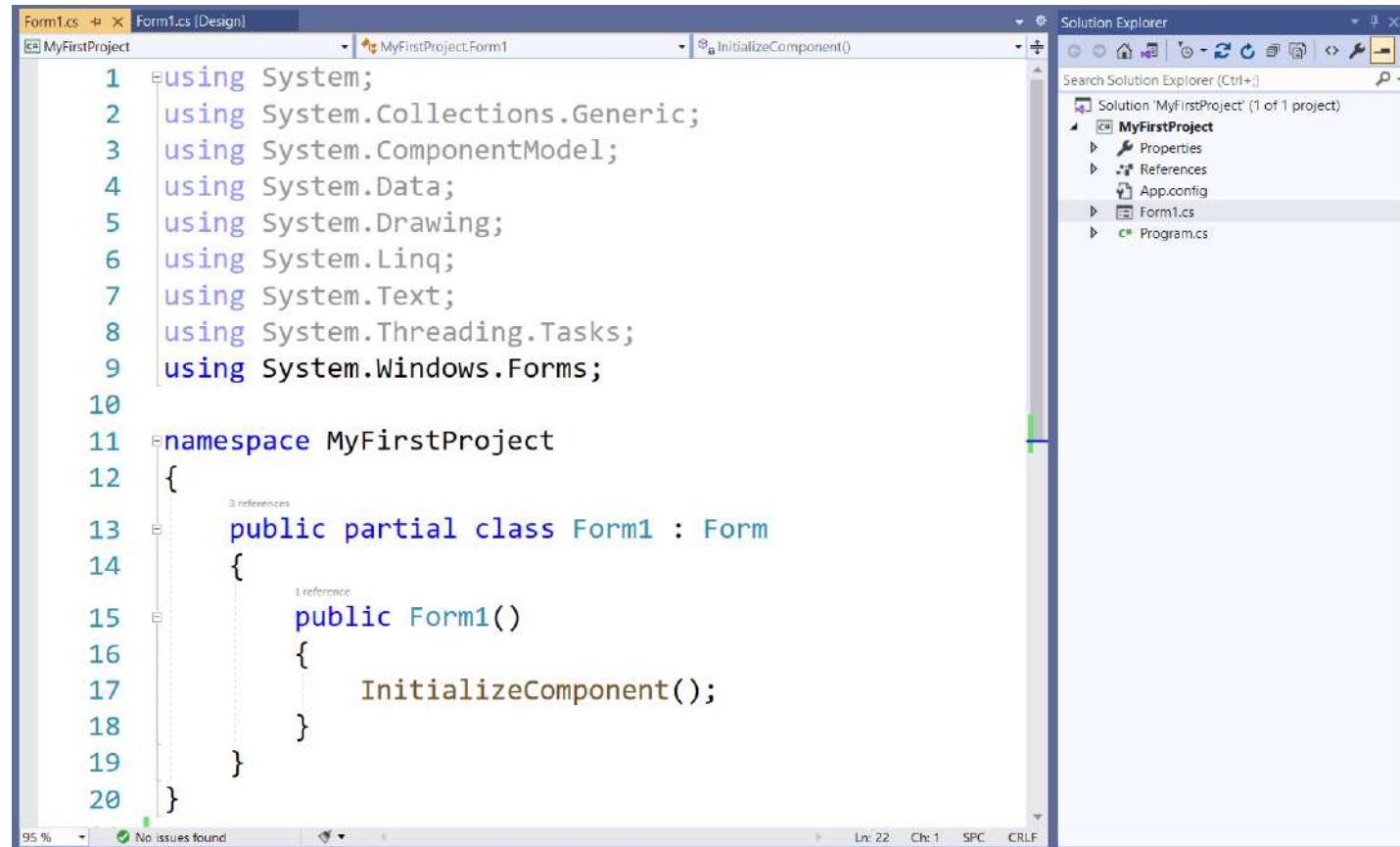


Introduction to C# Code

■ C# code is primarily organized in three ways: namespaces, classes, and methods.

- A namespace is a container that holds classes.
- A class is a container that holds methods
- A method is a group of one or more programming statements that performs some operation.

■ So, C# code is organized as methods, which are contained inside classes, which are contained inside namespaces.



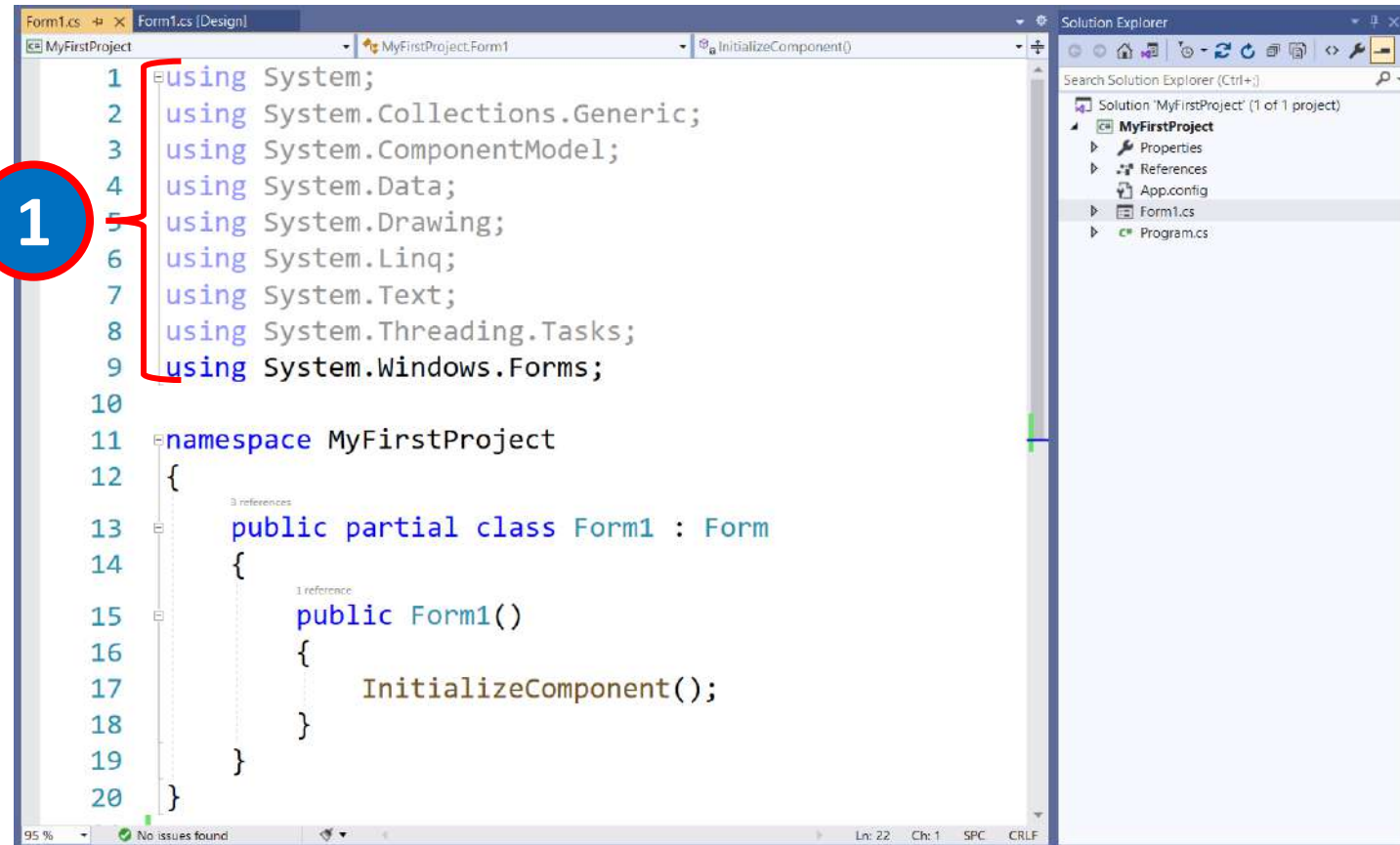
The screenshot displays the Visual Studio IDE with a C# project named 'MyFirstProject'. The main editor window shows the code for 'Form1.cs' in Design view. The code includes several 'using' statements for namespaces like 'System', 'System.Collections.Generic', 'System.ComponentModel', 'System.Data', 'System.Drawing', 'System.Linq', 'System.Text', 'System.Threading.Tasks', and 'System.Windows.Forms'. It then defines a 'namespace MyFirstProject' containing a 'public partial class Form1 : Form'. Inside the class, there is a 'public Form1()' constructor method that calls 'InitializeComponent()'. The Solution Explorer on the right shows the project structure with 'Form1.cs' selected. The status bar at the bottom indicates '95 %' zoom and 'No issues found'.

```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19     }
20 }
```

Introduction to C# Code

- The code in the .NET Framework is organized into namespaces.

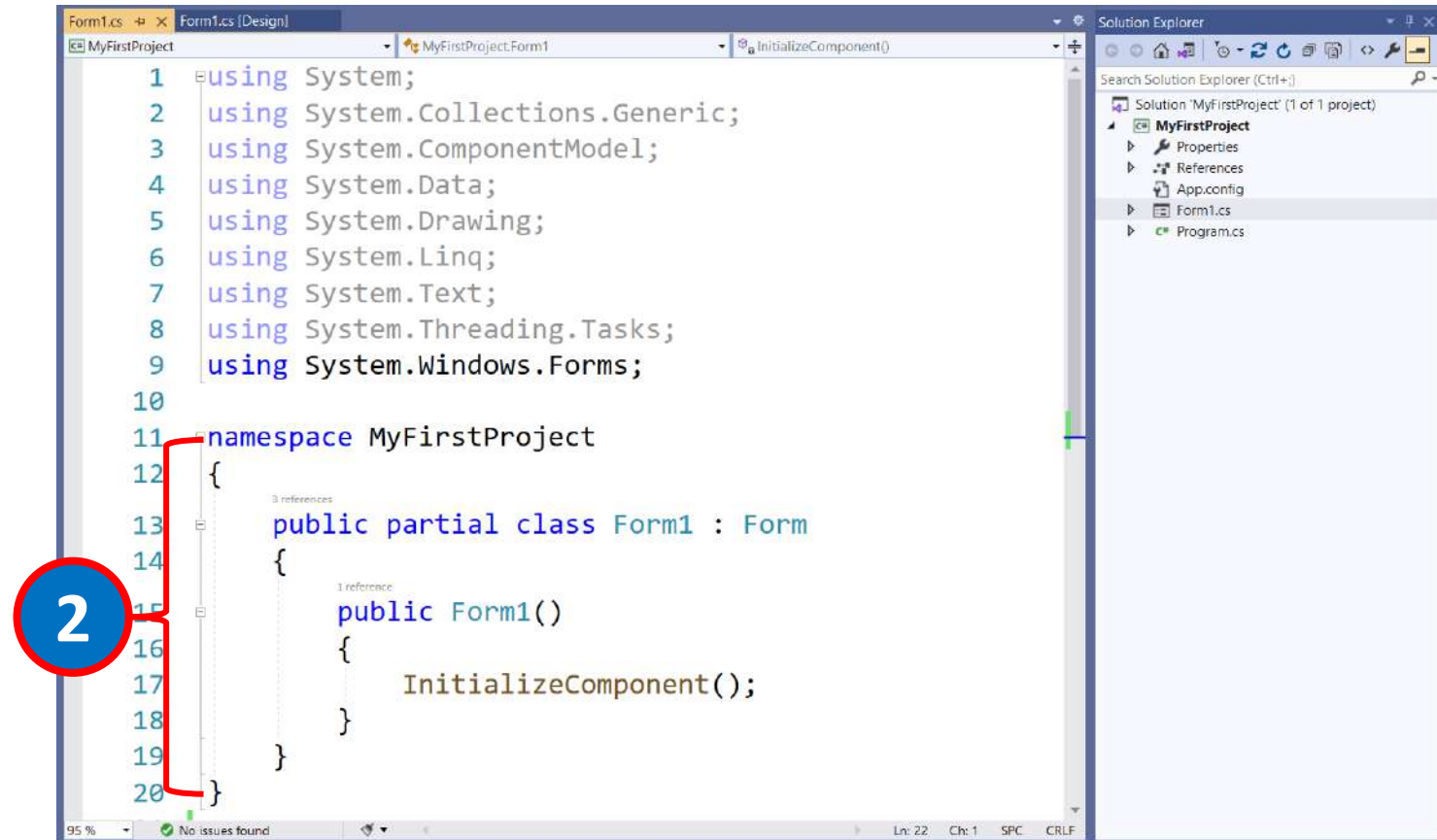
1) The series of using directives that appears at the top of a C# source code file indicate which namespaces in the .NET Framework the program will use. Each using directive identifies a namespace containing classes that a C# app should be able to use. The using directive in line 1 indicates that this example intends to use classes from the System namespace, which contains the predefined Console class.



```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19     }
20 }
```

Introduction to C# Code

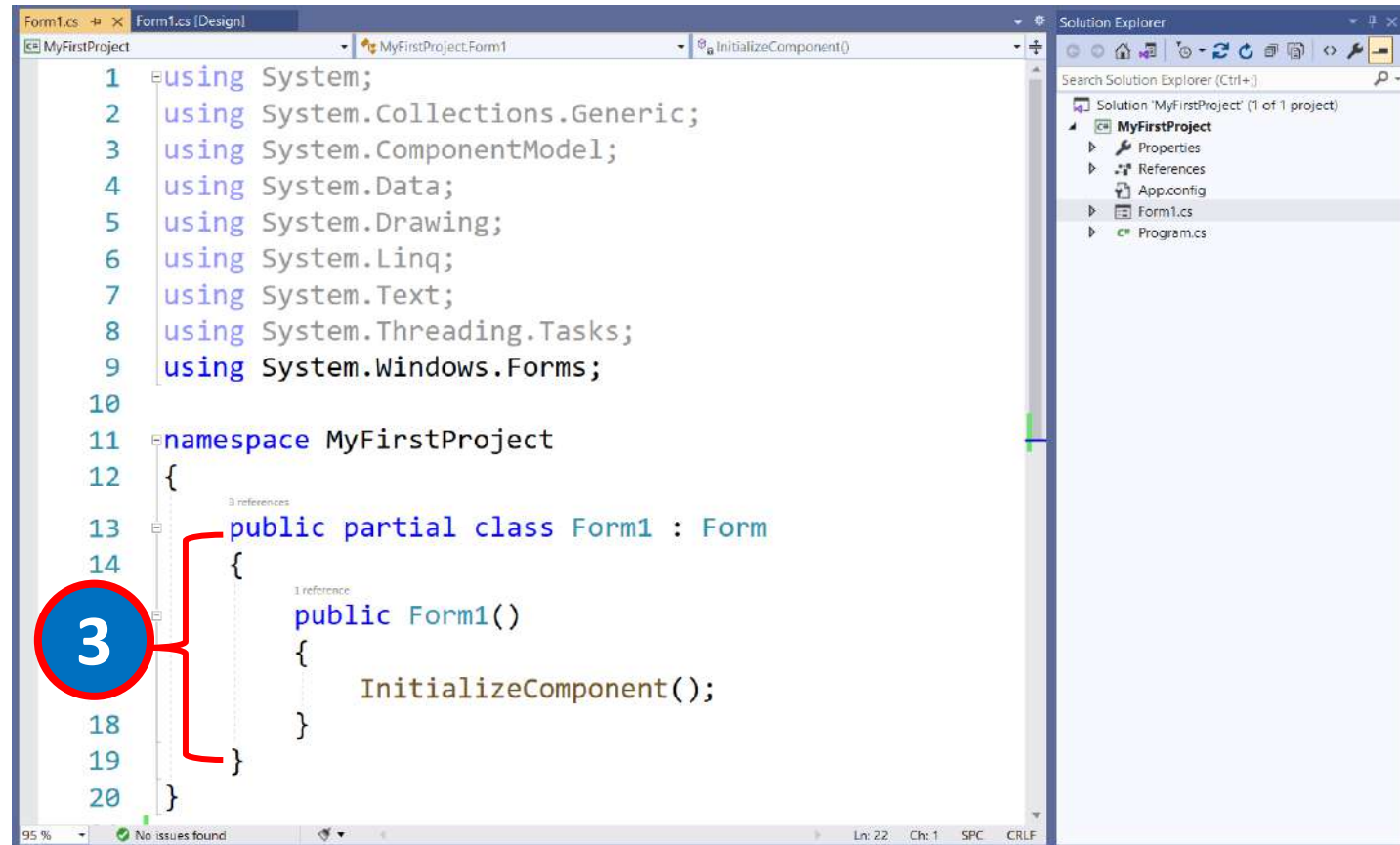
2) This section of code creates a namespace for the project. The line that reads namespace MyFirstProject marks the beginning of a namespace named MyFirstProject. Notice that the next line contains an opening brace { and that the last line in the file contains a corresponding closing brace }. All the code that appears between these braces is inside the MyFirstProject namespace.



```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19     }
20 }
```

Introduction to C# Code

3) This section of code is a class declaration. The line that reads `public partial class`, and so forth, marks the beginning of the class. The next line contains an opening brace `{`, and the last line in this section of code contains a corresponding closing brace `}`. All the code that appears between these braces is inside the class.



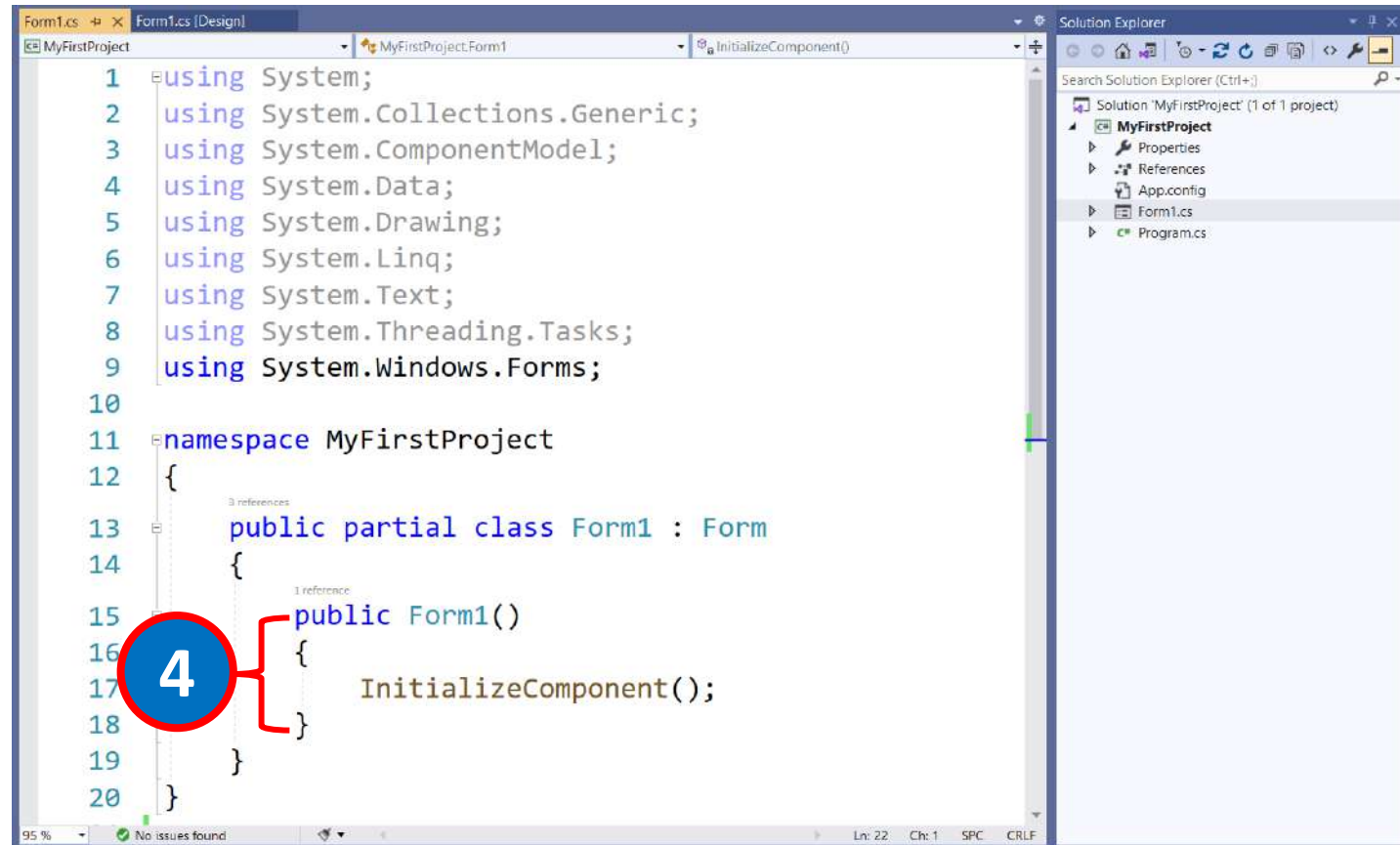
```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19     }
20 }
```

The screenshot shows the Visual Studio IDE with the file `Form1.cs` open. The code editor displays the C# code for a Windows Form. A red circle with the number 3 is placed over the class declaration section, which includes the `using` statements, the `namespace` declaration, and the `public partial class Form1` declaration with its opening and closing braces. The `InitializeComponent()` method call is also visible within the class body. The Solution Explorer on the right shows the project structure, including `Form1.cs`.

Introduction to C# Code

- 4) This section of code is a method. The line that reads `public Form1()` marks the beginning of the method. This is known as the entry point when the app begins execution.

The next line contains an opening brace `{`, and the last line in this section of code contains a corresponding closing brace `}`. The code that appears between these braces is inside the method.



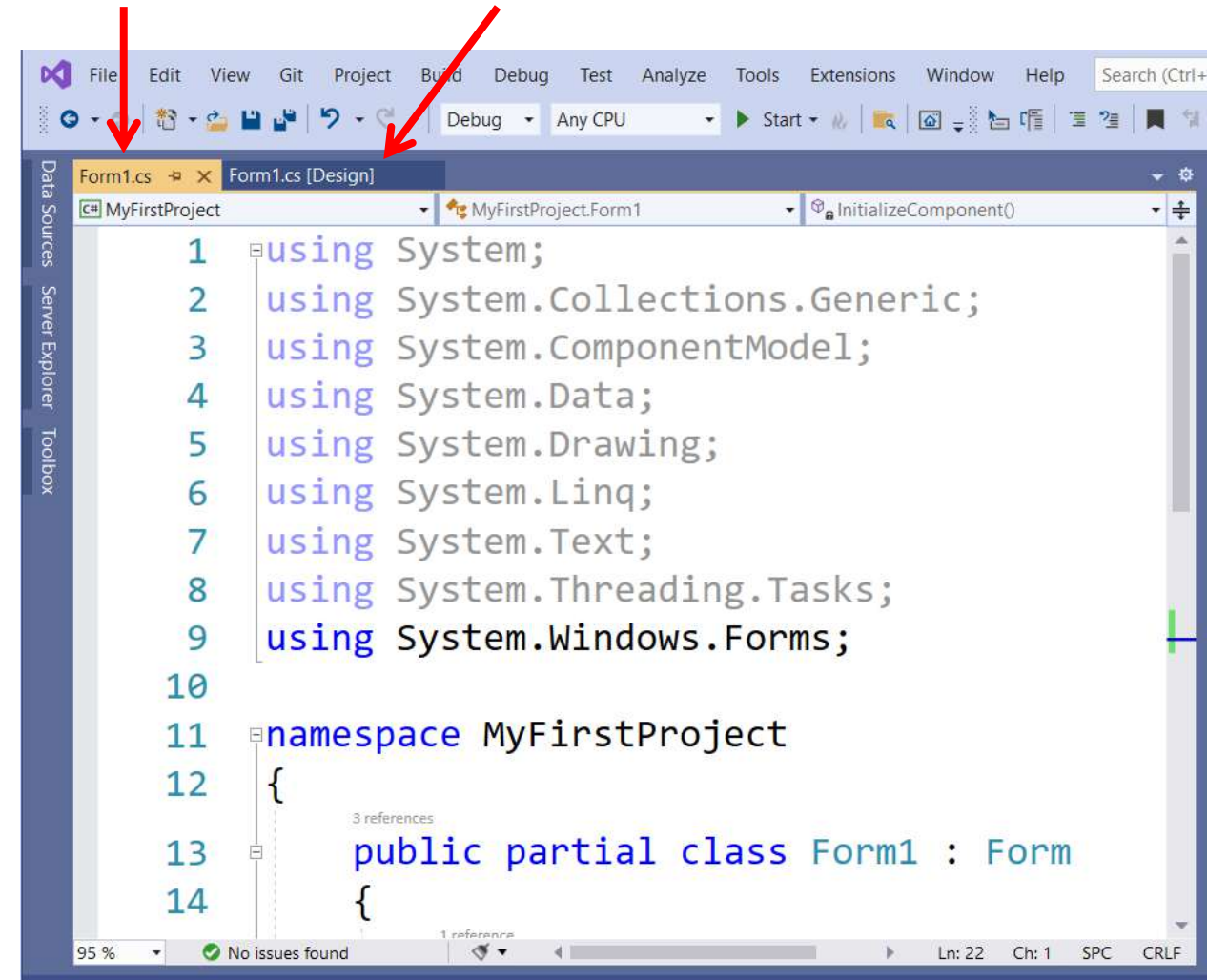
```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19     }
20 }
```

Switching between Code Editor and Designer

- When you open the code editor, it appears in the same part of the screen as the Designer.
- One way to quickly switch between the two windows is to use the tabs shown in the figure.
- Notice that the leftmost tab reads Form1.cs. That is the tab for the code editor. The rightmost tab reads Form1.cs [Design]. That is the tab for the Designer.

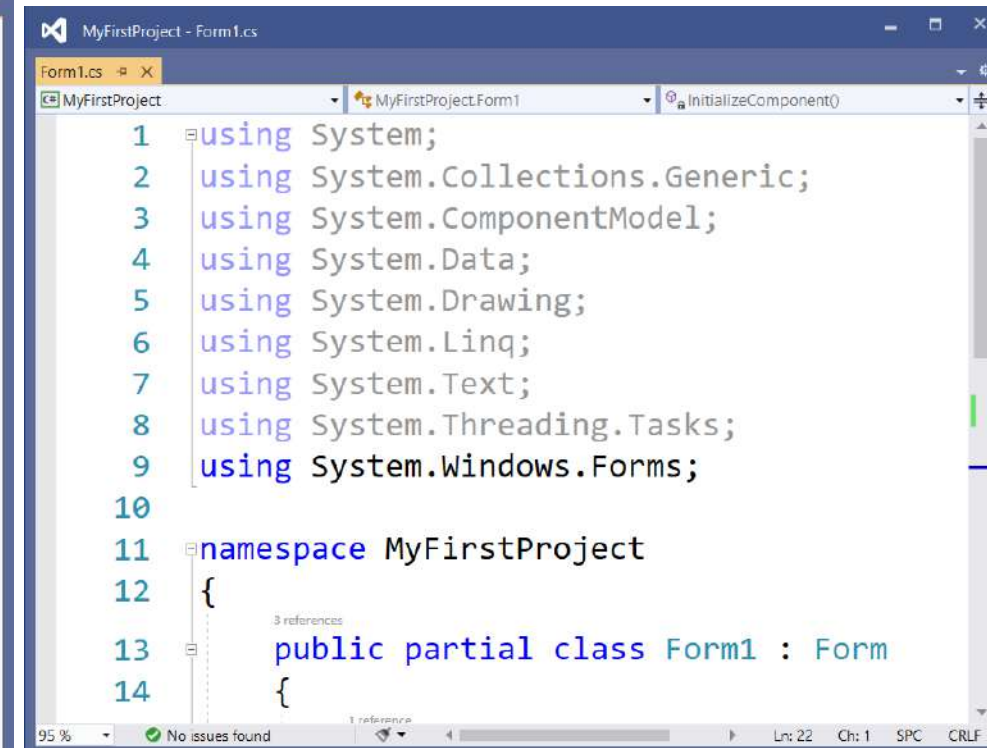
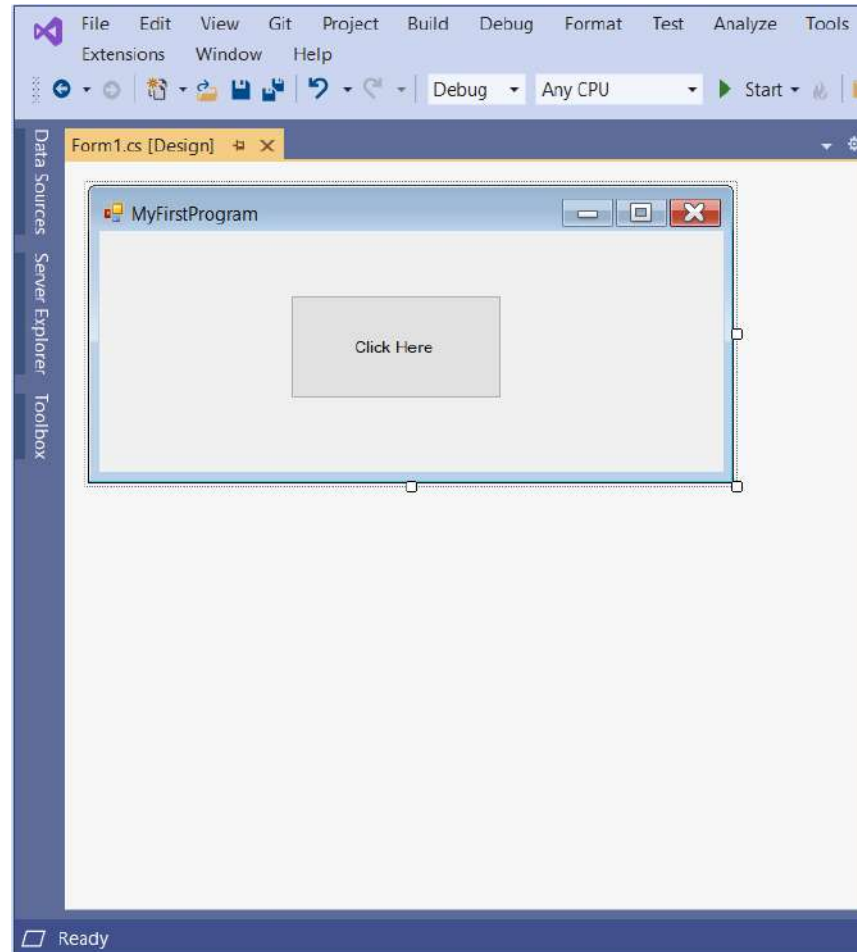
Code editor tab

Designer tab



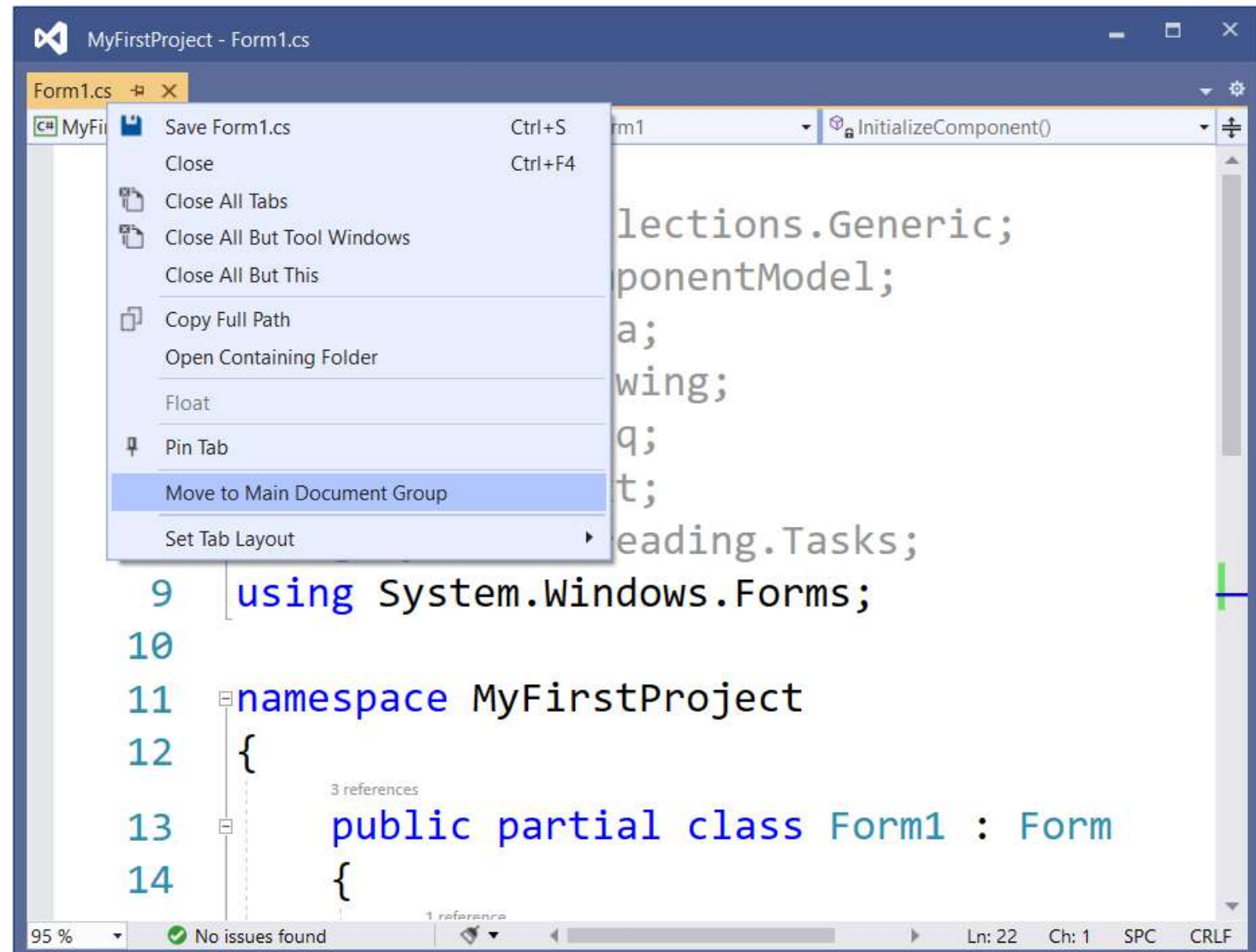
Switching between Code Editor and Designer

- You can also detach the code editor and move it to another part of the screen.
- This allows you to see the code editor and the Designer at the same time.
- Click the code editor tab and drag it to the desired location on the screen.



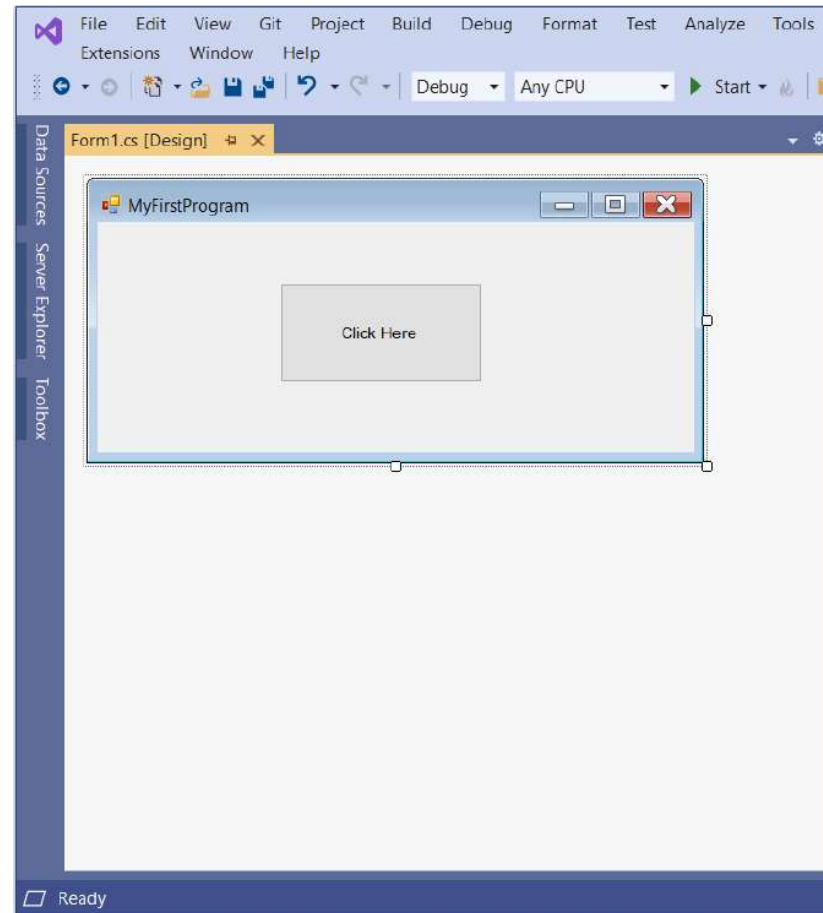
Switching between Code Editor and Designer

- To return the code editor to its position within the IDE, right-click the tab for the source code file in the code editor window and select Move to Main Document Group. This is shown in the Figure.

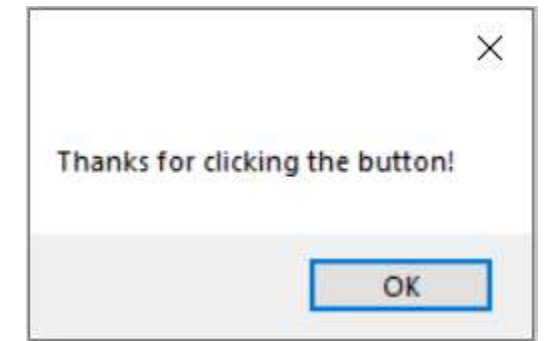
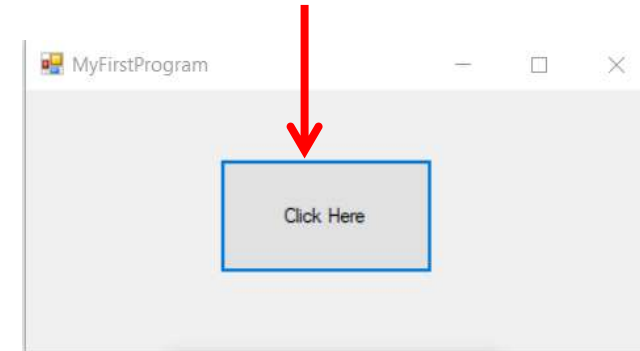


Adding Your Own Code to a Project

- Suppose that we want the application to display the message “Thanks for clicking the button!” when the user clicks the button.
- Write a special type of method known as an event handler.
- An **event handler** is a method that executes when a specific event takes place while an application is running.
- In this project, an event handler will execute when the user clicks the displayButton control.



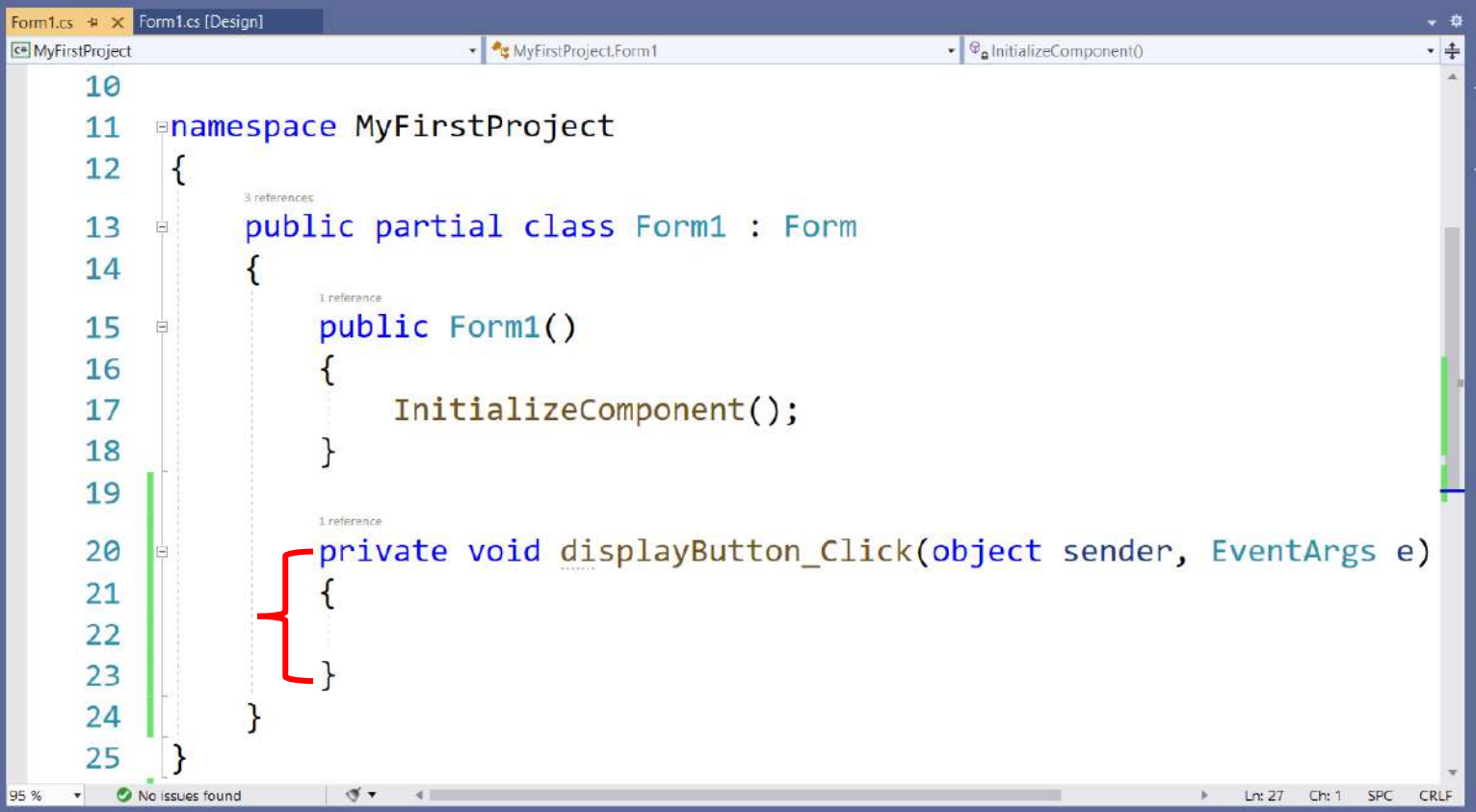
When you click this button



This window will appear

Adding Your Own Code to a Project

- To create the event handler, double-click the displayButton control in the Designer.
- This opens the Form1.cs file in the code editor, as shown in the figure, with some new code added to it.
- The event handler's name is displayButton_Click.



```
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19
20         private void displayButton_Click(object sender, EventArgs e)
21         {
22         }
23     }
24 }
25
```

Adding Your Own Code to a Project

- The `displayButton` portion of the name indicates that the event handler is associated with the `displayButton` control, and the `Click` portion of the name indicates that the event handler responds to `Click` events.
- When you see the name `displayButton_Click`, you understand that it is an event handler that executes when a `Click` event occurs on the `displayButton` control.

```
10
11 namespace MyFirstProject
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19
20         private void displayButton_Click(object sender, EventArgs e)
21         {
22         }
23     }
24 }
25
```

The event handler's name

The code goes here between braces

Adding Your Own Code to a Project

- The event handler that Visual Studio generates doesn't actually do anything.
- You can think of it as an empty container to which you can add your own code.
- Notice that the second line of the event handler is an opening brace { and the last line is a closing brace }.
- Any code that you want executed when the user clicks the displayButton control must be written between these braces.
- The code that displays the message "Thanks for clicking the button!" in a message box, which is a small pop-up window is:

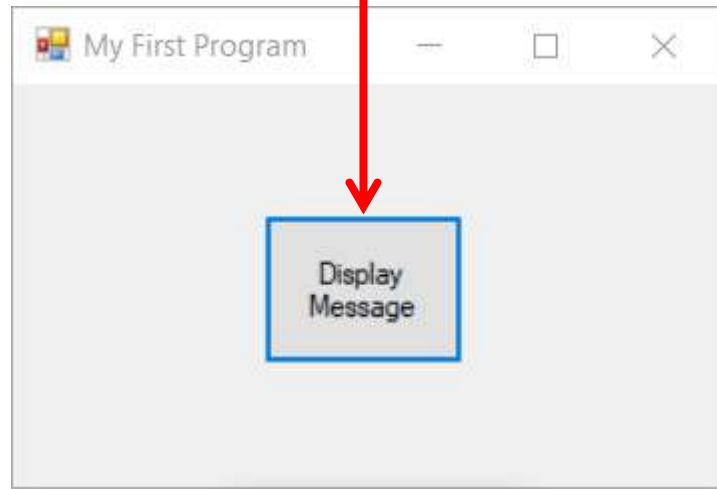
```
MessageBox.Show("Thanks for clicking the button!");
```

- When you call the MessageBox.Show method, you write a string of characters inside the parentheses.

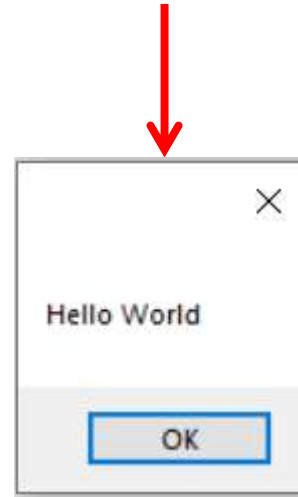
Creating the GUI for First Hello World C# App

- The process of creating this application is divided into two parts:
 - Create the application's GUI.
 - Write the code that causes the Hello World message to appear when the user clicks the Display Message button.

When you click this button



This window will appear



Creating the GUI for First Hello World C# App

■ **Part 1:** Creating the GUI for the Hello World application

- 1) Start Visual Studio.
- 2) Start a new project by clicking File on the menu bar and then selecting New Project. . .
- 3) The New Project window should appear. At the left side of the window, under Installed > Templates, make sure Visual C# is selected. Then, select Windows Forms Application as the type of application. In the Name text box (at the bottom of the window), change the name of the project to HelloWorldApp, and then click the Create button.

Configure your new project

Windows Forms App (.NET Framework) C# Windows Desktop

Project name
HelloWorldApp

Location
D:\Courses\C#\My Lectures on C#\Programming 2\Projects\

Solution
Create new solution

Solution name ⓘ
HelloWorldApp

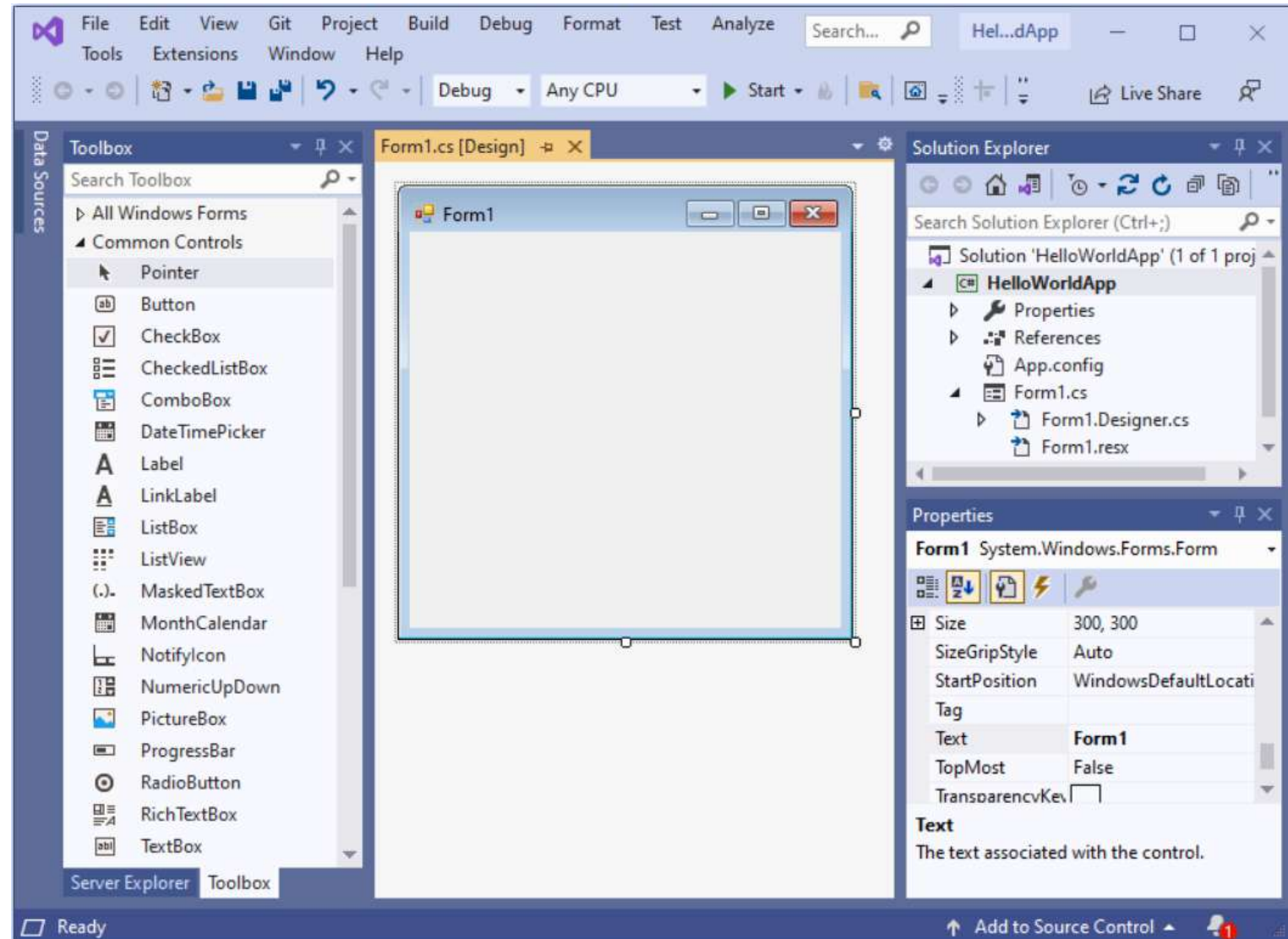
☒ Place solution and project in the same directory

Framework
.NET Framework 4.7.2

Back Create

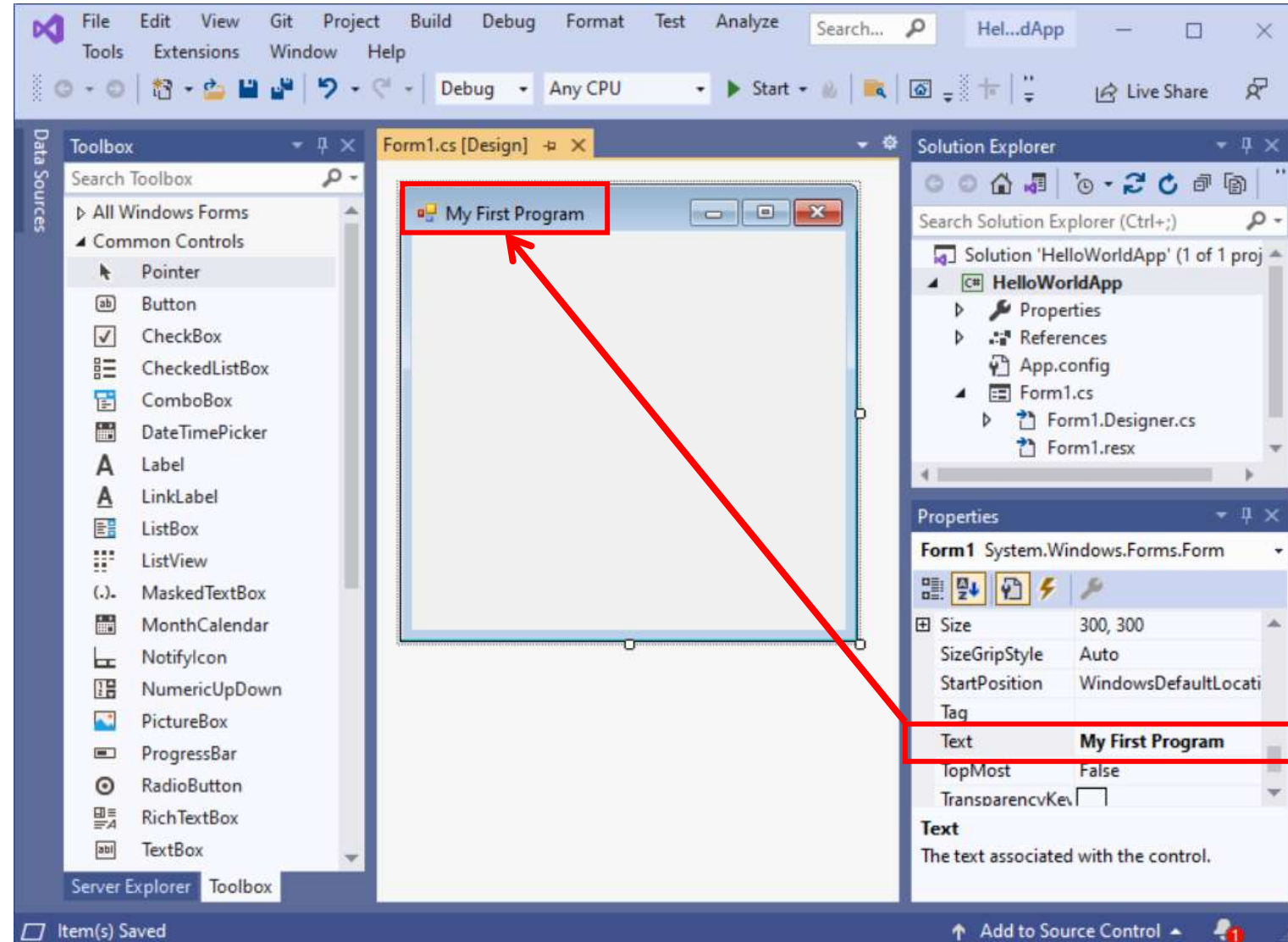
Creating the GUI for First Hello World C# App

- 4) Make sure the Toolbox, the Solution Explorer, and the Properties window are visible and that Auto Hide is turned off for each of these windows. The Visual Studio environment should appear as shown in the figure.



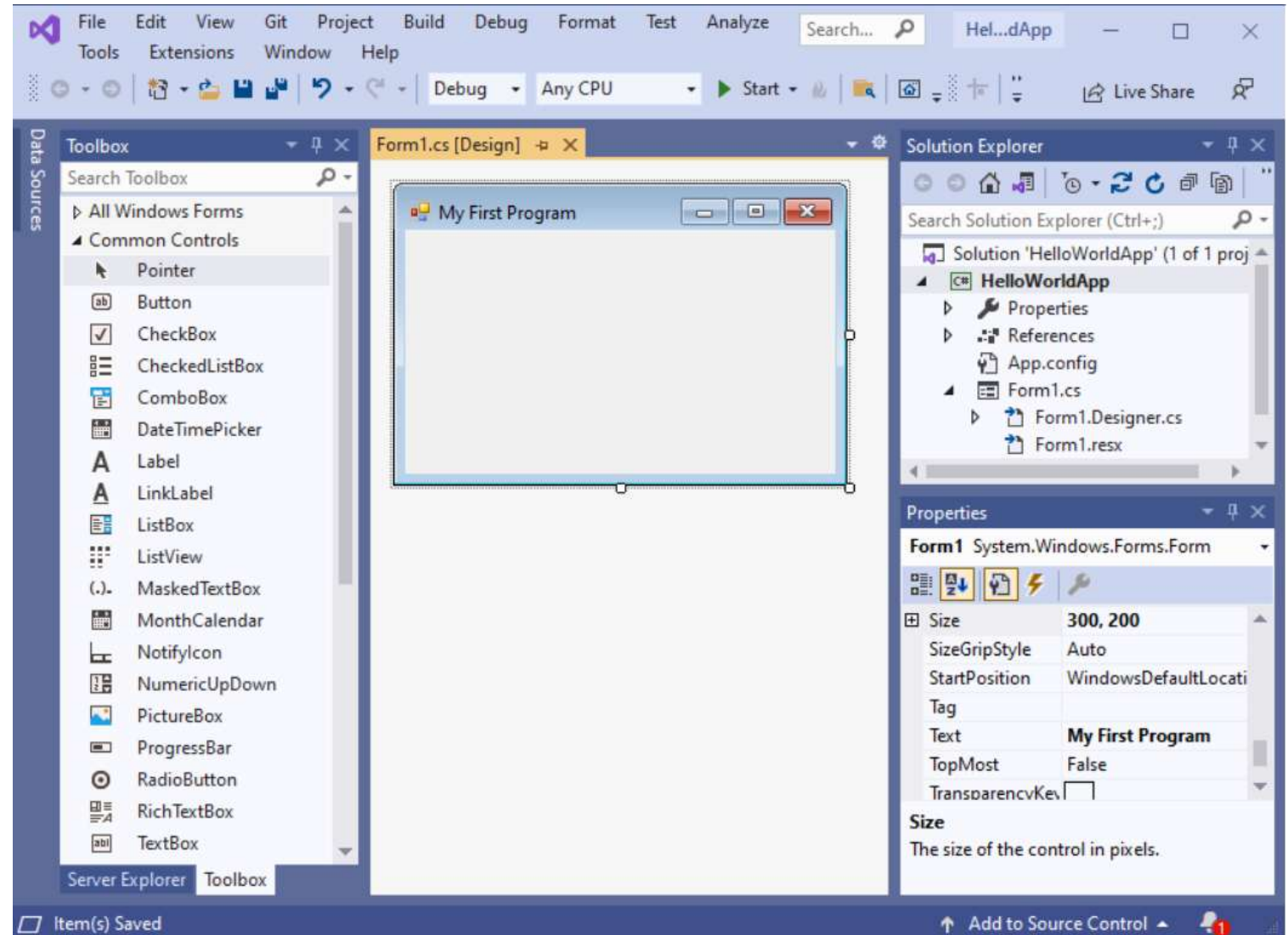
Creating the GUI for First Hello World C# App

- 5) Change the Form1 form's Text property to My First Program, as shown in the figure.



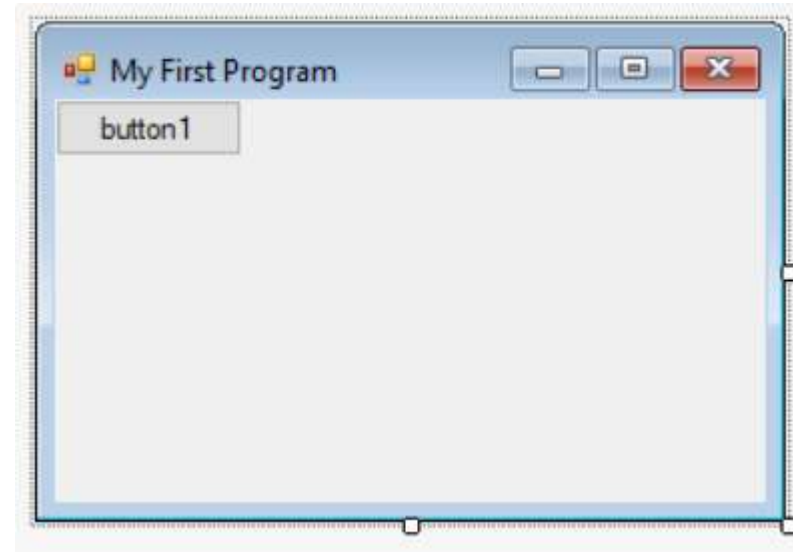
Creating the GUI for First Hello World C# App

- 6) The form's default size is too large for this application, so you need to make it smaller. Use the technique previously discussed to adjust the form's size with the mouse. The form should appear similar to that shown in the figure.

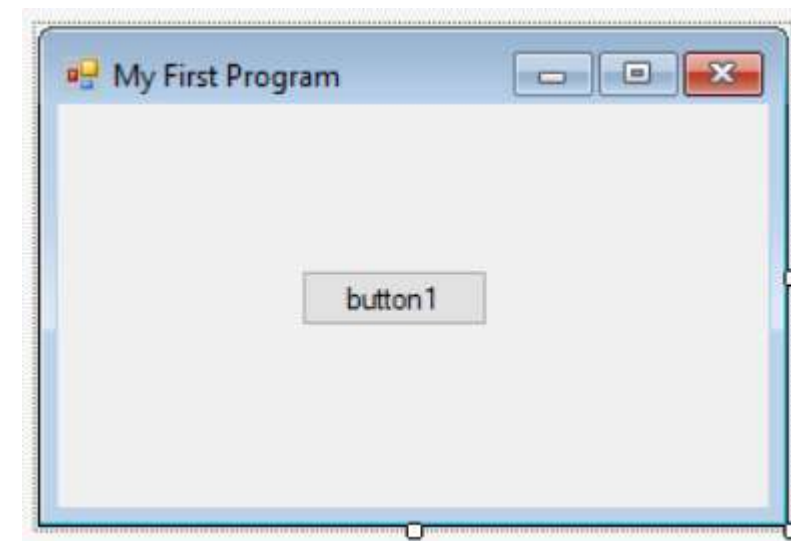


Creating the GUI for First Hello World C# App

7) Now you are ready to add a Button control to the form. Locate the Button tool in the Toolbox and double-click it. A Button control should appear on the form, as shown in the figure.

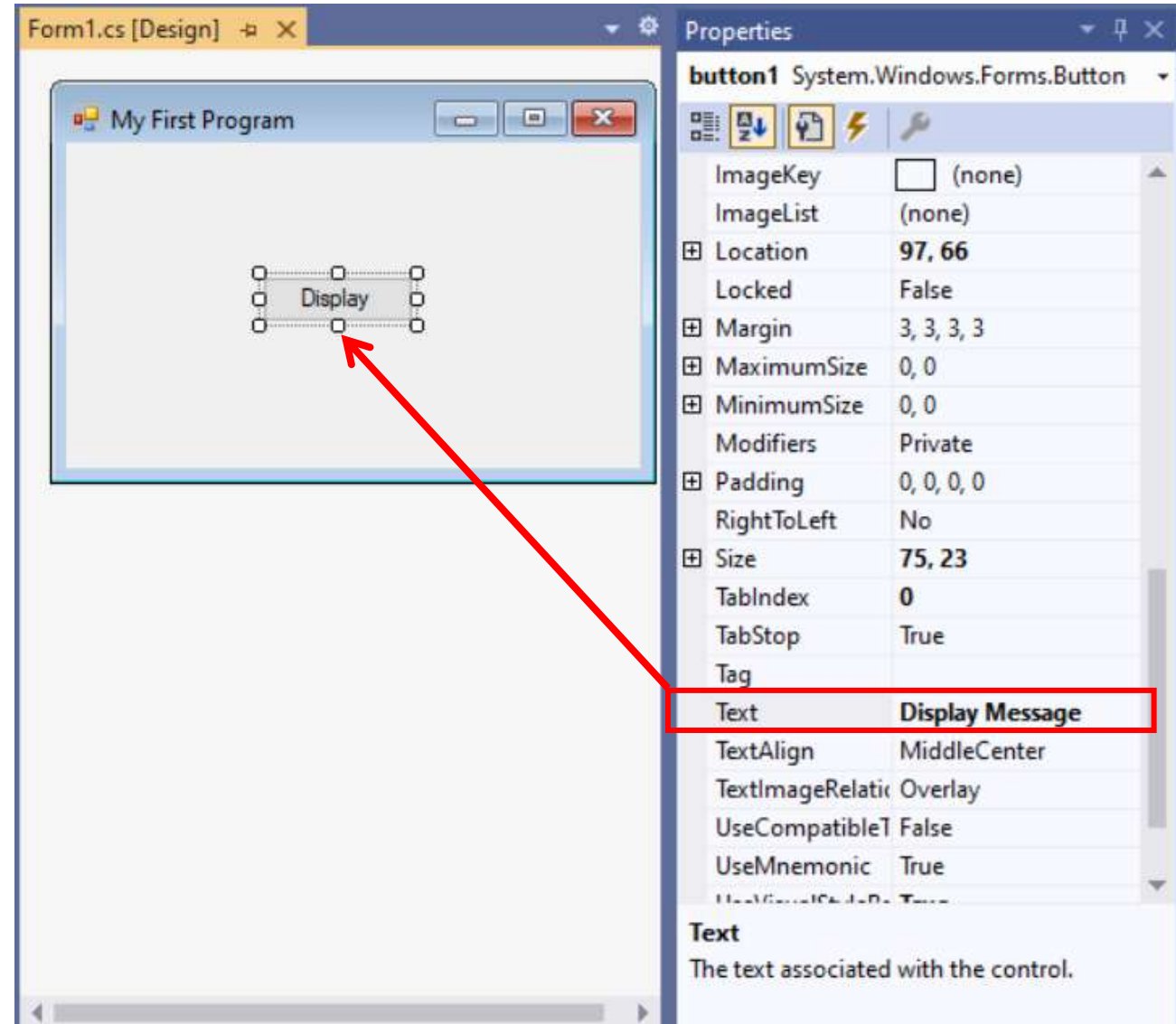


8) Move the Button control so it appears approximately in the center of the form, as shown in the figure.



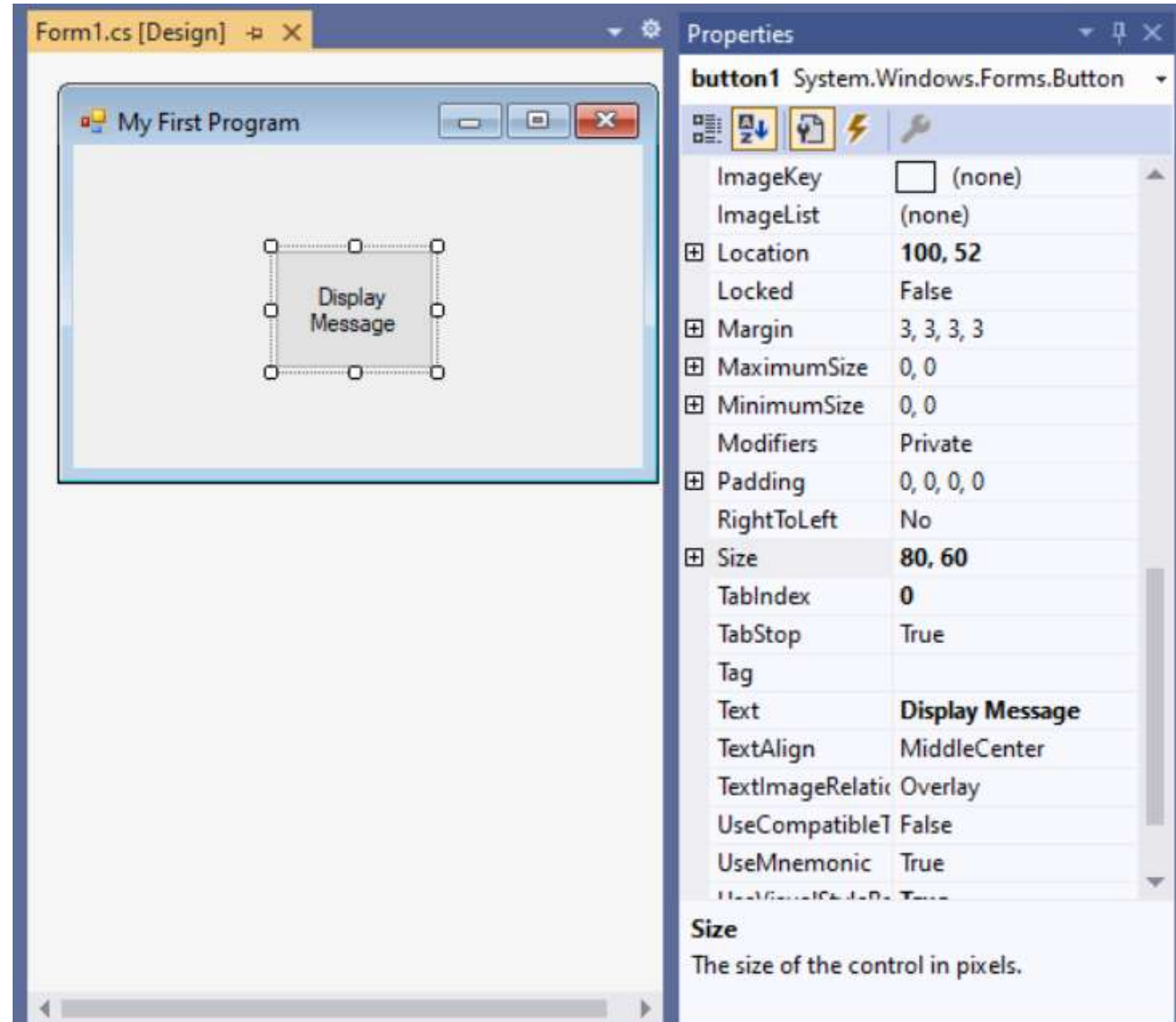
Creating the GUI for First Hello World C# App

- 9) Change the value of the Button control's Text property to Display Message. After doing this, notice that the text displayed on the button has changed, as shown in the figure.



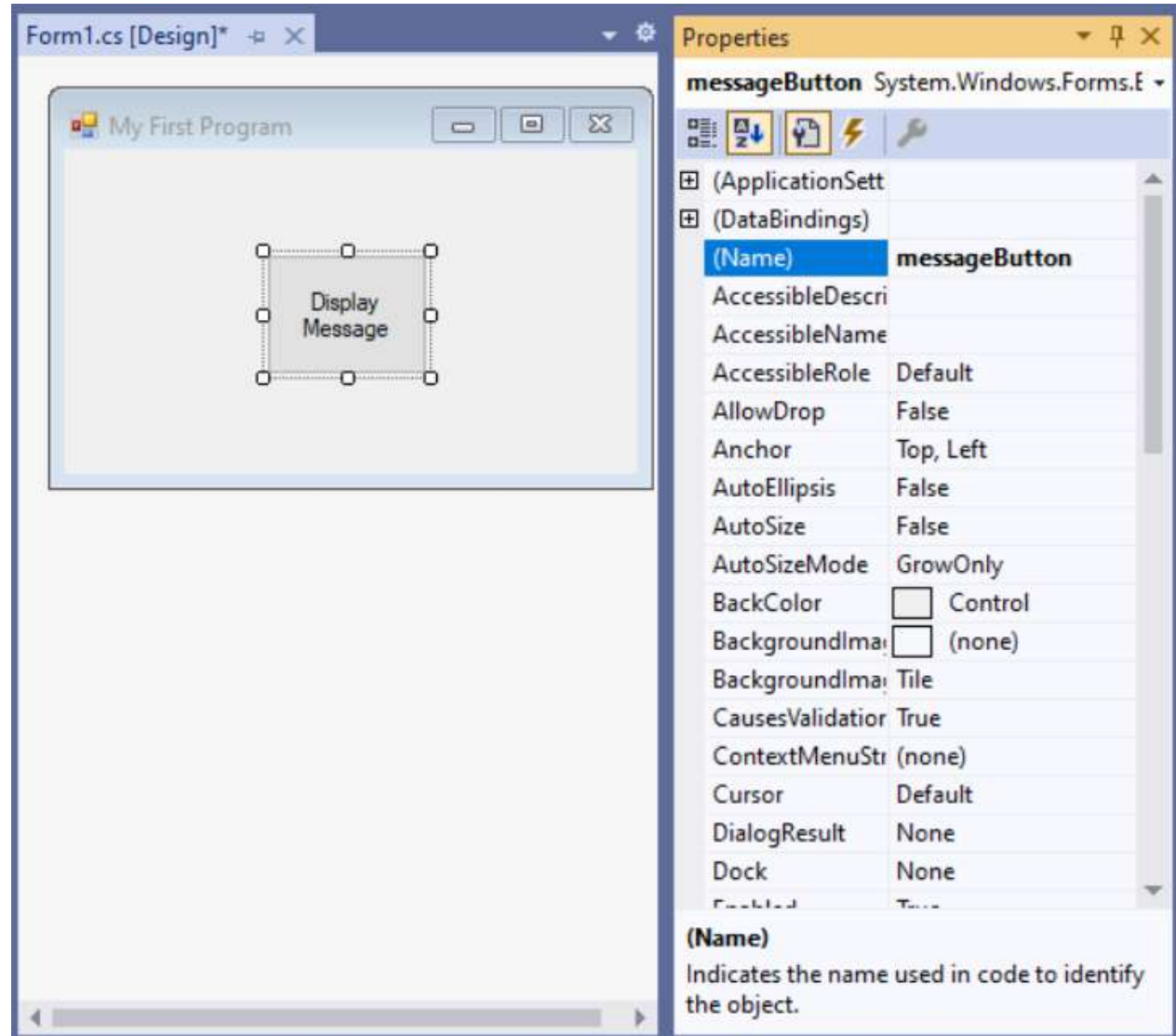
Creating the GUI for First Hello World C# App

10) The Button control isn't quite large enough to accommodate all of the text that you typed into its Text property, so enlarge the Button control, as shown in the figure.



Creating the GUI for First Hello World C# App

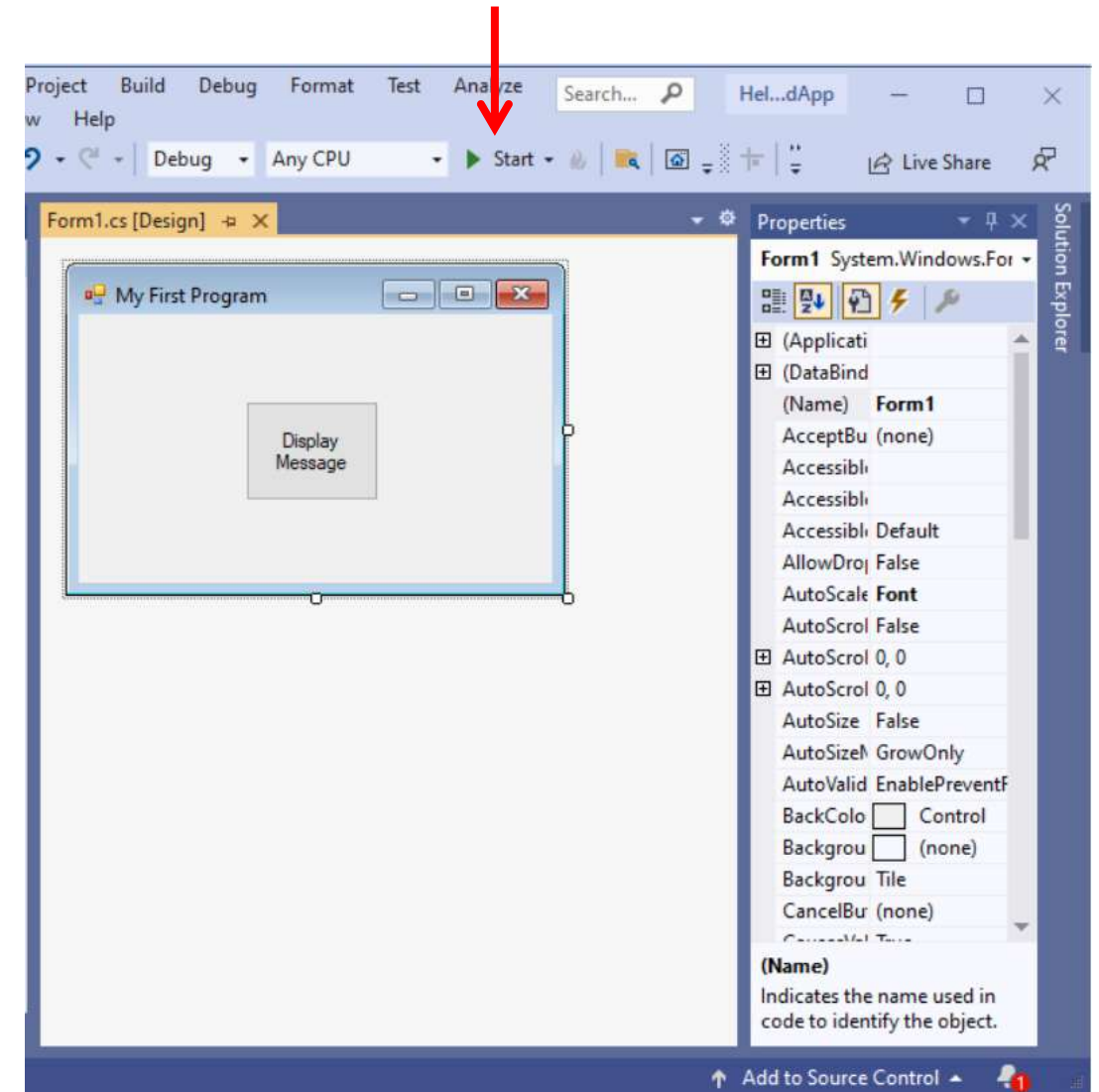
11) As mentioned before, a control's name should reflect the purpose of the control. The Button control that you created in this application will cause a message to be displayed when it is clicked. The name `button1` does not convey that purpose. Change the Button control's Name property to `messageButton`. The Properties window should appear as shown in the figure.



Creating the GUI for First Hello World C# App

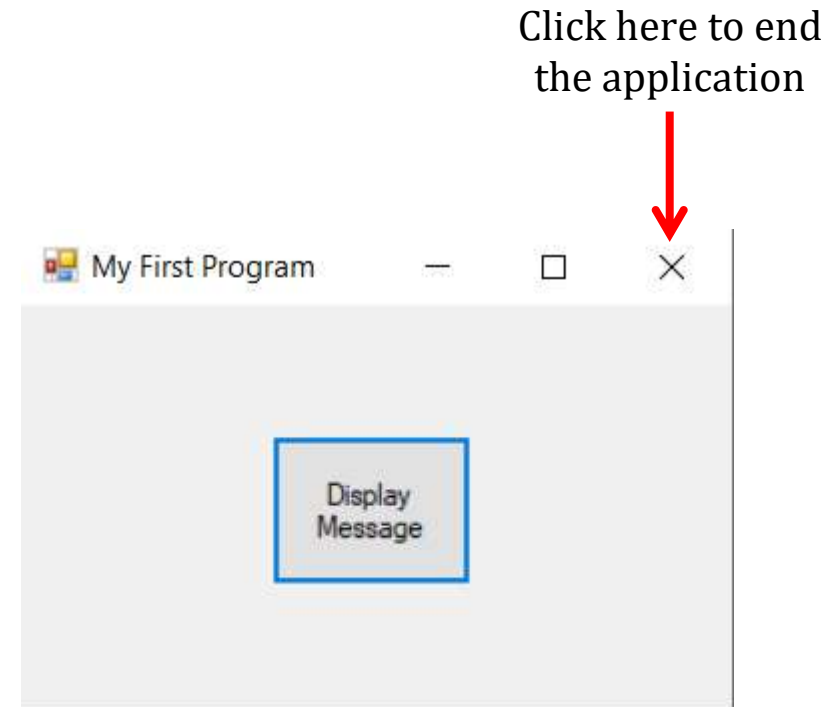
12) Click File on the Visual Studio menu bar and then click Save All to save the project.

13) You're only partially finished with the application, but you can run it now to see how the GUI looks on the screen. To run the application, press the F5 key on the keyboard or click the Start Debugging button (▶) on the toolbar. This causes the application to be compiled and executed. You will notice the appearance of the Visual Studio environment change somewhat, and you will see the application's form appear on the screen as shown in the figure.



Creating the GUI for First Hello World C# App

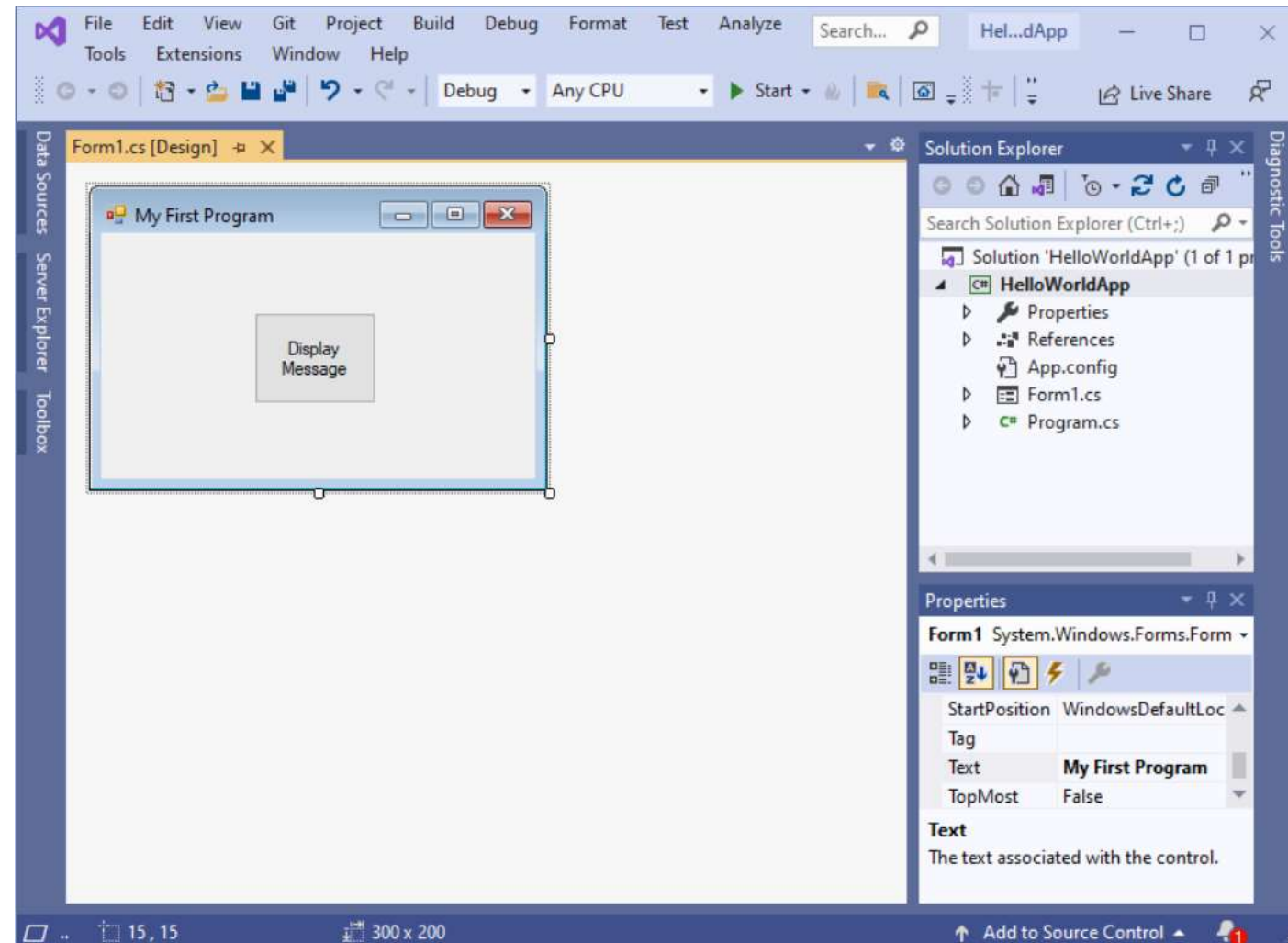
- Although the application is running, it is not capable of doing anything other than displaying the form.
- If you click the Display Message button, nothing will happen. That is because you have not yet written the code that executes when the button is clicked.
- You will do that in the next tutorial. To end the application, click the standard Windows close  button in the form's upper-right corner.



Writing Code for the Hello World Application

■ Part 2: Writing Code for the Hello World application

- 1) Make sure the Form1 form is visible in the Designer, as shown in the figure. If it is not, right-click Form1.cs in the Solution Explorer and then select View Designer from the pop-up menu.

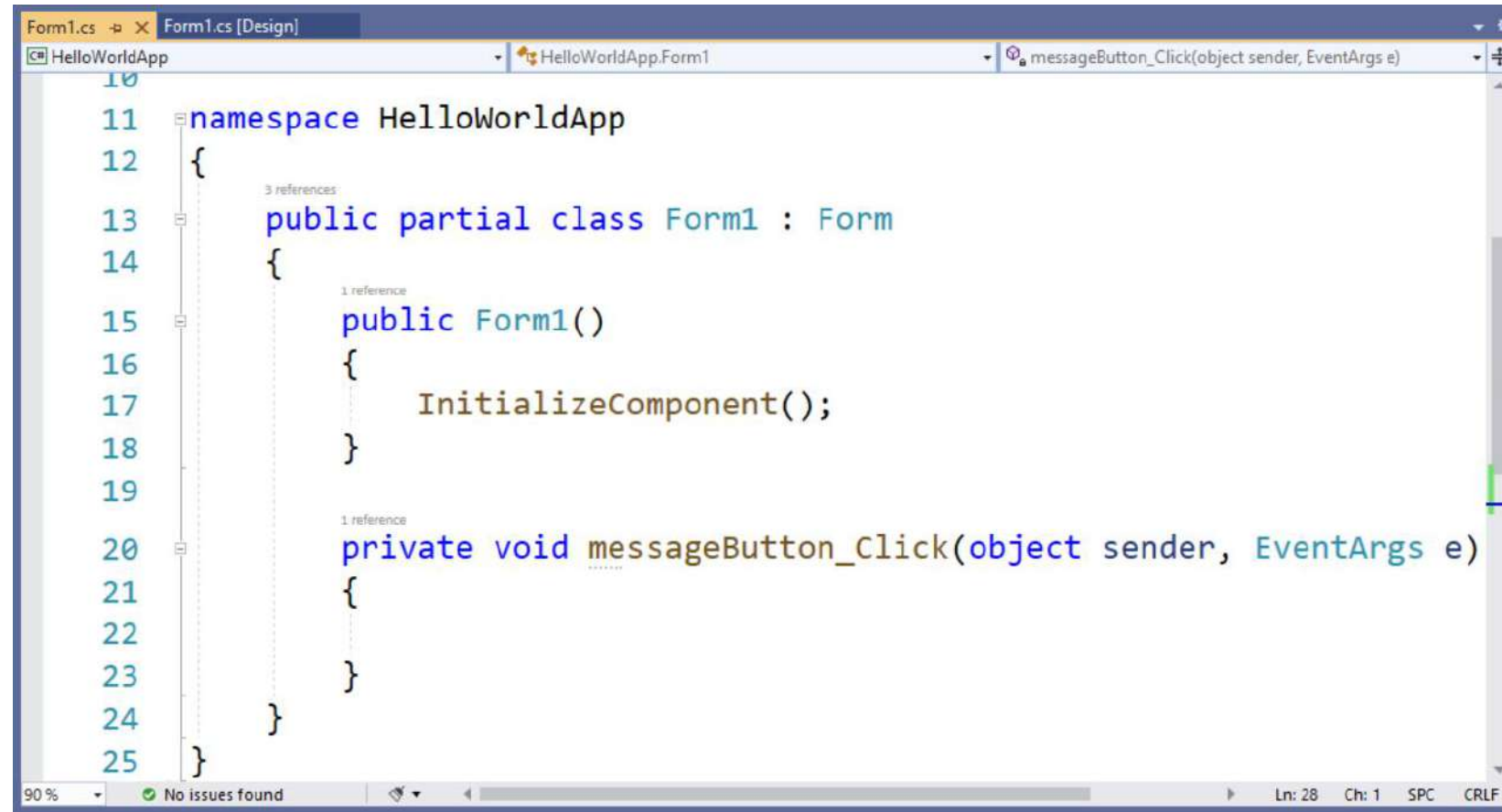


Writing Code for the Hello World Application

2) In the Designer, double-click the messageButton control.

This should cause the code editor to appear, as shown in the figure.

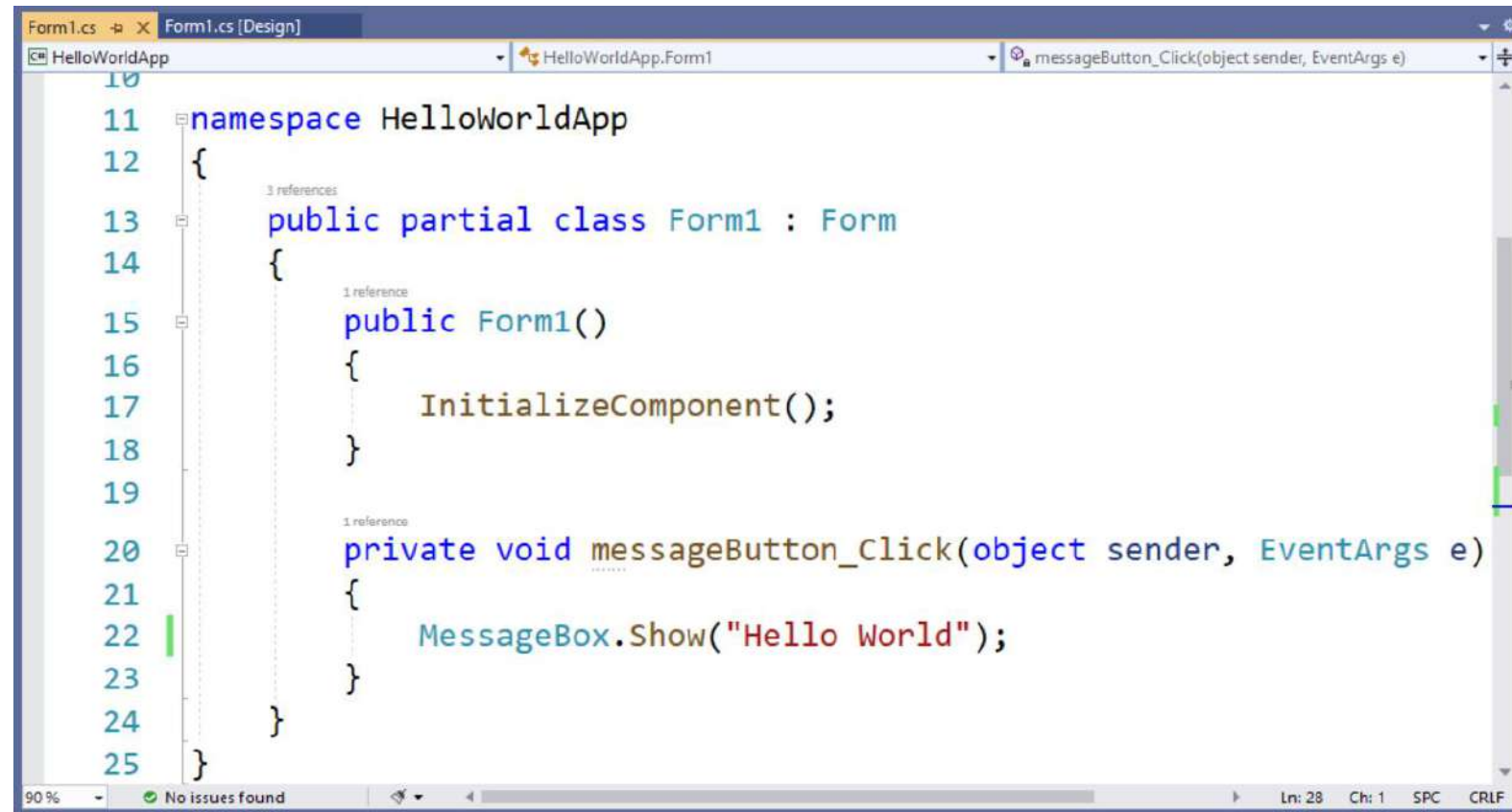
An empty event handler named messageButton_Click has been created.



```
10
11 namespace HelloWorldApp
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19
20         private void messageButton_Click(object sender, EventArgs e)
21         {
22         }
23     }
24 }
25
```

Writing Code for the Hello World Application

- 3) Inside the `messageButton_Click` event handler, type the following statement exactly as it is shown:
`MessageBox.Show("Hello World");`
- 4) Save the project.
- 5) Press the F5 key on the keyboard, or click the Start Debugging button () on the toolbar to compile and run the application.



The screenshot shows the Visual Studio IDE with the `Form1.cs` file open in the Design view. The code is as follows:

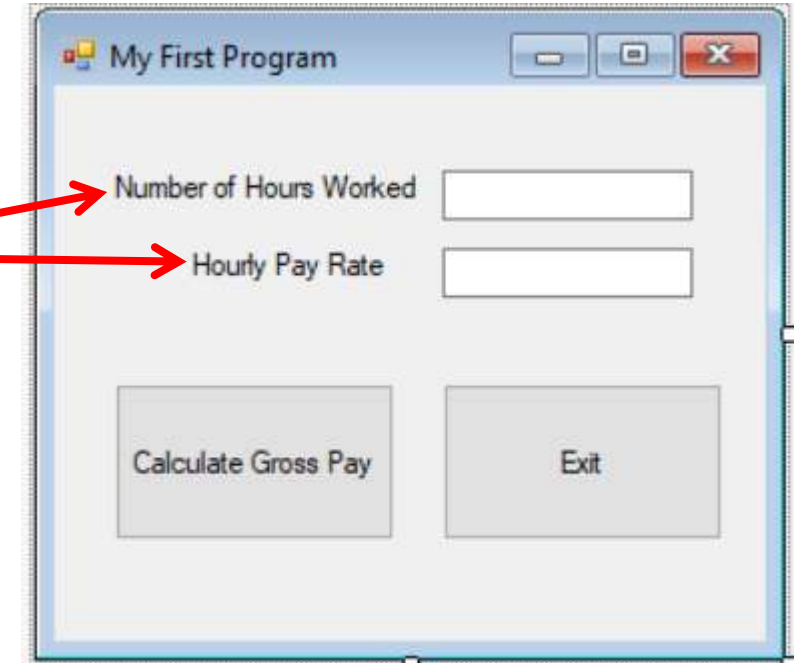
```
10
11 namespace HelloWorldApp
12 {
13     public partial class Form1 : Form
14     {
15         public Form1()
16         {
17             InitializeComponent();
18         }
19
20         private void messageButton_Click(object sender, EventArgs e)
21         {
22             MessageBox.Show("Hello World");
23         }
24     }
25 }
```

The status bar at the bottom indicates "No issues found" and the current position is "Ln: 28 Ch: 1 SPC CRLF".

Label Control

- A Label control displays text on a form.
- Label controls can be used to display unchanging text, or program output. An example of a form with two Label controls is shown in the figure.

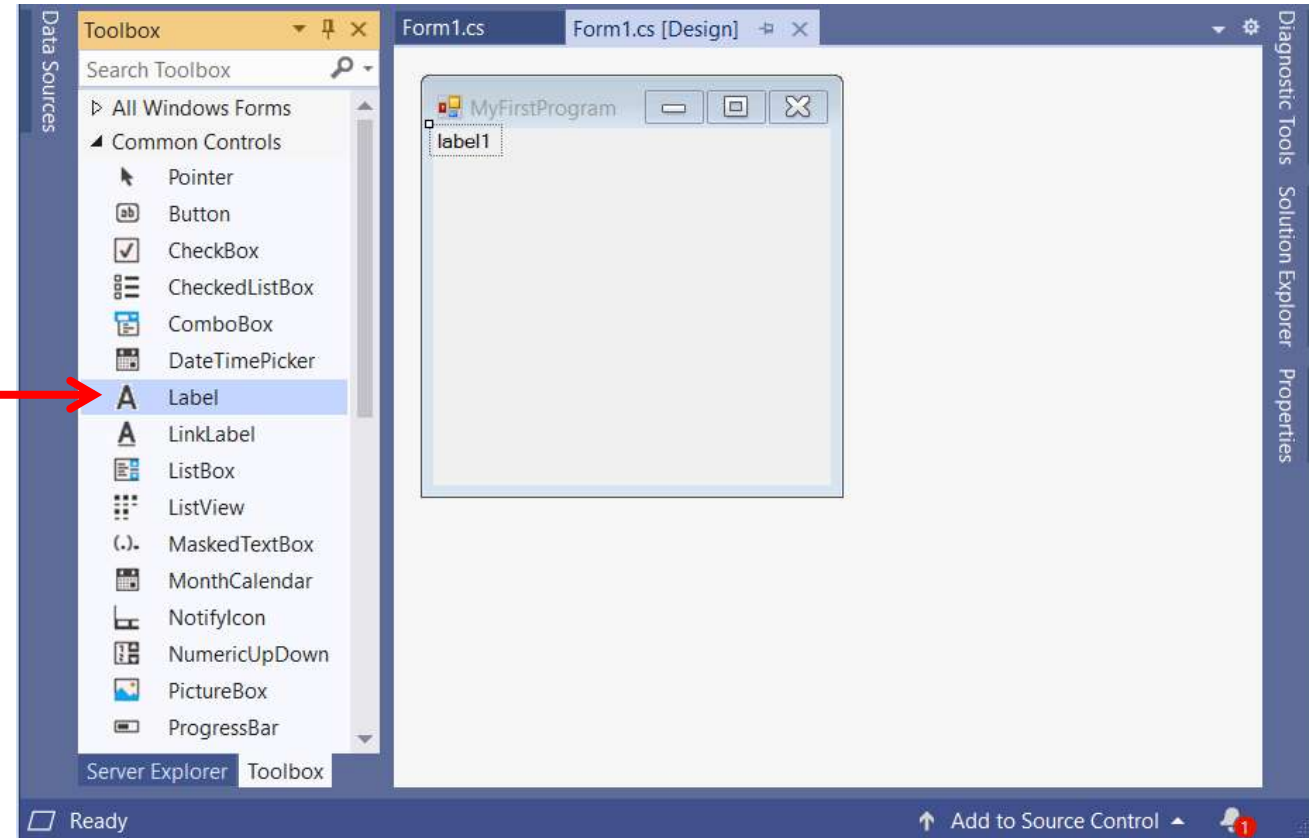
Label controls



The screenshot shows a Windows application window titled "My First Program". Inside the window, there is a form with two label controls and two buttons. The first label control is "Number of Hours Worked" followed by a text input field. The second label control is "Hourly Pay Rate" followed by another text input field. Below these, there are two buttons: "Calculate Gross Pay" and "Exit". Two red arrows originate from the text "Label controls" and point to the "Number of Hours Worked" and "Hourly Pay Rate" labels, highlighting them as examples of label controls.

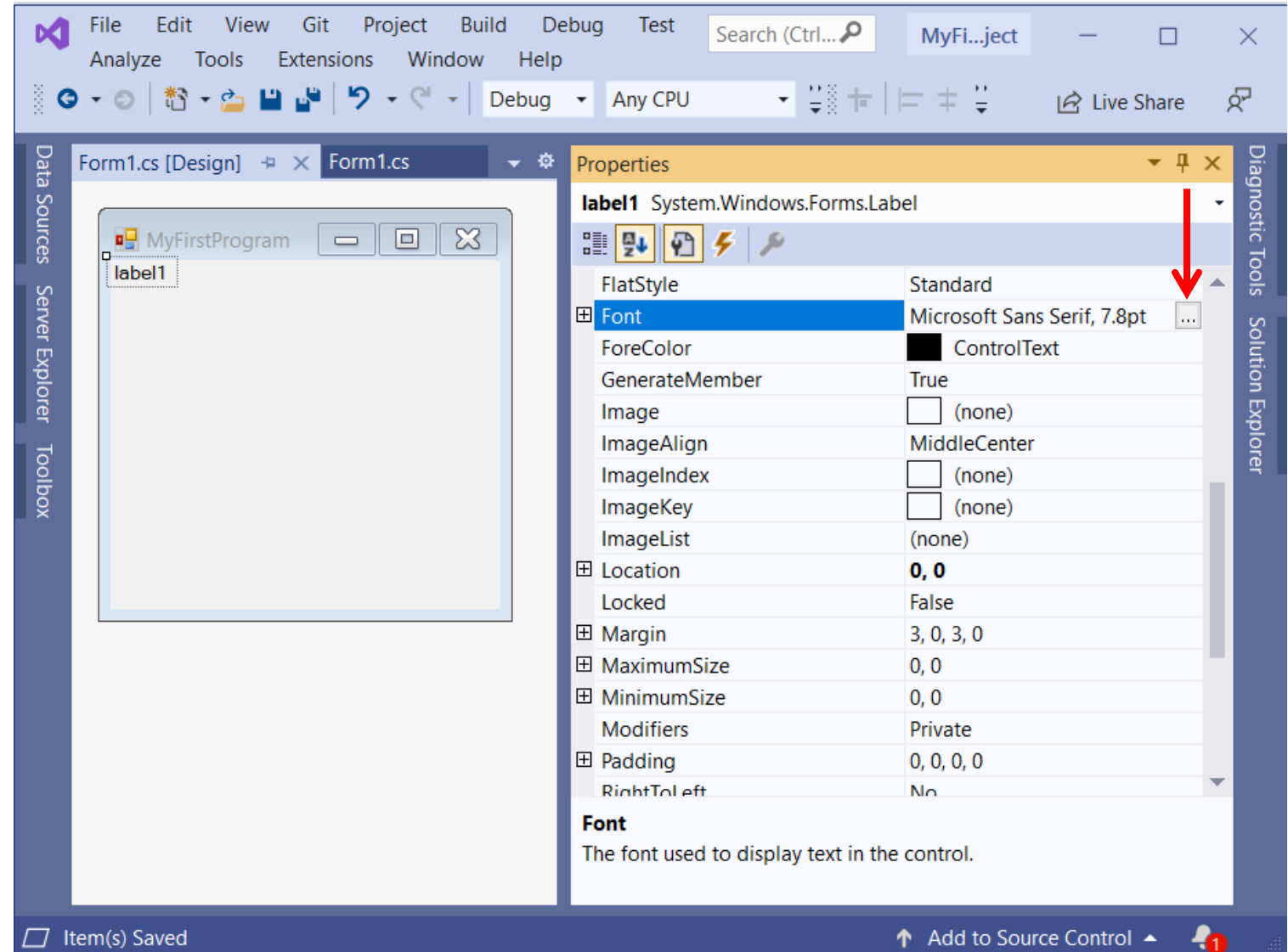
Label Control

- You'll find the Label control tool in the Common Controls group of the Toolbox, as shown in the figure.
- To create a Label control on a form, you double-click the Label control tool in the Toolbox.



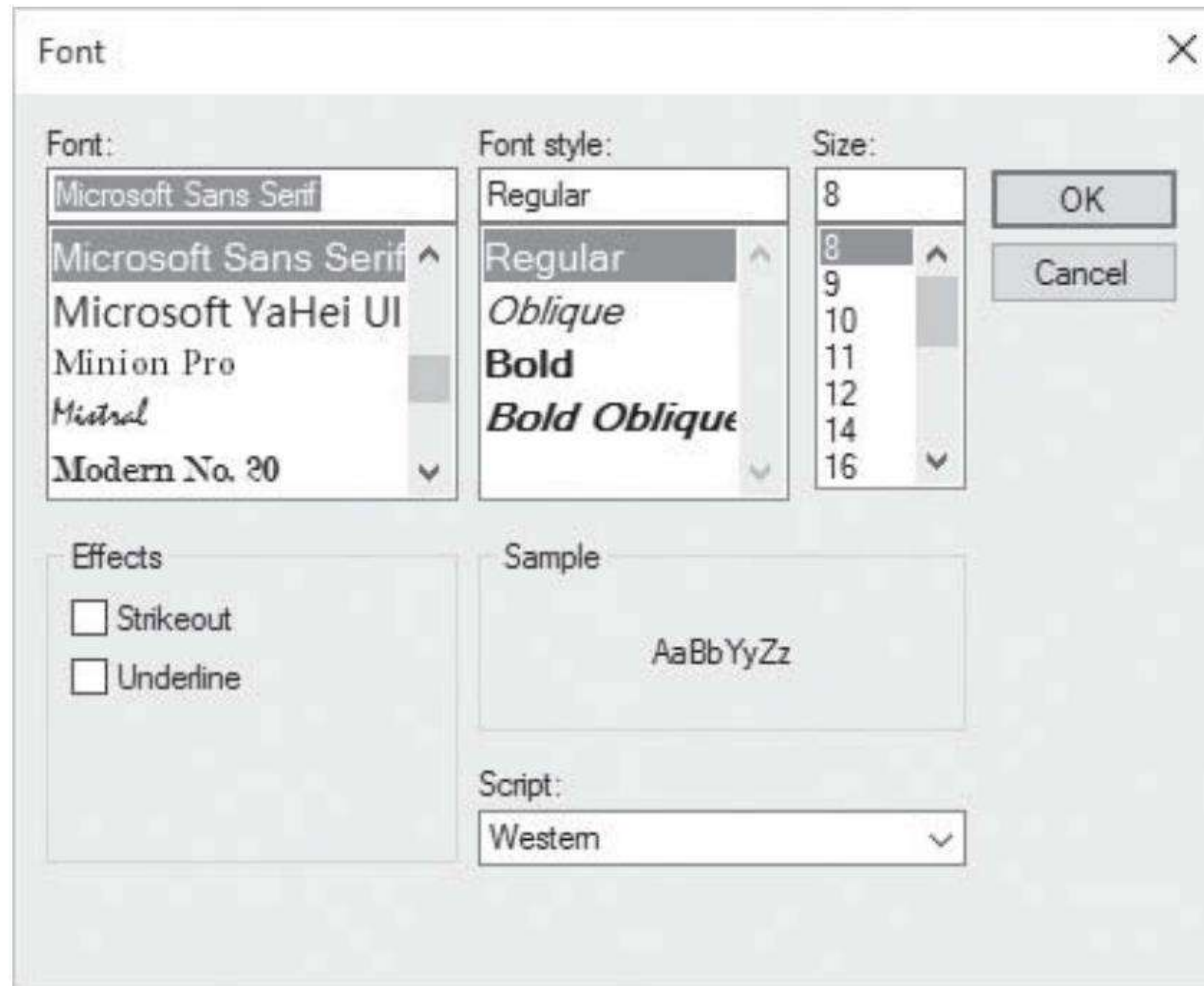
Label Control (Font Property)

- The Font property allows you to set the font, font style, and size of the control's text.
- When you select the Font property in the Properties window, an ellipses button (...) appears next to the property's value, as shown in the figure.



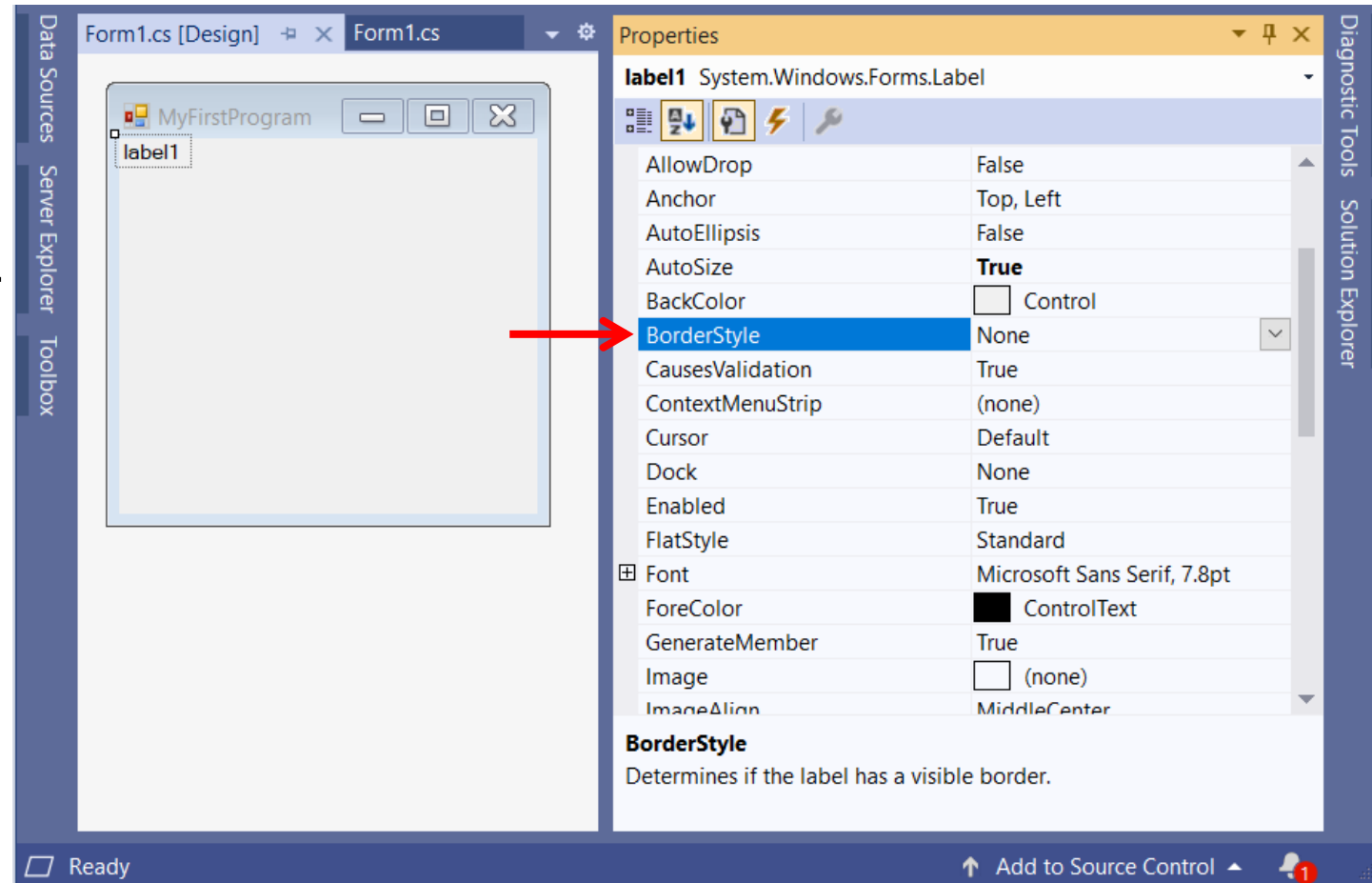
Label Control (Font Property)

- When you click the ellipses button, the Font dialog box appears, as shown in the figure.



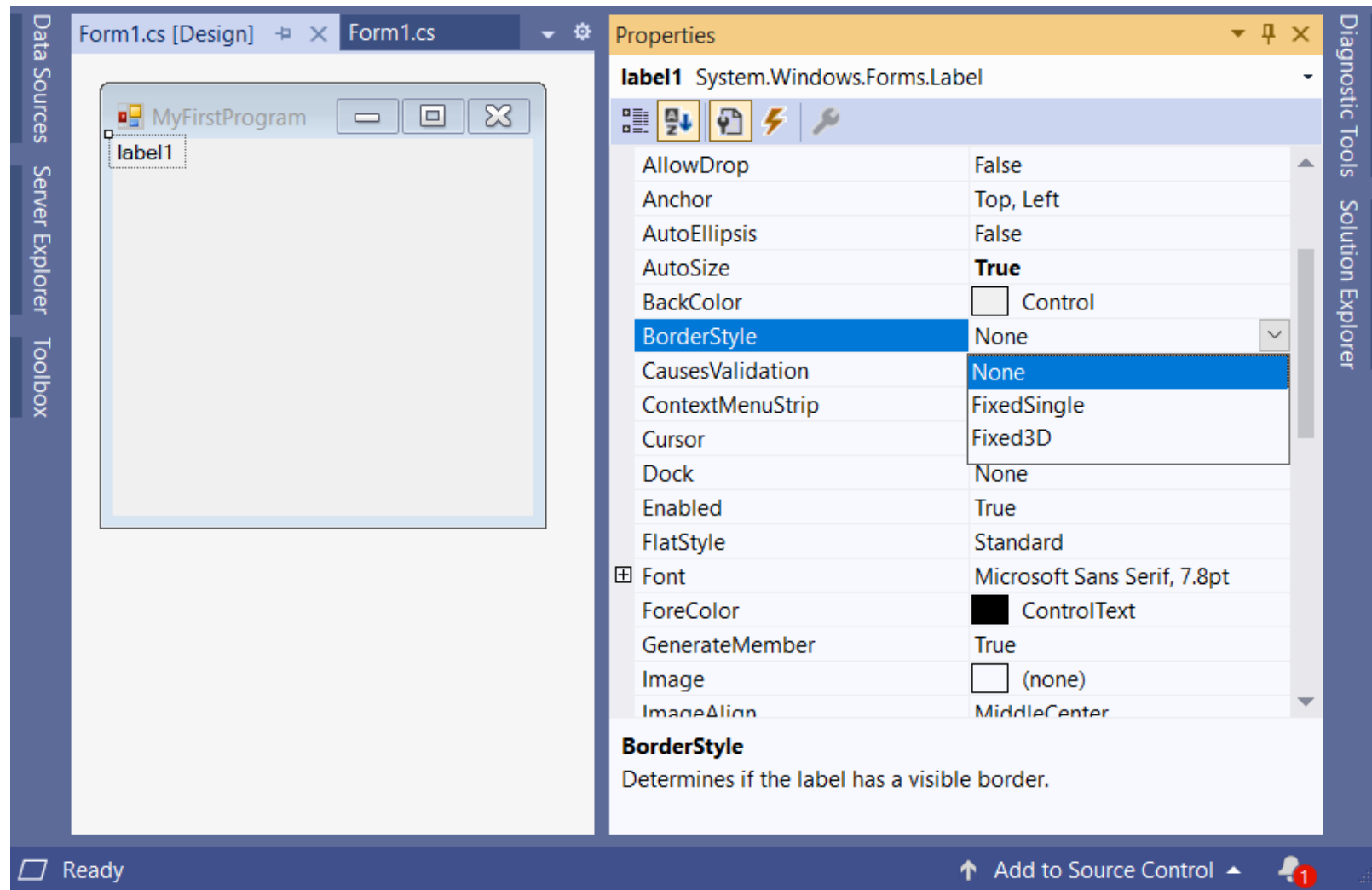
Label Control (BorderStyle Property)

- Label controls have a BorderStyle property that allows you to display a border around the control's text.
- The BorderStyle property may have one of three values:
 - None → No border will appear around the control's text, The property is set to **None** by default.
 - FixedSingle → The control's text will be outlined with a thin border.
 - Fixed3D → The control's text will have a recessed 3D appearance.



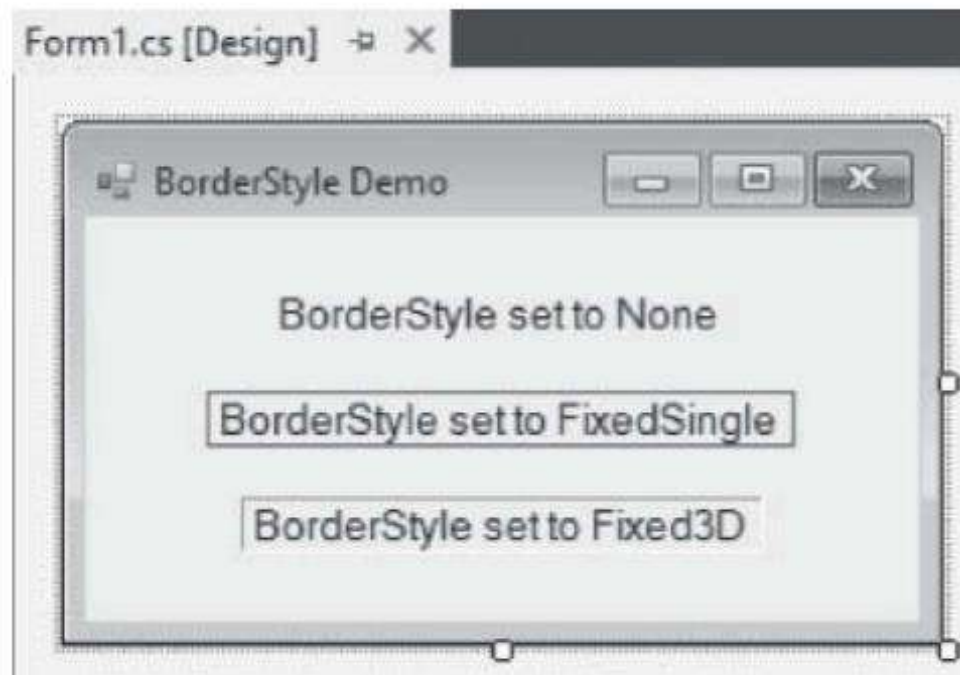
Label Control (BorderStyle Property)

- To change the BorderStyle property, select it in the Properties window and then click the down-arrow button () that appears next to the property's value.
- As shown in the figure, a drop-down list will appear containing the three possible values for this property. Select the desired value and the control's text will be updated.



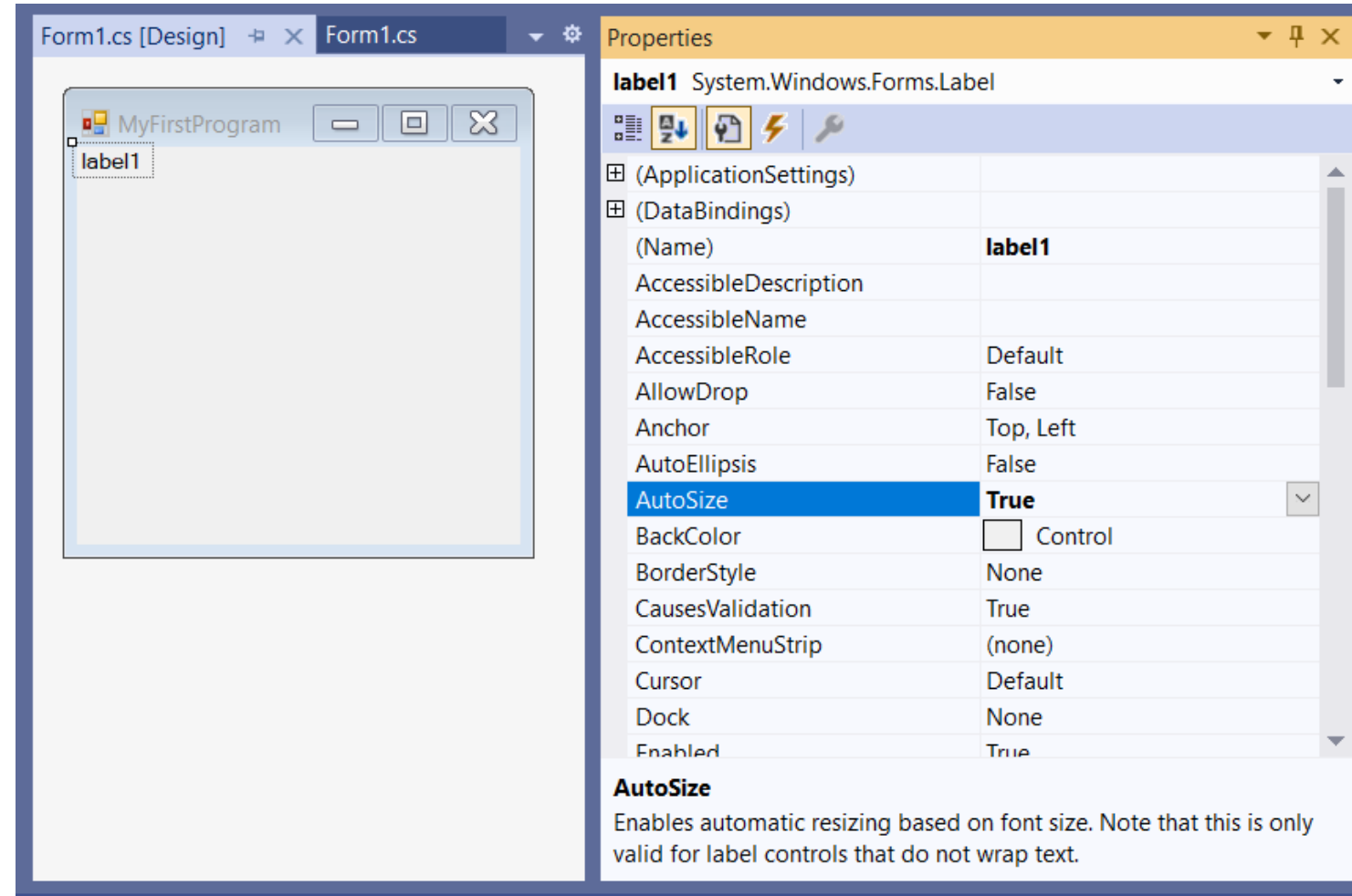
Label Control (BorderStyle Property)

- This figure shows an example of Label controls with each BorderStyle setting.
- The image on the left in the figure shows the form in the Designer, and the image on the right shows the form at runtime.



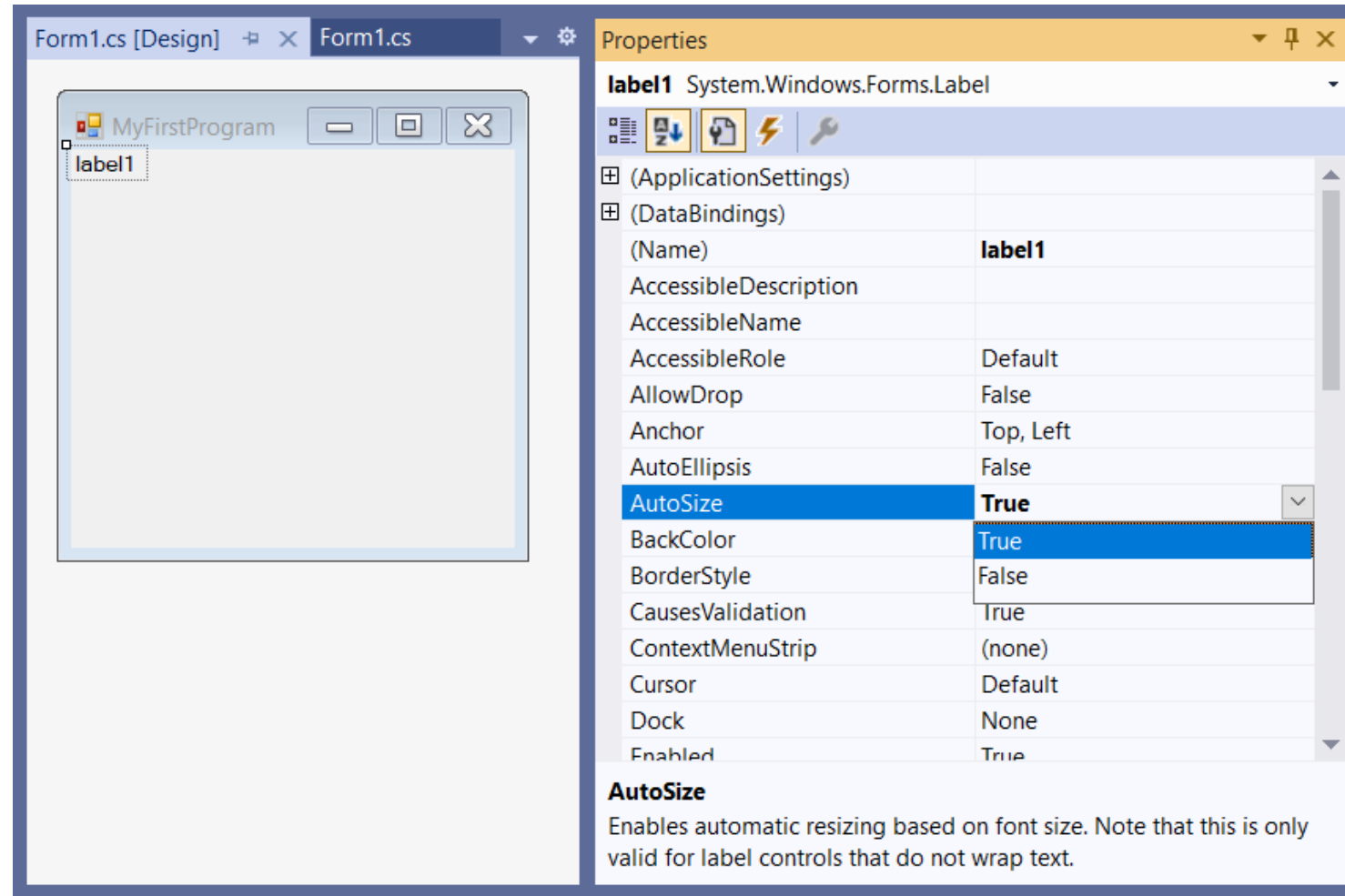
Label Control (AutoSize Property)

- Label controls have an AutoSize property that controls the way they can be resized.
- The AutoSize property is a Boolean property, which means that it can be set to one of two possible values: **True** or **False**.
- By default, a Label control's AutoSize property is set to **True**, which means that the control automatically resizes itself to accommodate the size of the text it displays.



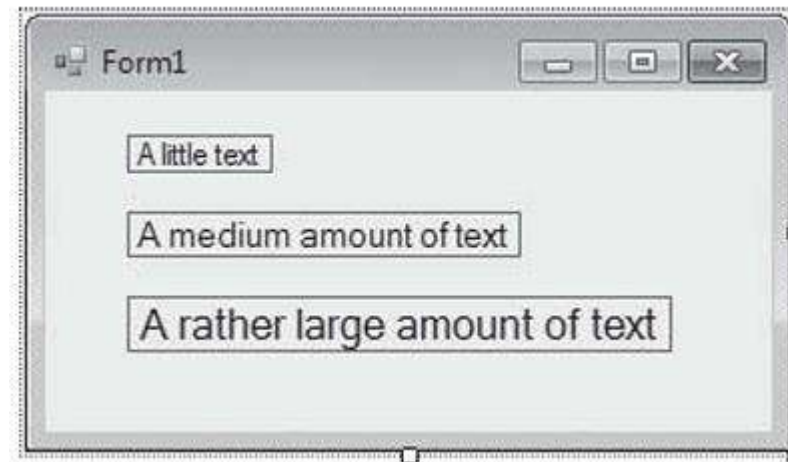
Label Control (AutoSize Property)

- When a Label control's AutoSize property is set to True, you cannot manually change the size of the control by clicking and dragging its bounding box.
- If you want to manually change the size of a Label control, you have to set its AutoSize property to False.
- When AutoSize is set to **False**, sizing handles will appear around the control, allowing you to click and drag the bounding box to resize the control.



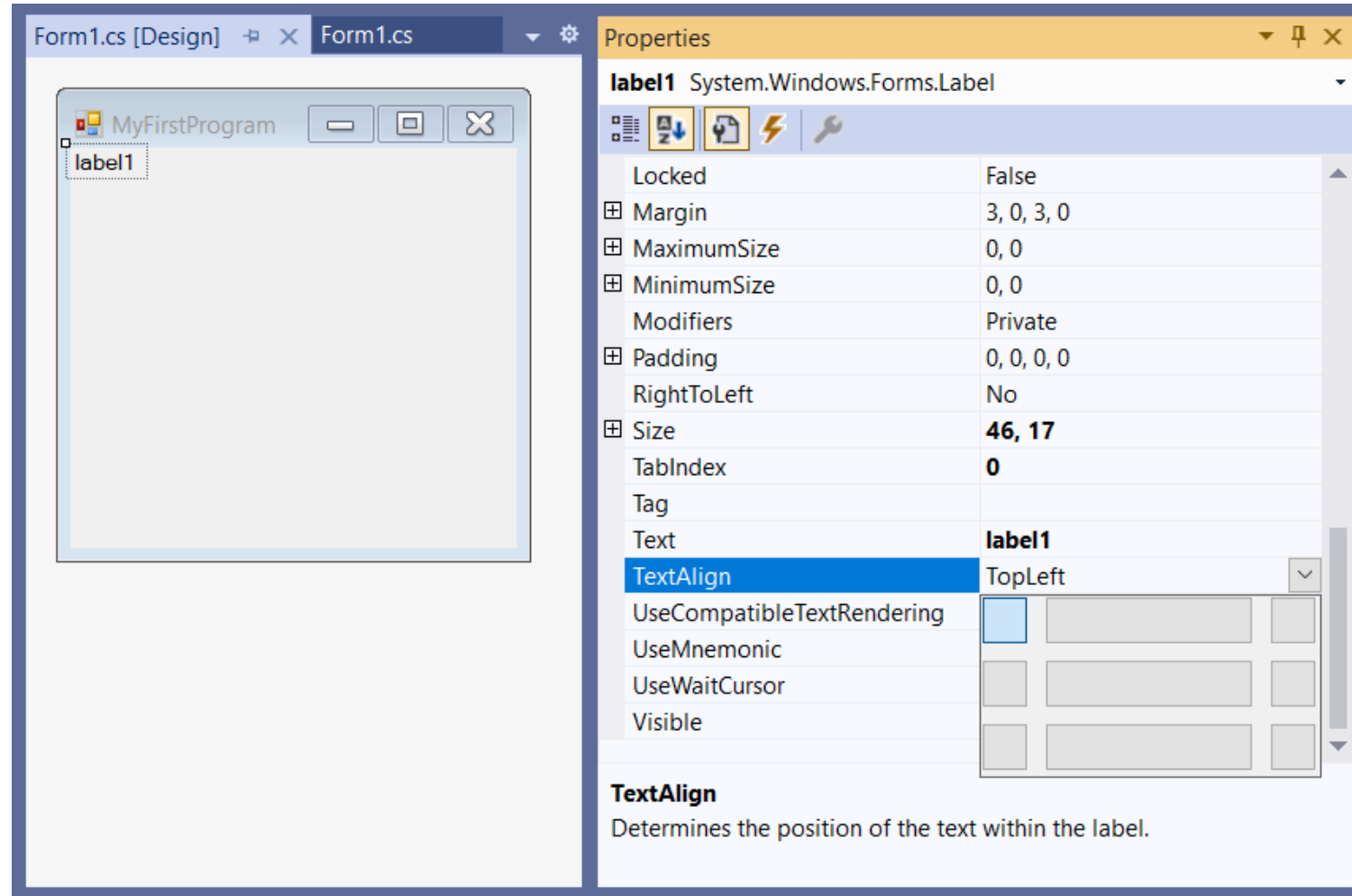
Label Control (AutoSize Property)

- For example, look at the three Label controls in the figure. Each of the controls displays different amounts of text at different font sizes. Because each control's `BorderStyle` property is set to `FixedSingle`, you can see that each control is just large enough to accommodate its text.



Label Control (TextAlign Property)

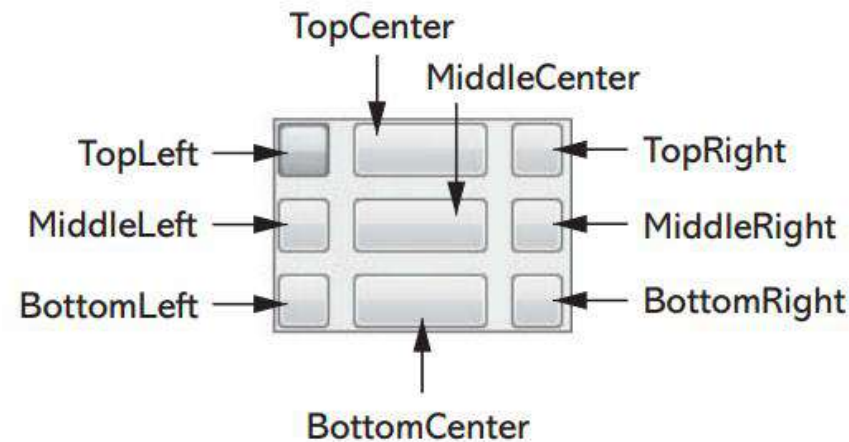
- When you set a Label control's AutoSize property to False and then manually resize the control, it sometimes becomes necessary to change the way the label's text is aligned.
- By default, a label's text is aligned with the top and left edges of the label's bounding box.



Label Control (TextAlign Property)

■ The TextAlign property may be set to any of the following values:

- TopLeft
- TopCenter
- TopRight
- MiddleLeft
- MiddleCenter
- MiddleRight
- BottomLeft
- BottomCenter
- BottomRight



Label Control (Text Property)

- The Text Property Accepts Strings Only. Assume that an application has a Label control named resultLabel. The following statement will cause an error because it is attempting to store the number 5 in the resultLabel control's Text property:

```
resultLabel.Text = 5;
```

ERROR!

- However, this does not mean that you cannot display a number in a label. To display a number in a label, put quotation marks around the number to become a string. The following statement will work:

```
resultLabel.Text = "5";
```

- If you want to clear the text that is displayed in a Label control, simply assign an empty string (" ") to the control's Text property, as shown here:

```
resultLabel.Text = " ";
```

Display Output in a Label Control

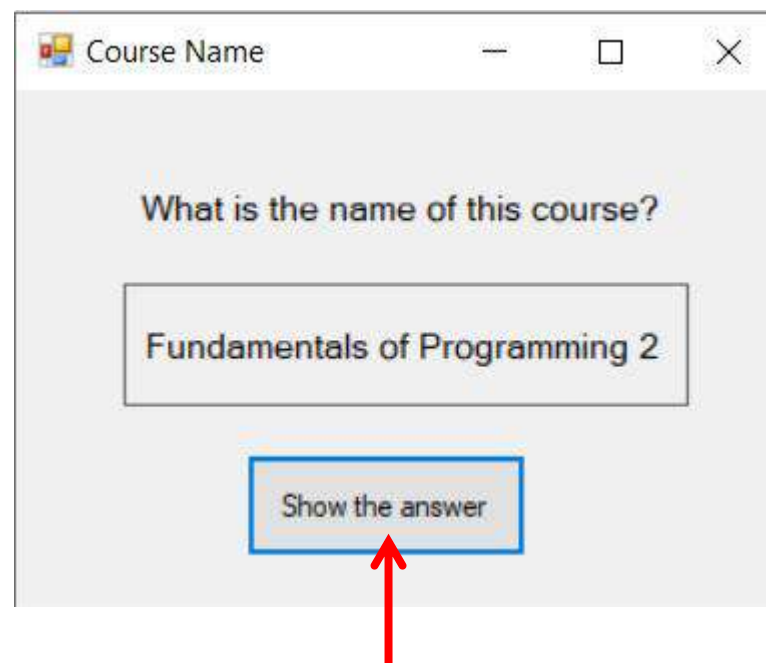
- Suppose you are creating an application that performs a calculation and you want to display the result of the calculation at a specific location on the form.
- The general steps that you would follow:
 - **Part 1:** Creating the application's GUI
 - 1) Place a Label control on the form at the location where you want the result to be displayed.
 - 2) In the Properties window, erase the contents of the Label control's Text property. (the control will not initially display anything when the application runs).
 - **Part 2:** Writing the application's code
 - 1) Write the necessary statements to perform the calculation and then store the result of the calculation in the Label control's Text property.

Example 1

- Create a GUI application that asks for the name of this course. When the user clicks a button, the answer to the question is displayed on the form.



A screenshot of a GUI window titled "Course Name". The window contains the text "What is the name of this course?" followed by an empty rectangular input field. Below the input field is a button labeled "Show the answer".

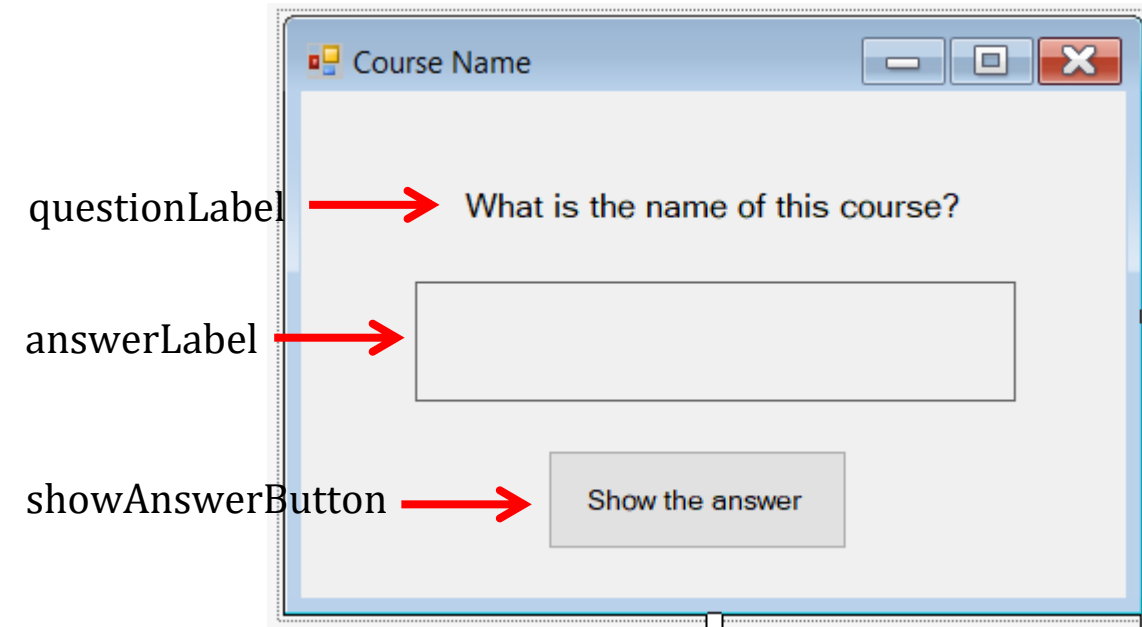


A screenshot of the same GUI window titled "Course Name". The input field now contains the text "Fundamentals of Programming 2". A red arrow points to the "Show the answer" button.

Example 1

1) Insert three controls into the form, as shown in the figure below:

- A Label control named questionLabel. This label displays the question.
- A Label control named answerLabel. This label initially appears empty, but will be used to display the answer to the trivia question.
- A Button control named showAnswerButton. When the user clicks this button, the answer to the trivia question is displayed.



Example 1

- The following table lists the property settings for each control:

Control Name	Control Type	Property Settings
questionLabel	Label	AutoSize: False BorderStyle: None Font: Microsoft Sans Serif (Style: Regular, Size: 10 point) Text: What is the name of this course? TextAlign: MiddleCenter
answerLabel	Label	AutoSize: False BorderStyle: FixedSingle Font: Microsoft Sans Serif (Style: Regular, Size: 10 point) Text: (The contents of the Text property have been erased.) TextAlign: MiddleCenter
showAnswerButton	Button	Size: 110, 23 Text: Show the Answer

Example 1

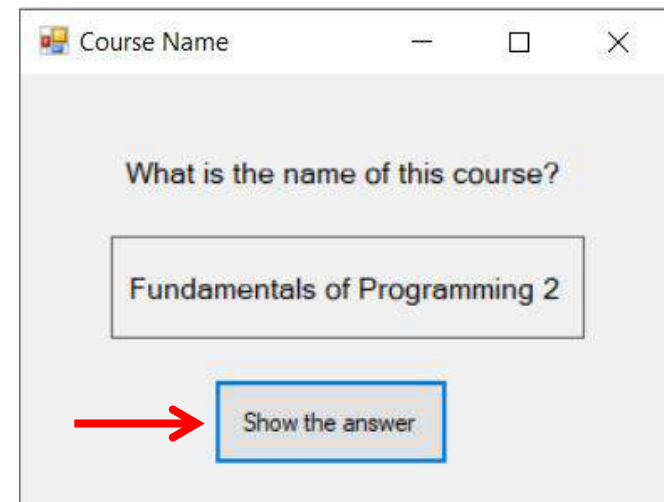
2) Inside the showAnswerButton_Click event handler, type the following statement:

answerLabel.Text = "Fundamentals of Programming 2");

3) Save the project.

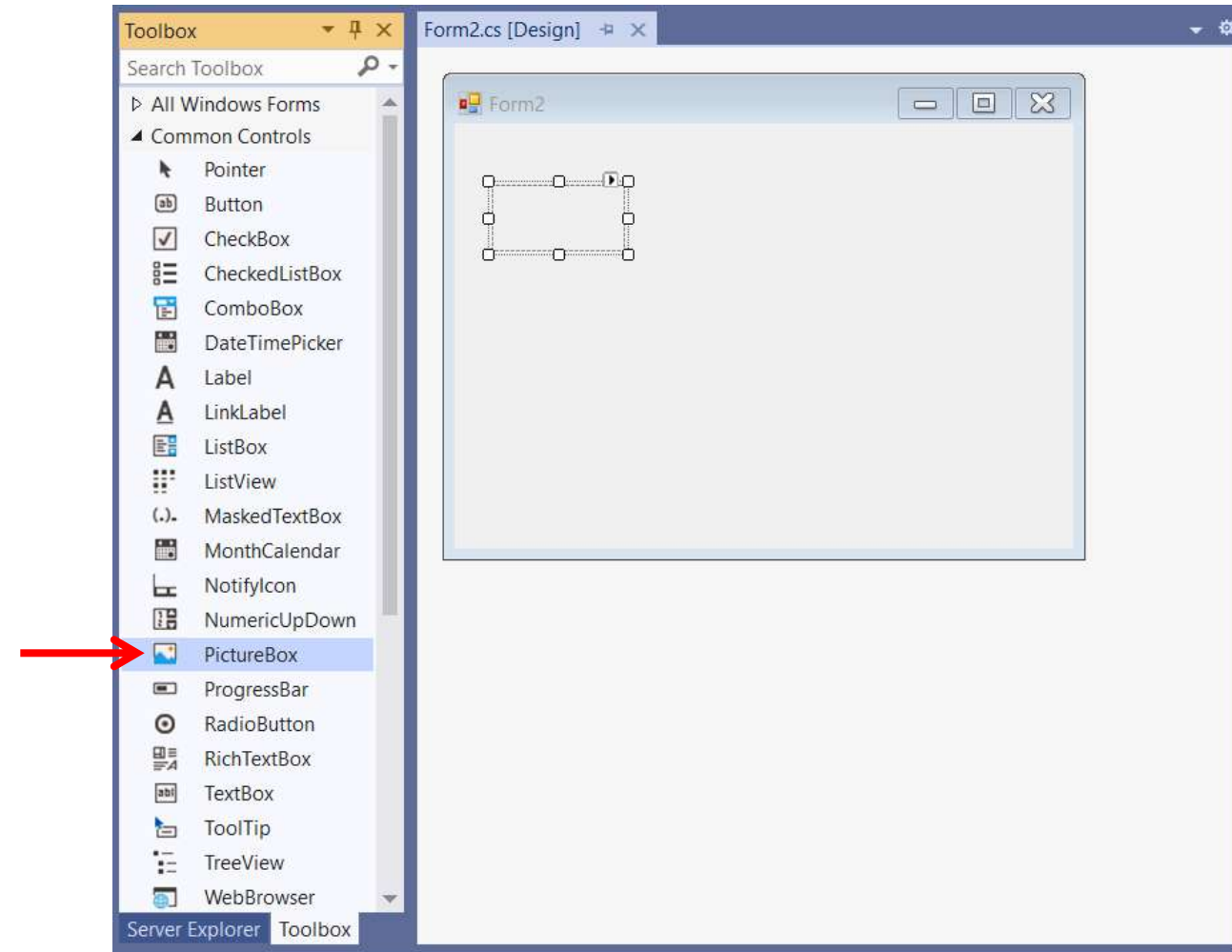
4) Press the F5 key on the keyboard, or click the Start Debugging button on the toolbar to compile and run the application. Click the "Show the answer" button to see the answer of the question.

```
0 references
public partial class Form1 : Form
{
    1 reference
    public Form1()
    {
        InitializeComponent();
    }
    1 reference
    private void showAnswerButton_Click(object sender, EventArgs e)
    {
        answerLabel.Text = "Fundamentals of Programming 2";
    }
}
```



PictureBox Control

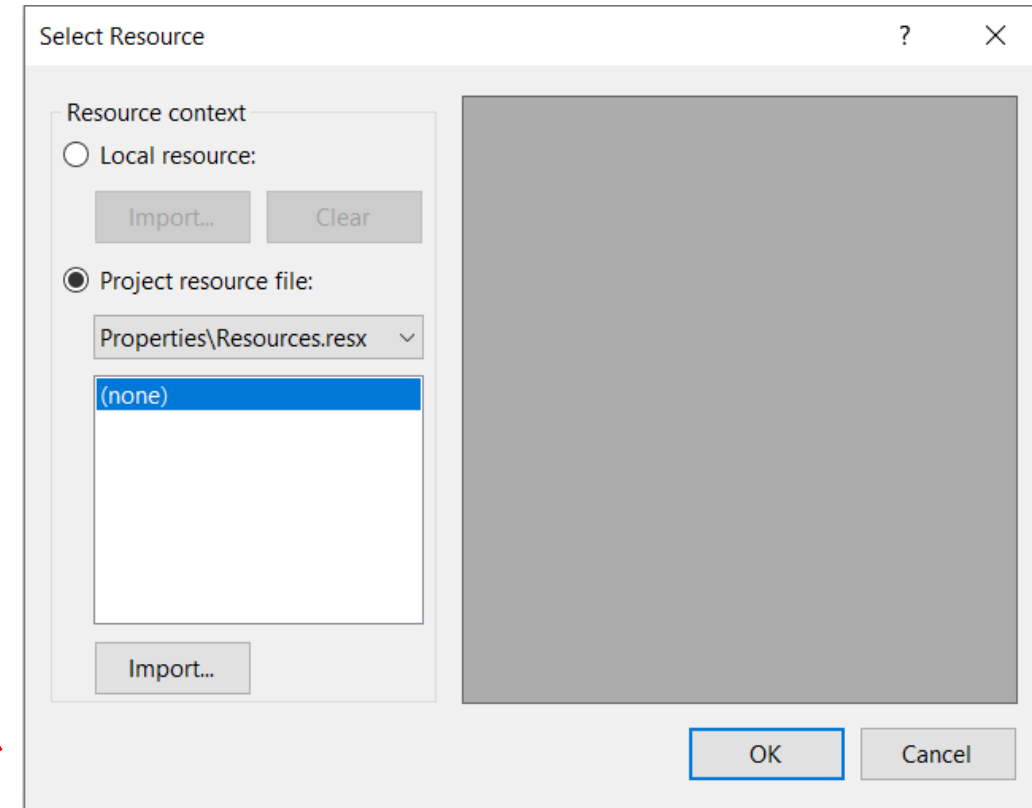
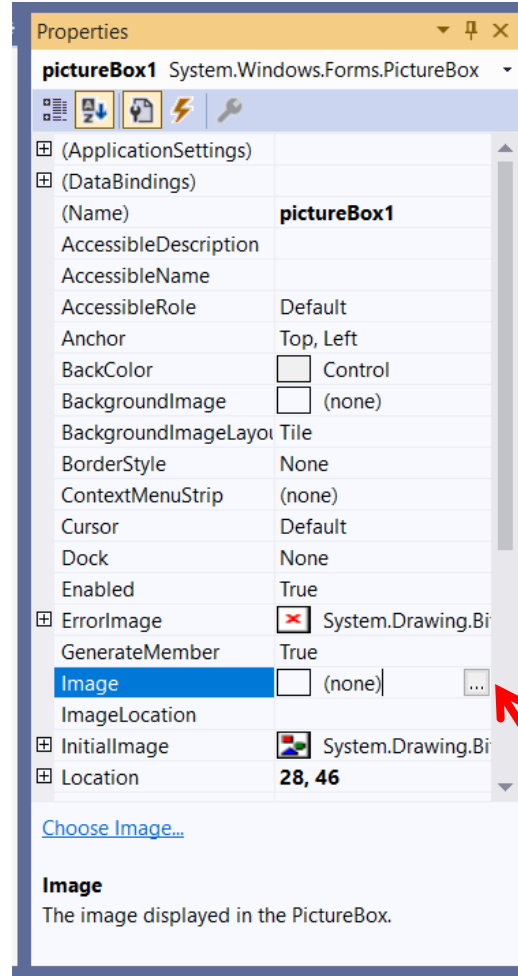
- A PictureBox control displays a graphic image on a form.
- A PictureBox control can have a Click event handler that responds when the user clicks the control at run time.
- In the Toolbox, the PictureBox tool is located in the Common Controls group. When you double-click the tool, an empty PictureBox control is created on the form, as shown in the figure.



PictureBox Control

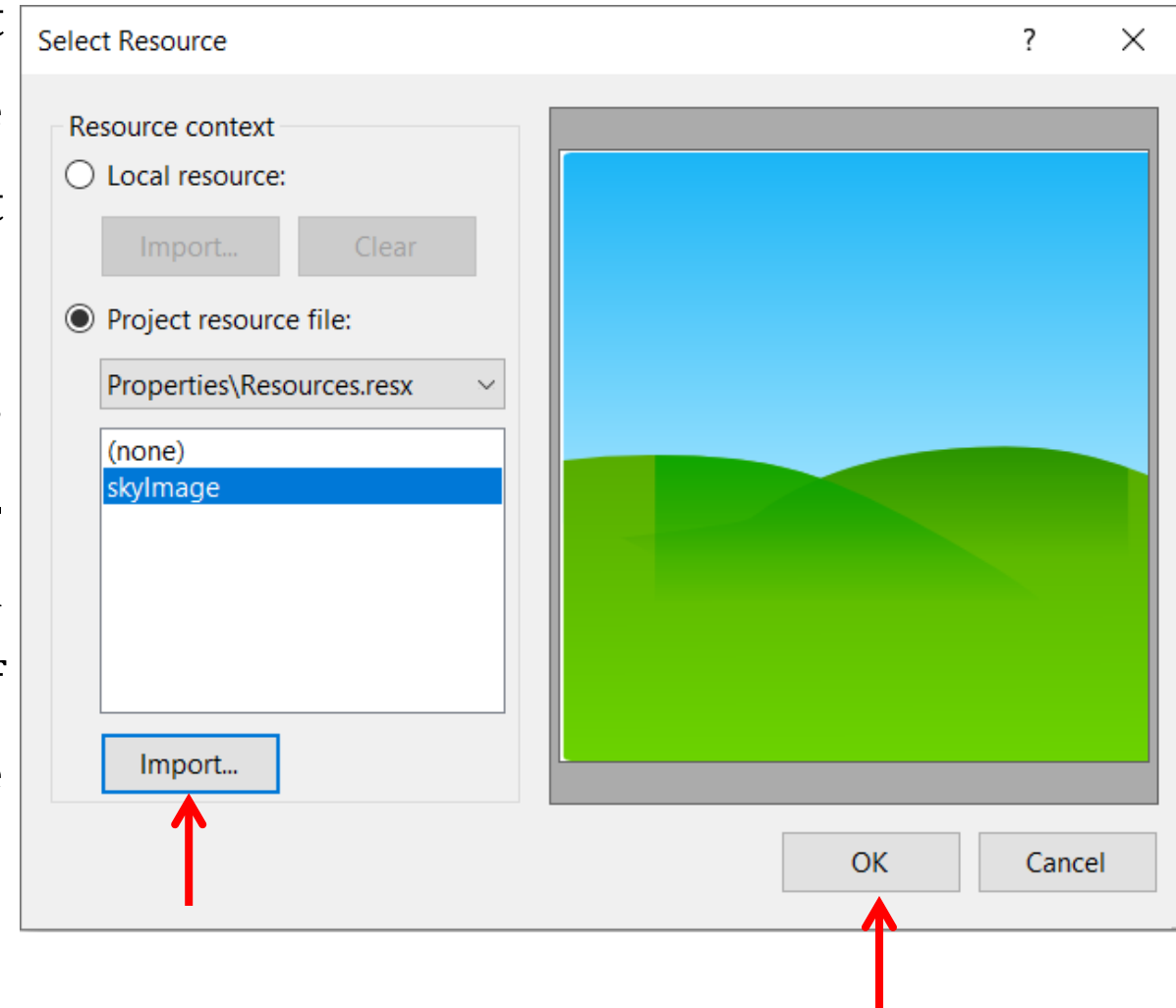
- Once you have created a PictureBox control, use its Image property to specify the image that it will display. Follow these steps:

- 1) Click the Image property in the Properties window.
- 2) Click the ellipses button (⋮) Select Resource window will appear.



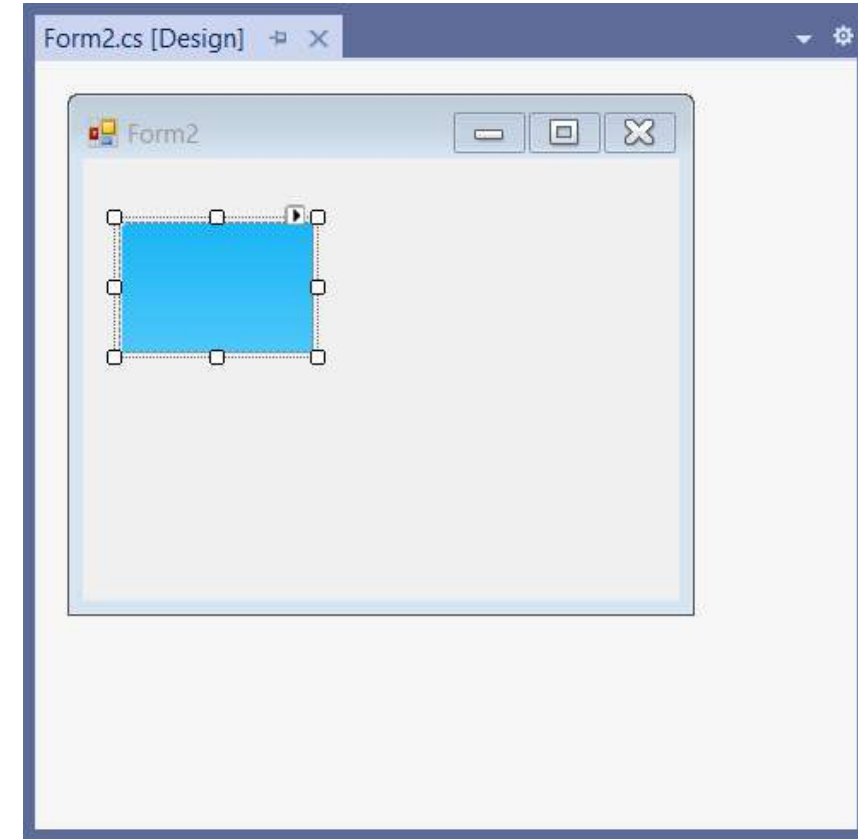
PictureBox Control

- 3) In the Select Resource window, click the Import button. An Open dialog box will appear. Use the dialog box to locate and select the image file that you want to display.
- 4) After you select an image file, you will see its contents displayed in the Select Resource window. This indicates that the image has been imported into the project. This figure shows an example of the Select Resource window after we have selected and imported an image.
- 5) Click the OK button.



PictureBox Control

- The selected image will appear in the PictureBox control, as shown in the figure.
- Depending on the size of the image, you might see only part of it displayed. This is the case in here because the image is larger than the PictureBox control.
- The next step is to set the `SizeMode` property and adjust the size of the control.



PictureBox Control (SizeMode Property)

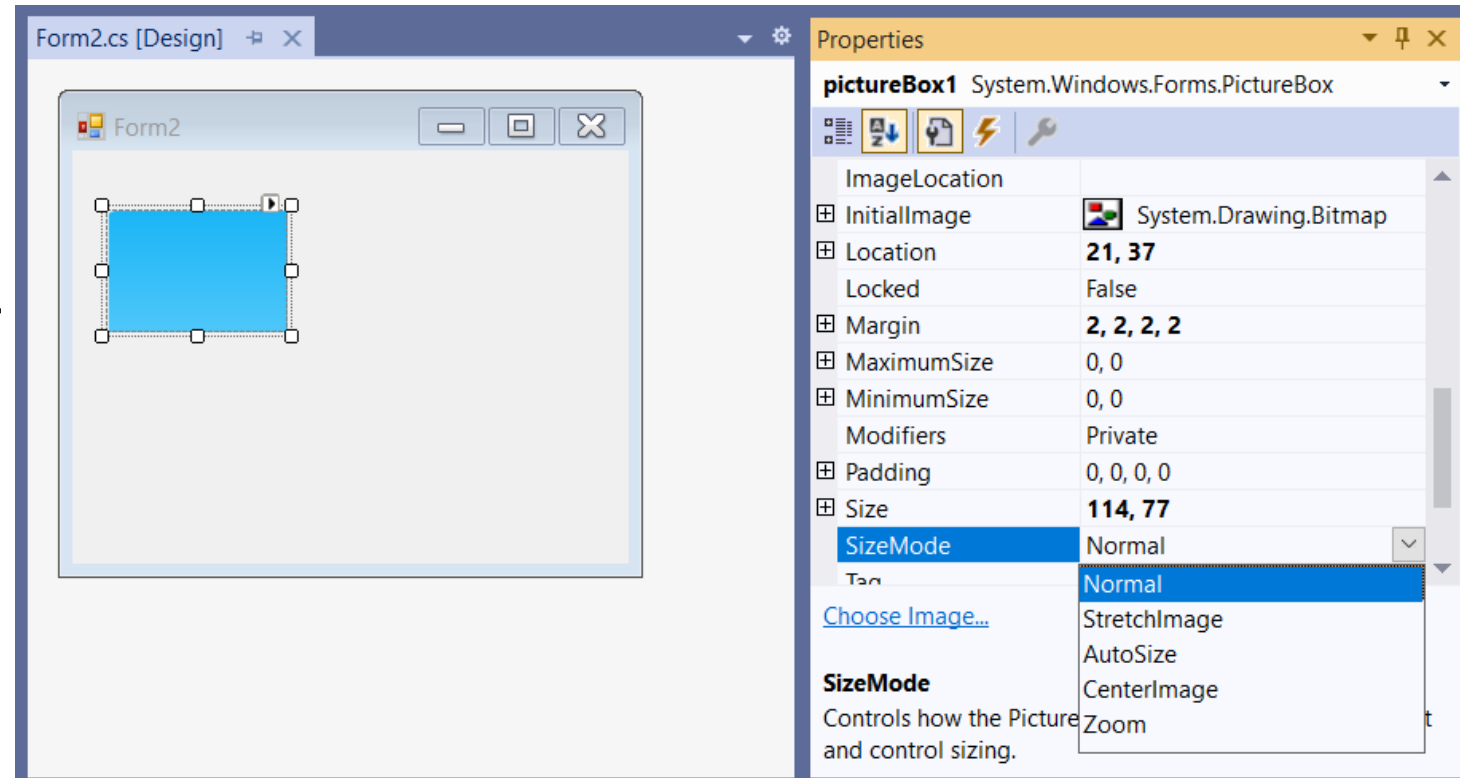
■ The PictureBox control's SizeMode property specifies how the control's image is to be displayed.

■ It can be set to one of the following values:

- **Normal:** This is the default value. The image will be positioned in the upper-left corner of the PictureBox control. If the image is too big to fit in the PictureBox control, it will be clipped.

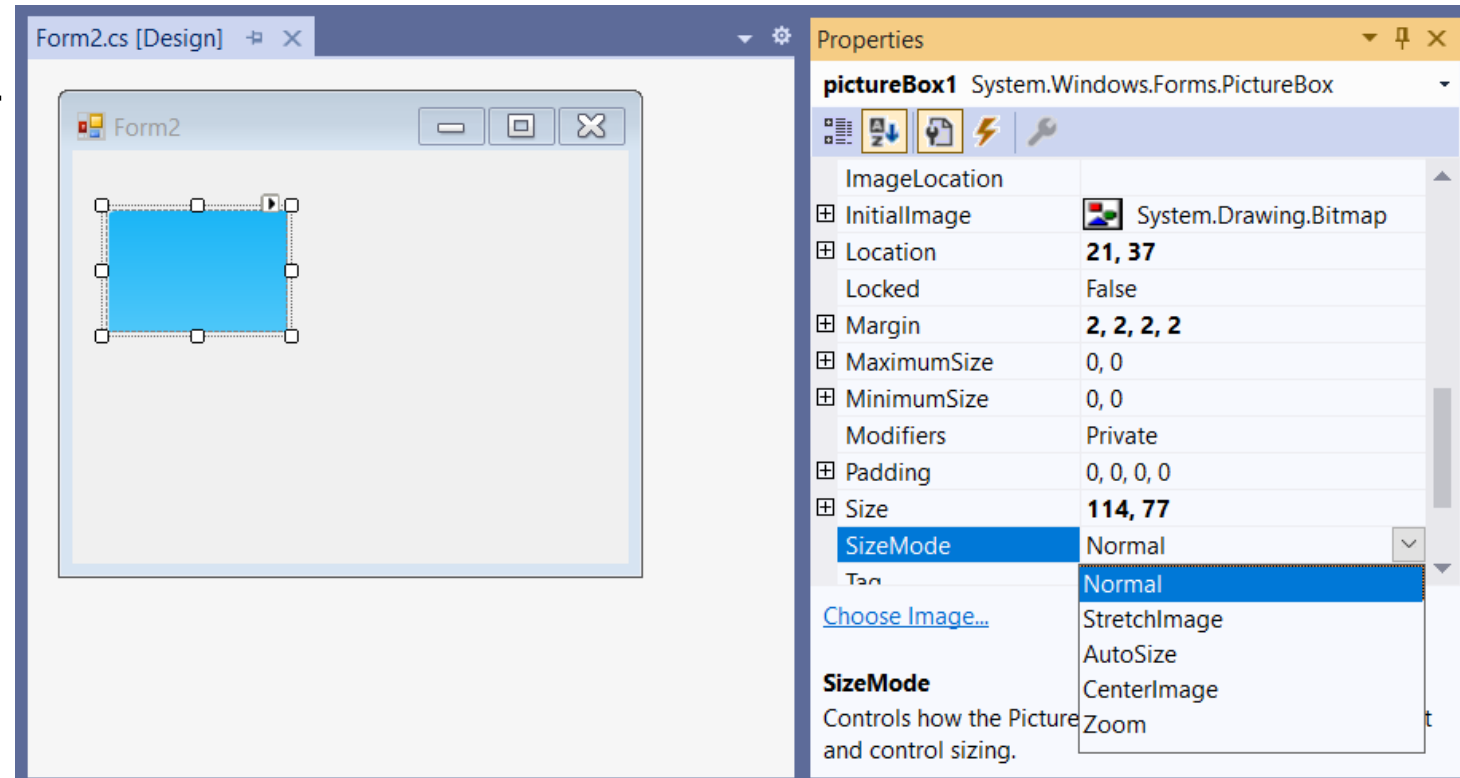
- **StretchImage:** It resizes the image both horizontally and vertically to fit in

the PictureBox control. If the image is resized more in one direction than the other, it will appear stretched.



PictureBox Control (SizeMode Property)

- **AutoSize:** The PictureBox control is automatically resized to fit the size of the image.
- **CenterImage:** It centers the image in the PictureBox control without resizing it.
- **Zoom:** It uniformly resizes the image to fit in the PictureBox without losing its original aspect ratio. (Aspect ratio is the image's width to height ratio.) This causes the image to be resized without appearing stretched.

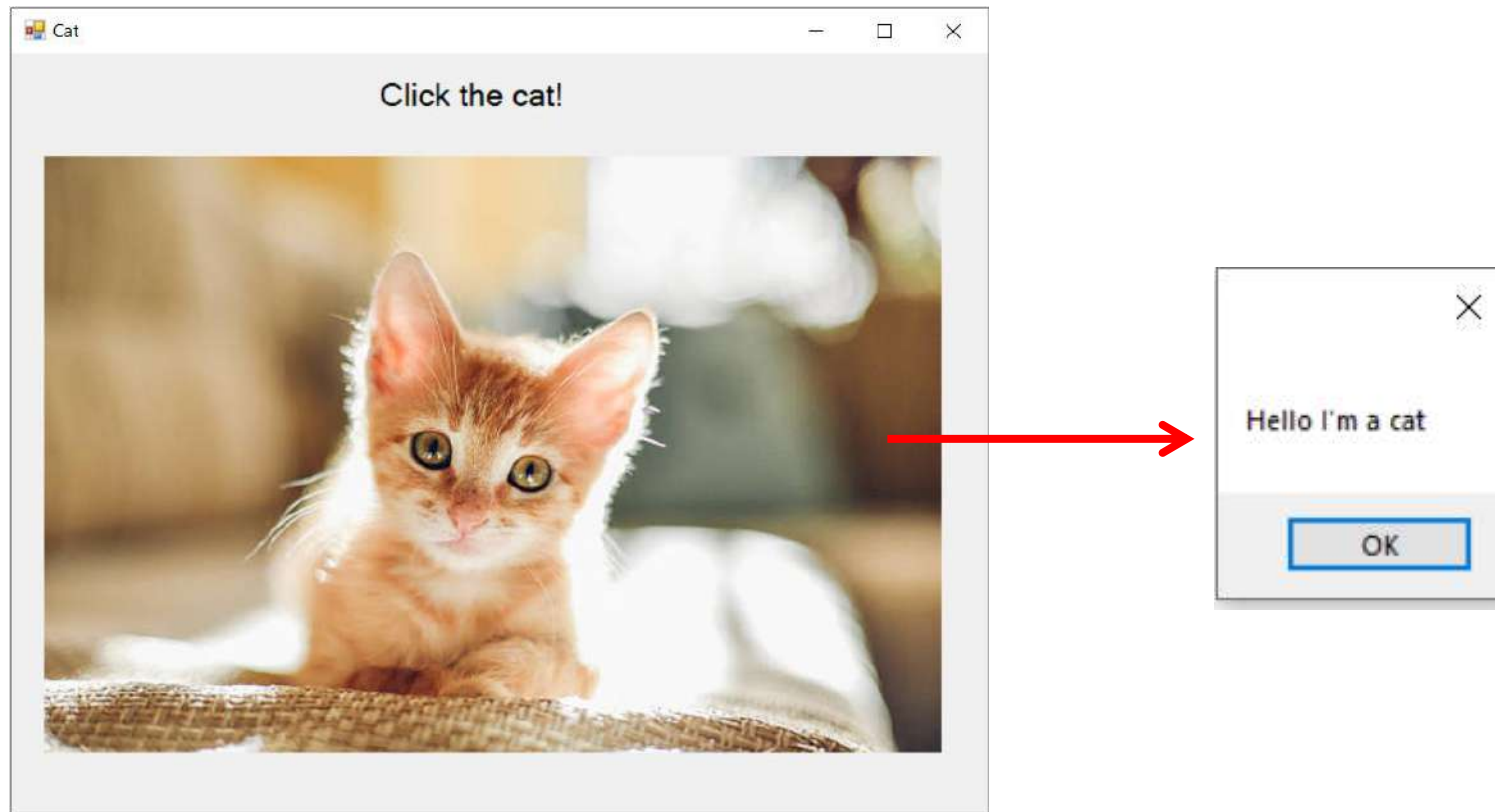


Creating Clickable Images

- Buttons aren't the only controls that can respond to Click events.
- PictureBox controls can, too. That means an application can display an image and perform some action when the user clicks the image.
- To make an image clickable, you simply have to create a Click event handler for the PictureBox control that displays the image:
 - 1) Double-click the PictureBox control in the Designer. This creates an empty Click event handler in the form's source code file.
 - 2) In the code editor, write statements inside the event handler that you want to execute when the image is clicked.

Example 2

- Create a GUI application that contains a PictureBox control. When the user clicks this control, a message box will appear displaying the string “Hello I’m a cat”.



Example 2

- 1) Add a PictureBox control into the form. Change its name to catPictureBox. Set the SizeMode property to Zoom and the BorderStyle property to FixedSingle.
- 2) Double-click the catPictureBox control in the Designer. This will automatically open the Form1.cs file in the code editor and create a Click event handler for the catPictureBox control.
- 3) Write the statement that you want to execute inside the catPictureBox event handler, as shown on the right.
- 4) Run the application and click the PictureBox, a message box will appear displaying the string “Hello I’m a cat”.

```
namespace MyFirstProject
{
    3 references
    public partial class Form2 : Form
    {
        1 reference
        public Form2()
        {
            InitializeComponent();
        }

        1 reference
        private void catPictureBox_Click(object sender, EventArgs e)
        {
            MessageBox.Show("Hello I'm a cat");
        }
    }
}
```

Example 3

- Create a GUI application that displays the flags of Egypt, UAE, and Palestine in PictureBox controls. When the user clicks any of these flags, the name of that flag's country will appear in a Label control.



Example 3

1) Set up the application's form as shown below.



Example 3

2) Set up the properties of the controls as shown in the following table.

Control Name	Control Type	Property Settings
instructionLabel	Label	Text: Click a flag to see the name of the country
egyptPictureBox	PictureBox	Image: Select and import Egypt's flag image BorderStyle: FixedSingle SizeMode: Zoom
palestinePictureBox	PictureBox	Image: Select and import Palestine's flag image BorderStyle: FixedSingle SizeMode: Zoom
UAEPictureBox	PictureBox	Image: Select and import UAE's flag image BorderStyle: FixedSingle SizeMode: Zoom
countryLabel	Label	AutoSize: False BorderStyle: FixedSingle Font: Microsoft Sans Serif (Style: Regular, Size: 16 point) Text: (The contents of the Text property have been erased.) TextAlign: MiddleCenter

Example 3

- 3) Write the statement that you want to execute inside the PictureBox event handler controls, as shown on the right.
- 4) Run the application and click any of these PictureBox controls to see the output.

```
1 reference  
private void egyptPictureBox_Click(object sender, EventArgs e)  
{  
    countryLabel1.Text = "Egypt";  
}
```

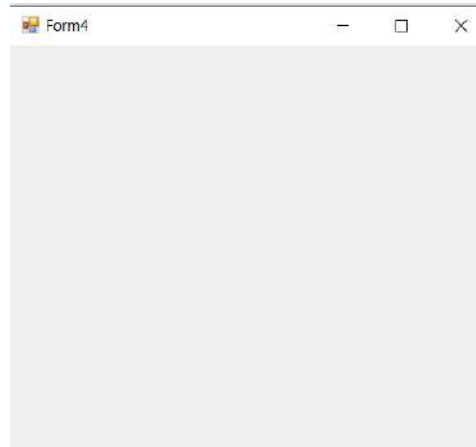
```
1 reference  
private void palestinePictureBox_Click(object sender, EventArgs e)  
{  
    countryLabel1.Text = "Palestine";  
}
```

```
1 reference  
private void UAEPictureBox_Click(object sender, EventArgs e)  
{  
    countryLabel1.Text = "UAE";  
}
```

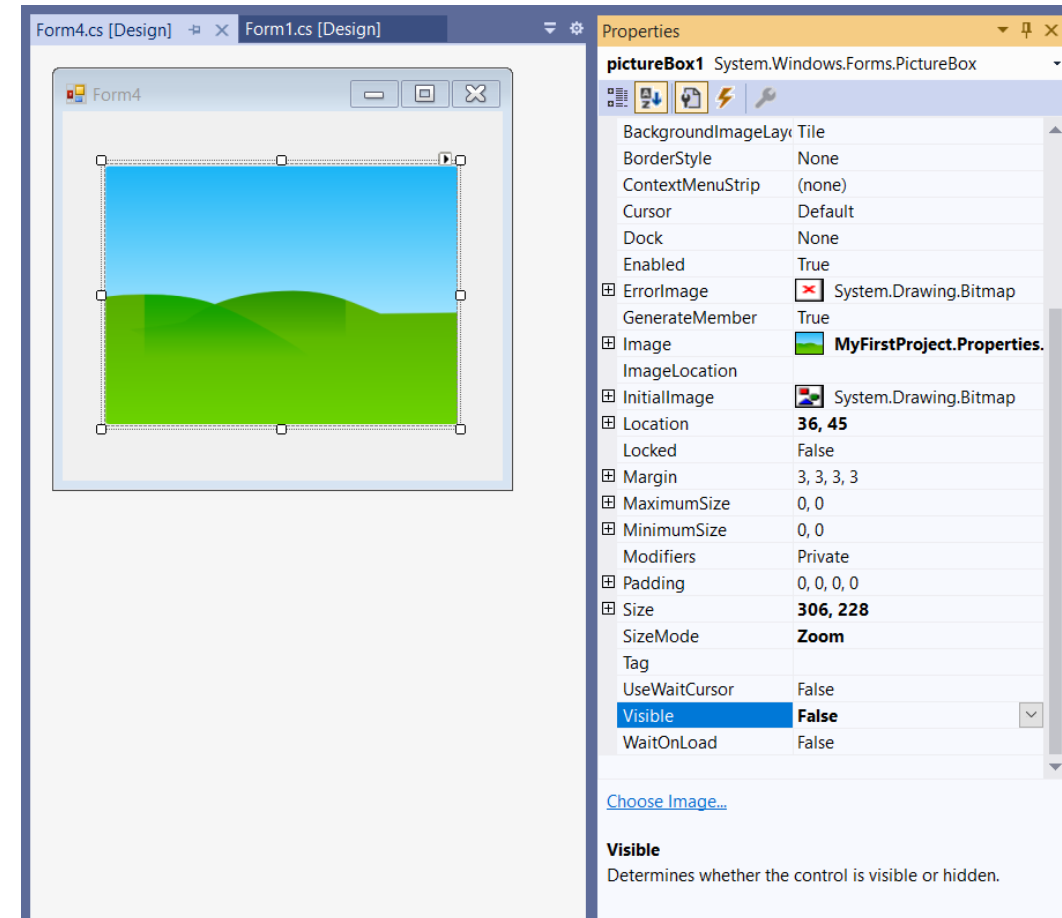
PictureBox Control (Visible Property)

- Most controls have a Visible property that determines whether the control is visible on the form at run time.
- The Visible property is a Boolean property, which means it can be set only to the values **True** or **False**.
- When you use the Properties window to change a control's Visible property at design time, the control will still be visible in the Designer. When you run the application, the control will not be visible on the form, as shown in the figure.

The form at run time



The form in the Designer



PictureBox Control (Visible Property)

- A control's Visible property can also be modified in code by an assignment statement, which makes it possible to hide or display a control while the application is running.
- For example, the PictureBox control shown in the previous figure is named skyPictureBox.
- The following statement sets the control's Visible property to true:

```
skyPictureBox.Visible = true;
```

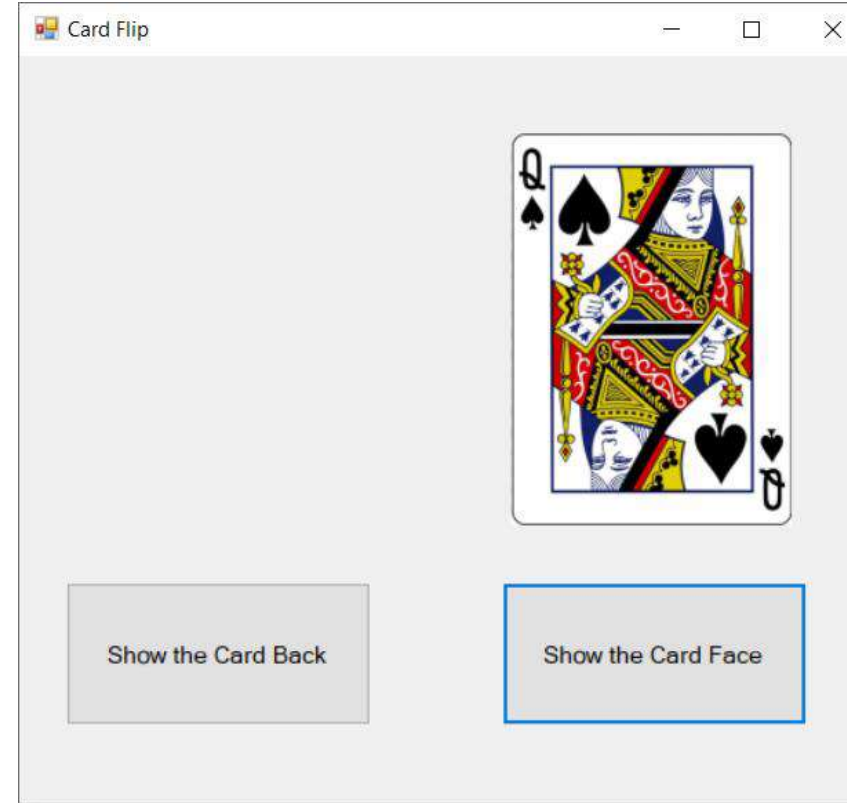
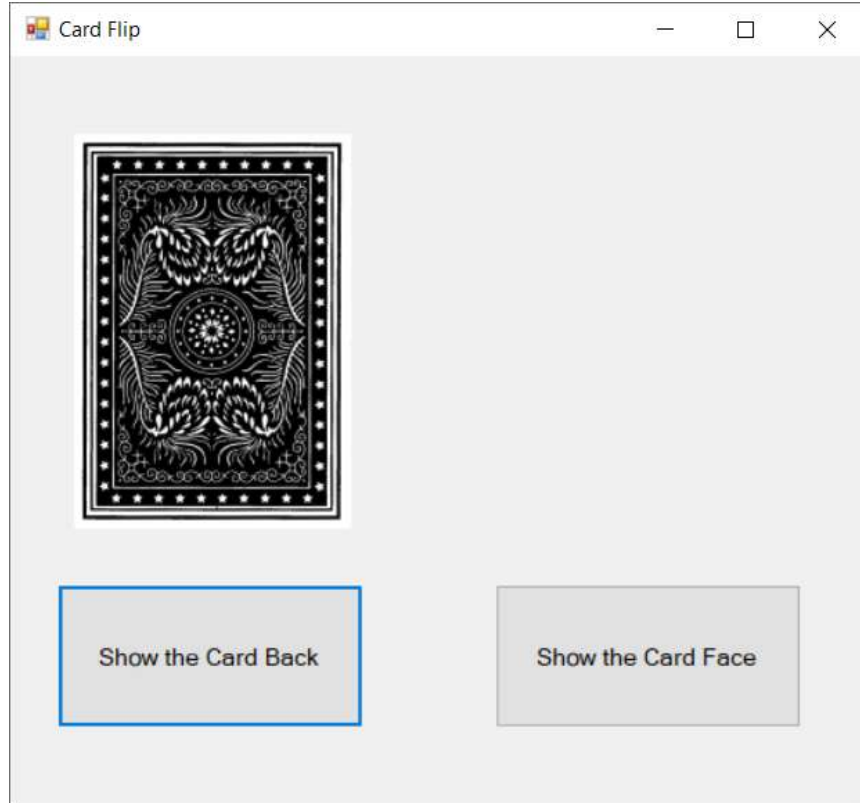
- When this statement executes, the skyPictureBox control will become visible.
- Like wise, the following statement sets the control's Visible property to false:

```
skyPictureBox.Visible = false;
```

- When this statement executes, the skyPictureBox control will become invisible.

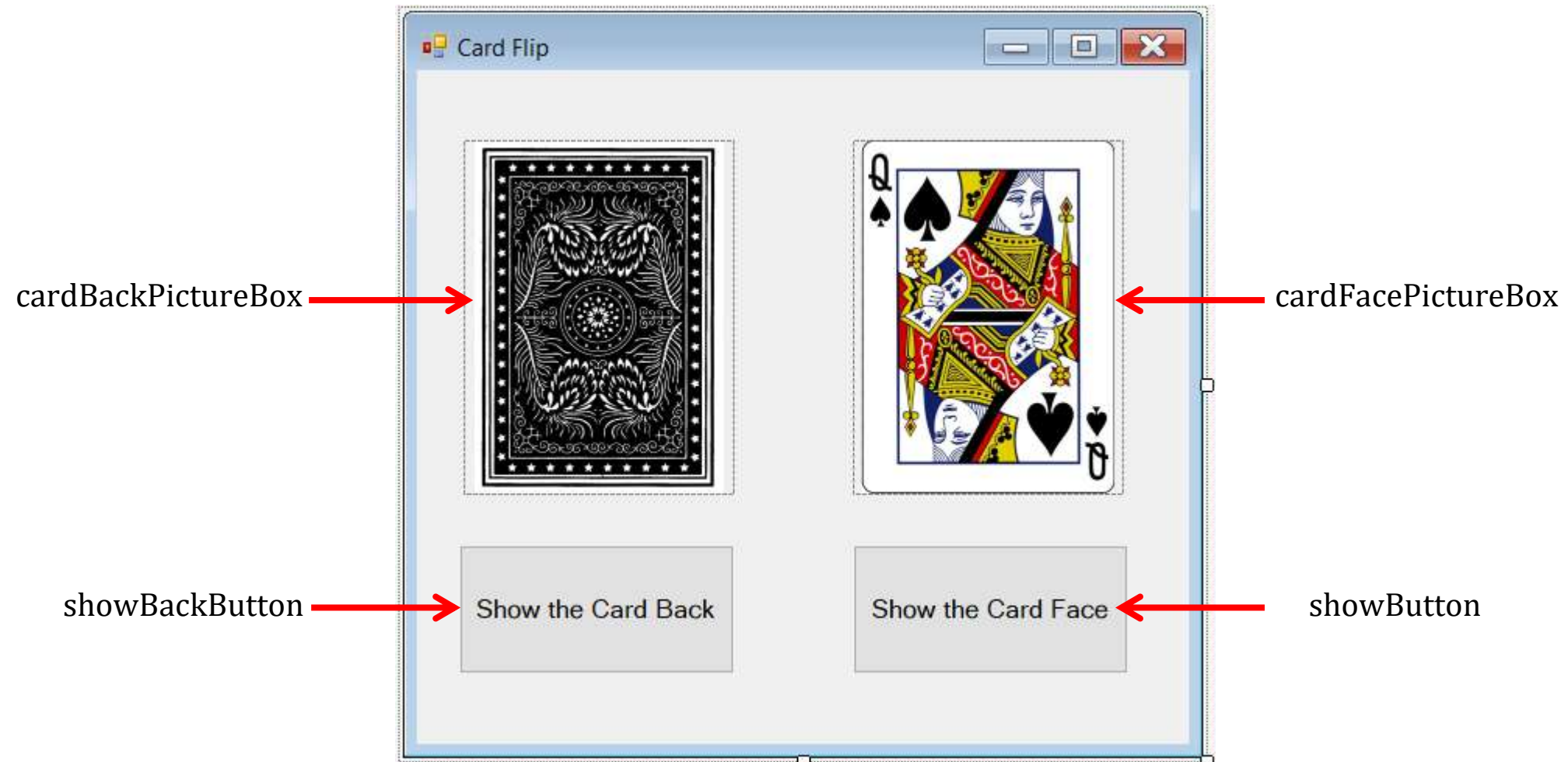
Example 4

- Create a GUI application that simulates a card being flipped over. When the application runs, the form will initially display the back of a poker card, as shown on the left Figure. When the user clicks the “Show the Card Face” button, the card will be flipped over to show its face, as shown on the right.



Example 4

- 1) Set up the application's form as shown below.



Example 4

2) Set up the properties of the controls as shown in the following table.

Control Name	Control Type	Property Settings
cardBackPictureBox	PictureBox	Image: Select and import the back of a poker card image Size: 100, 140 SizeMode: Zoom Visible: True
cardFacePictureBox	PictureBox	Image: Select and import the face of a poker card image BorderStyle: FixedSingle SizeMode: Zoom Visible: False
showBackButton	Button	Text: Show the Card Back Size: Manually resize the button to accommodate the text
showFaceButton	Button	Text: Show the Card Face Size: Manually resize the button to accommodate the text

Example 4

- 3) Write the statement that you want to execute inside the event handler of Button controls, as shown on the right.
- 4) Run the application and click the buttons to view the output.

```
public partial class Form4 : Form
{
    1 reference
    public Form4()
    {
        InitializeComponent();
    }

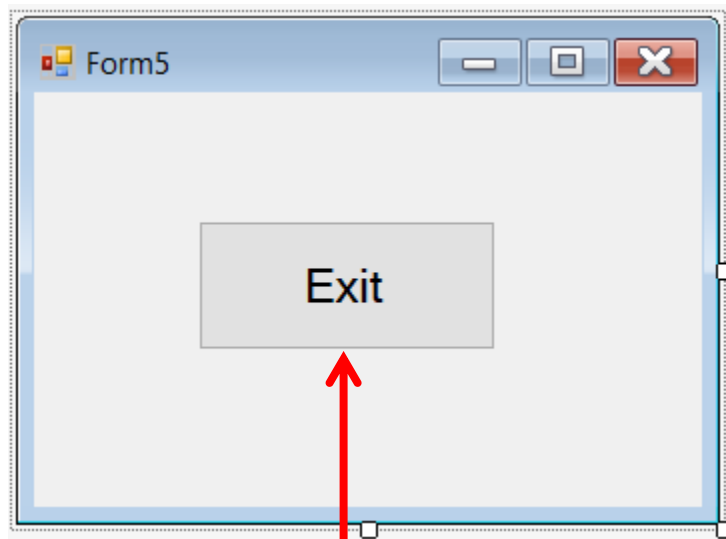
    1 reference
    private void showFaceButton_Click(object sender, EventArgs e)
    {
        cardBackPictureBox.Visible = false;
        cardFacePictureBox.Visible = true;
    }

    1 reference
    private void showBackButton_Click(object sender, EventArgs e)
    {
        cardBackPictureBox.Visible = true;
        cardFacePictureBox.Visible = false;
    }
}
```

Close an Application's Form

- All the applications that are previously created require the user to click the standard Windows close button (X) to close the application.
- To close an application's form in code, use the statement:

```
this.Close();
```



exitButton

```
namespace MyFirstProject
{
    2 references
    public partial class Form5 : Form
    {
        0 references
        public Form5()
        {
            InitializeComponent();
        }

        1 reference
        private void exitButton_Click(object sender, EventArgs e)
        {
            this.Close();
        }
    }
}
```

Comments

- There are two common types of comments:
 - **Single-line comments:** appears on one line in a program. It starts with `//` and can be placed anywhere in the line.
 - **Block comments:** can occupy multiple consecutive lines in a program. It starts with `/*` and ends with `*/`

THANK YOU